APUNTES DEL CURSO

Estadística básica con R para la investigación en ELE

De los datos del aula a las pruebas estadísticas



Adrián Cabedo Nebot
Universitat de València
adrian.cabedo@uv.es

Máster en Español como Lengua Extranjera
Universitat de Barcelona
23 de octubre de 2026

Índice

Cómo usar estos apuntes . . . 5
¿Por qué estadística en un máster de ELE? . . . 5
Prepara tu espacio de trabajo . . . 6
Nuestros datos: 240 aprendientes de ELE . . . 6
Las variables . . . 7
1 R y estadística descriptiva . . . 8
1.1 Población y muestra: la nota media del DELE . . . 8
1.2 Tipos de variables . . . 9
1.3 R y RStudio . . . 9
1.4 R como calculadora y objetos . . . 10
1.5 Vectores y funciones . . . 10
1.6 Cargar nuestros datos . . . 11
1.7 Tendencia central . . . 11
1.8 La media persigue a los extremos . . . 12
1.9 Cómo se calcula cada medida . . . 13
1.10 Dispersión: ¿cuánto varían los datos? . . . 13
1.11 Medidas de dispersión . . . 14
1.12 La desviación típica, paso a paso . . . 15
1.13 Cuartiles y percentiles . . . 16
1.14 `summary()` y frecuencias . . . 17
1.15 Resumir por grupos con `dplyr` . . . 17
1.16 Explora cualquier resumen por grupos . . . 18
1.17 Filtrar, seleccionar y crear columnas . . . 19
1.18 Práctica 1 (20 min) . . . 19
1.19 Práctica 1: soluciones . . . 20
2 Visualización de datos . . . 21
2.1 ¿Por qué visualizar? El cuarteto de Anscombe . . . 21
2.2 La gramática de gráficos, capa a capa . . . 21
2.3 Histograma: una variable numérica . . . 22
2.4 ¿Qué anchura de barra? . . . 24
2.5 Distribuciones por grupos . . . 24
2.6 Puntos, cajas y violines . . . 27
2.7 Barras, dispersión y facetas . . . 27
2.8 Convierte tu gráfico en interactivo . . . 29
2.9 Tu resultado: un gráfico que se puede explorar . . . 30
2.10 Motion chart: los datos en movimiento . . . 31
2.11 Tu resultado: el motion chart . . . 32
2.12 Cómo leer el motion chart . . . 33
2.13 ¿Qué gráfico elijo? . . . 33
2.14 Mapas de calor . . . 34
2.15 Leer un mapa de calor . . . 36
2.16 Buenas prácticas visuales . . . 37
2.17 Práctica 2 (20 min) . . . 37
2.18 Práctica 2: soluciones . . . 37
3 Estadística inferencial . . . 39
3.1 Descriptiva frente a inferencial . . . 39
3.2 La distribución normal . . . 39
3.3 ¿Son normales mis datos? . . . 40
3.4 De la muestra a la población . . . 42

3.5 Error estándar e intervalo de confianza ... 43
3.6 Qué significa un IC del 95 % ... 44
3.7 La lógica del contraste de hipótesis ... 45
3.8 El valor p, visualmente ... 46
3.9 Errores tipo I y II, y potencia ... 47
3.10 Tamaño del efecto: la nota DELE con y sin estancia ... 48
3.11 Árbol de decisión: ¿qué prueba uso? ... 49
3.12 Prueba t: ficha ... 50
3.13 La idea de t: señal frente a ruido ... 51
3.14 Prueba t: ¿influye la estancia en la nota? ... 52
3.15 Tu resultado, pieza a pieza ... 53
3.16 Muestras relacionadas y *d* de Cohen ... 54
3.17 Wilcoxon: comparar rangos ... 55
3.18 Wilcoxon en R ... 55
3.19 Correlación ... 56
3.20 Correlación en R ... 57
3.21 Práctica 3 (20 min) ... 58
3.22 Práctica 3: soluciones ... 58
4 Chi cuadrado y ANOVA ... 59
4.1 Tabla de contingencia ... 59
4.2 Chi cuadrado: ficha ... 59
4.3 Observadas, esperadas y residuos ... 60
4.4 χ^2 paso a paso ... 61
4.5 Tamaño del efecto y Fisher ... 62
4.6 Más categorías: residuos como mapa de calor ... 63
4.7 Cambiamos de pregunta: ¿difiere la nota según la L1? ... 64
4.8 ¿Por qué no hacer muchas pruebas t? ... 65
4.9 La idea de F ... 66
4.10 ANOVA: ficha ... 67
4.11 ANOVA de un factor ... 68
4.12 ¿Qué grupos difieren? Tukey ... 69
4.13 Tu resultado de Tukey, explicado ... 70
4.14 Tamaño del efecto y dos factores ... 71
4.15 ¿Qué es una interacción? ... 72
4.16 Kruskal-Wallis y resumen ... 73
4.17 Práctica 4 (20 min) ... 74
4.18 Práctica 4: soluciones ... 74
5 Árboles de decisión ... 76
5.1 Un árbol es un juego de preguntas ... 76
5.2 Anatomía de un árbol ... 77
5.3 Dos tipos de árbol ... 77
5.4 ¿Qué es una buena pregunta? La impureza ... 78
5.5 El mejor corte, en directo ... 79
5.6 El algoritmo en cuatro pasos ... 80
5.7 Construye tu primer árbol ... 80
5.8 Cómo leer `print(arbol)` ... 81
5.9 Dibújalo ... 82
5.10 Mira cómo crece ... 83
5.11 Explora las reglas ... 84
5.12 Sigue a un aprendiente por el árbol ... 84
5.13 Las reglas, en texto y en tabla ... 85
5.14 ¿Acierta? ... 85
5.15 El sobreajuste: la tabla de complejidad ... 86

5.16 Entrenamiento frente a validación cruzada ... 89
5.17 Podar el árbol ... 89
5.18 Ponerlo a prueba con datos nuevos ... 90
5.19 La matriz de confusión ... 91
5.20 ¿Qué variables importan? ... 91
5.21 Árbol de regresión: la nota ... 92
5.22 Predice la nota de un aprendiente ... 93
5.23 ¿Árbol o regresión logística? ... 94
5.24 Para saber más: bosques aleatorios ... 95
5.25 Cómo informar de un árbol ... 96
5.26 Práctica: árboles (20 min) ... 96
5.27 Práctica de árboles: soluciones ... 97
6 Plus: Estadística de corpus y modelos ... 98
6.1 Aprendientes frente a nativos ... 98
6.2 Log-likelihood (G^2): el caso de *pues* ... 98
6.3 LL y log-ratio en R ... 99
6.4 El mapa de palabras clave ... 100
6.5 Simple Maths: la fórmula de Sketch Engine ... 100
6.6 ¿Cómo cambia el ranking con *n*? ... 102
6.7 Con textos reales: `quanteda` ... 102
6.8 Segunda parte del Plus: de comparar a predecir ... 103
6.9 La idea de la regresión ... 104
6.10 Regresión lineal: predecir la nota ... 105
6.11 ¿Por qué no una recta para un sí/no? ... 106
6.12 Probabilidad, odds y log-odds ... 107
6.13 Regresión logística en R ... 108
6.14 Probabilidades predichas ... 109
6.15 Práctica Plus: mini-investigación (20 min) ... 110
6.16 Práctica Plus: soluciones ... 110
7 De la pregunta al párrafo de resultados ... 112
7.1 ¿Qué método elijo? ... 112
7.2 Cómo informar de un resultado ... 112
7.3 Errores frecuentes ... 113
7.4 Lo que hemos recorrido ... 113
7.5 Recursos ... 113

Cómo usar estos apuntes

Estos apuntes recogen todo el contenido de la presentación del curso, ordenado para leerlo con calma o imprimirlo. Cada apartado corresponde a una diapositiva y suele tener tres partes:

- **El código**, con el mismo número de paso que en clase (*Paso 1*, *Paso 2*…) y, debajo, la salida que devuelve R. El código es **acumulativo**: cada paso usa los objetos creados en los anteriores, así que conviene ejecutarlo en orden en un mismo script.
- **Las figuras y tablas**. Las que en clase eran interactivas aparecen aquí en una versión estática que resume lo que se veía al manipularlas.
- **En pocas palabras**: la explicación de qué significa el resultado y por qué importa.

Para reproducir los análisis necesitas R (o posit.cloud), los paquetes `tidyverse`, `rpart`, `rpart.plot`, `pheatmap` y `plotly`, y la carpeta `datos/` con `ele.csv`, `errores.csv` y `frecuencias.csv`. Los datos son **simulados** con fines docentes, aunque se comportan como podrían hacerlo unos datos reales de aprendientes de ELE. Si una salida tuya no coincide con la de estos apuntes, revisa el paso anterior.

Los apuntes siguen el orden del curso: cinco sesiones (descriptiva, visualización, inferencia, chi cuadrado y ANOVA, y árboles de decisión), un capítulo **Plus** con material complementario (estadística de corpus y modelos de regresión lineal y logística) y un cierre sobre cómo elegir el método e informar de los resultados.

La versión interactiva de todos los gráficos está en la presentación HTML del curso (`estadistica_ELE_interactiva.html`). Cuando la explicación habla de mover un deslizador o pulsar un botón, se refiere a esa versión.

¿Por qué estadística en un máster de ELE?

- **¿Funciona mejor este método didáctico que aquel?** → Prueba **t** (2 grupos) o **ANOVA** (3 o más) · Sesiones 3 y 4
- **¿Los aprendientes sinohablantes cometen más errores de *ser/estar*?** → Recuentos → **Wilcoxon** o **Kruskal-Wallis** · Sesiones 3 y 4
- **¿Influye una estancia en España en el uso del subjuntivo?** → Dos categóricas → **χ^2**; muchos predictores → **árbol** o **logística** · Sesiones 4, 5 y Plus
- **¿Qué palabras sobreusan los aprendientes frente a los nativos?** → Dos corpus → **log-likelihood** + **log-ratio** · Plus

! Idea clave

La estadística no sustituye a la interpretación lingüística: la **apoya** y la hace **replicable**.

En pocas palabras

Muchas preguntas que nos hacemos como docentes o investigadores de ELE se pueden responder con datos: si un método funciona mejor que otro, si ciertos aprendientes cometen más errores o si una experiencia, como vivir en España, cambia la forma de usar la lengua.

Cada pregunta del apartado lleva asociada la prueba estadística que la responde. No hace falta entender todavía esos nombres: lo importante es ver que **el tipo de pregunta decide el tipo de análisis**.

La estadística no sustituye a la interpretación lingüística. Nos ayuda a comprobar si lo que observamos en el aula es un patrón real o una casualidad, y permite que otra persona repita el análisis y llegue al mismo resultado.

Prepara tu espacio de trabajo

Opción A · RStudio en el aula

- Abrir RStudio → *File* → *New Project*
- Copiar la carpeta `datos/` dentro del proyecto

Opción B · posit.cloud (sin instalar nada)

- https://posit.cloud → *New Project* → *New RStudio Project*
- Subir `ele.csv`, `errores.csv` y `frecuencias.csv` a una carpeta `datos/`

Paso 0 · Instalar los paquetes (solo una vez)

```
install.packages(c("tidyverse", "rpart", "rpart.plot",
                   "pheatmap", "plotly", "DT"))
```

Paso 1 · Cargarlos (en cada sesión)

```
library(tidyverse)
library(rpart)
library(rpart.plot)
library(plotly)
```

En pocas palabras

Antes de analizar nada necesitamos un lugar donde trabajar. Hay dos opciones equivalentes: **RStudio** instalado en el ordenador o **posit.cloud**, que es RStudio en el navegador y no requiere instalar nada.

En ambos casos conviene crear un **proyecto**: una carpeta que R toma como punto de partida. Así, cuando escribamos `datos/ele.csv`, R sabrá dónde buscar.

Los **paquetes** son ampliaciones de R escritas por otras personas. `install.packages()` los descarga y se hace **una sola vez** por ordenador. `library()` los activa y hay que repetirlo **cada vez** que se abre R. Es como instalar una aplicación en el móvil y luego abrirla cada vez que se quiere usar.

Nuestros datos: 240 aprendientes de ELE

id	L1	nivel	estancia	horas	nota_dele	err_ser_estar	subjuntivo
A001	inglés	A2	sí	247	78	4	correcto
A002	inglés	B2	no	174	57	6	incorrecto
A003	inglés	A2	no	55	49	6	incorrecto
A004	italiano	A2	no	103	64	2	incorrecto
A005	inglés	B2	sí	364	75	4	correcto
A006	chino	B2	sí	354	69	0	correcto
A007	chino	A2	sí	129	48	5	incorrecto
A008	árabe	B2	no	303	62	3	correcto
A009	árabe	B1	no	277	54	3	incorrecto
A010	árabe	B1	sí	269	75	5	incorrecto

Datos **simulados** con fines docentes, con patrones plausibles en ELE. Filtra y ordena la tabla.

En pocas palabras

Este es el conjunto de datos que usaremos durante todo el curso. Cada **fila** es un aprendiente y cada **columna**, una característica suya: su lengua materna, su nivel, sus horas de clase, su nota, etc.

Los datos son **simulados**: no corresponden a personas reales, pero se han construido para que se comporten como podrían hacerlo unos datos reales de ELE. Por ejemplo, quienes tienen más horas de instrucción tienden a sacar mejor nota.

La tabla es interactiva: se puede ordenar pulsando en una columna y filtrar con las casillas de arriba. Merece la pena dedicar un minuto a mirarla, porque **conocer bien los datos es el primer paso de cualquier análisis**.

Las variables

Variable	Qué mide	Tipo
`L1`	Lengua materna (árabe, chino, inglés, italiano)	Categórica nominal
`nivel`	Nivel MCER (A2–C1)	Categórica ordinal
`estancia`	¿Ha vivido en un país hispanohablante?	Categórica binaria
`horas`	Horas de instrucción	Numérica continua
`nota_dele`	Nota en una prueba tipo DELE (0–100)	Numérica continua
`err_ser_estar`	Errores de *ser*/*estar* en una redacción	Numérica discreta (recuento)
`subjuntivo`	Uso del subjuntivo en un contexto obligatorio	Categórica binaria

En pocas palabras

Una **variable** es cualquier característica que cambia de una persona a otra. Esta tabla describe las siete variables del curso y, sobre todo, su **tipo**.

- **Categóricas**: clasifican en grupos. Pueden no tener orden (la L1), tenerlo (el nivel MCER, de A2 a C1) o tener solo dos valores (estancia: sí/no).
- **Numéricas**: miden cantidades. Pueden ser continuas, como la nota o las horas, o recuentos, como el número de errores.

El tipo no es un detalle técnico. Determina qué resumen tiene sentido (no se puede calcular la «media» de una lengua materna), qué gráfico conviene y qué prueba estadística podremos aplicar.

1 R y estadística descriptiva

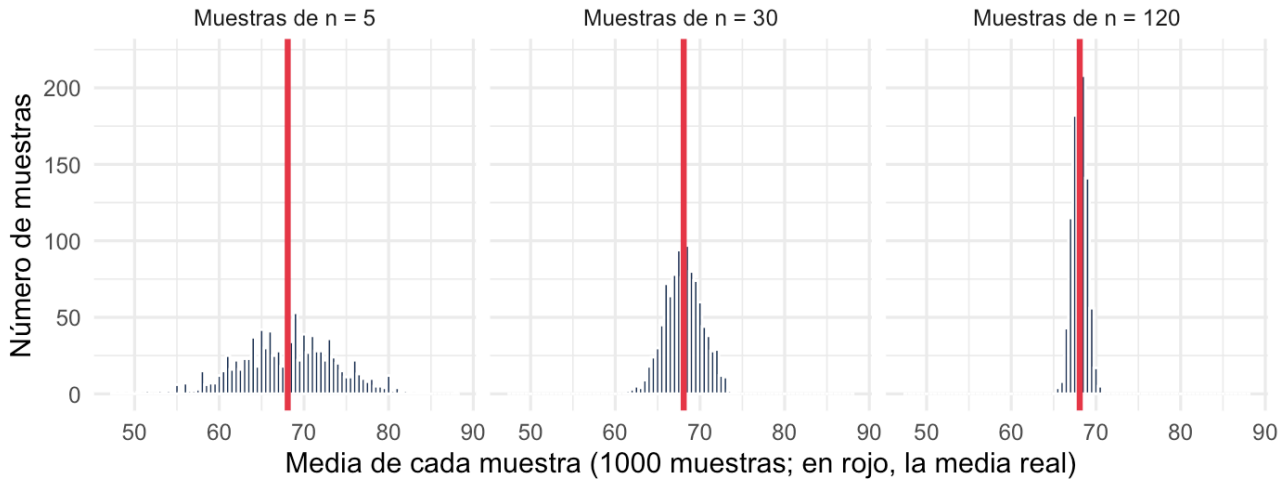
Sesión 1 · 60 minutos

> ℹ La pregunta de esta sesión
>
> ¿Cómo es un aprendiente «típico»… y cuánto se parecen entre sí?

Contenidos: RStudio y posit.cloud · Objetos y vectores · Cargar un .csv · Media, mediana, moda · DE y cuartiles · Resúmenes por grupos.

1.1 Población y muestra: la nota media del DELE

Variable: `nota_dele`, la nota de cada aprendiente en una prueba tipo DELE (de 0 a 100). Hacemos como si nuestros 240 aprendientes fueran **toda la población**: su nota media real es 68,1.



> En pocas palabras
>
> La **población** es el grupo completo sobre el que queremos saber algo; por ejemplo, todos los aprendientes de ELE. Casi nunca podemos estudiarla entera, así que trabajamos con una **muestra**: un subconjunto de personas elegidas al azar.
>
> La variable que estudiamos aquí es la **nota en la prueba tipo DELE** (`nota_dele`, de 0 a 100). Para poder comprobar cuánto se equivoca una muestra, hacemos un truco: tratamos a nuestros 240 aprendientes como si fueran la población entera. Así conocemos la respuesta correcta: la nota media real es **68,1**.
>
> Cada pulsación de «Extraer una muestra» elige al azar a 30 de esos 240 aprendientes (los puntos que se iluminan; su color indica su nota) y calcula **su** nota media. Una muestra puede dar 66,3; otra, 70,2; otra, 68,9. Ninguna coincide exactamente con 68,1 (la línea roja), pero todas se acercan. La diferencia entre la media de la muestra y la real es el **error de muestreo**. Si repetimos muchas veces, las medias forman una campana alrededor del valor verdadero.
>
> Con muestras **más grandes**, las medias se agrupan más cerca del valor real: con n = 5 es fácil obtener una media de 60 o de 76; con n = 120, casi todas caen entre 66 y 70. Esta idea, que una muestra grande se equivoca menos, es la base de toda la estadística inferencial que veremos en la sesión 3.

1.2 Tipos de variables

- **Nominal**: Categorías sin orden **L1**, estancia → moda, frecuencias, barras
- **Ordinal**: Categorías con orden **nivel MCER** → mediana, barras
- **Discreta**: Recuentos enteros **n.º de errores** → mediana, boxplot
- **Continua**: Medidas con decimales **nota, horas** → media, DE, histograma

! Idea clave

El **tipo de variable** decide qué estadístico, qué gráfico y qué prueba usamos. Lo veremos en el árbol para elegir la prueba de la sesión 3.

En pocas palabras

Las cuatro tarjetas resumen los tipos de variable y lo que podemos hacer con cada uno.

- **Nominal**: categorías sin orden, como la L1. Solo podemos contar cuántos hay en cada una.
- **Ordinal**: categorías con orden, como A2 < B1 < B2 < C1. Podemos ordenar, pero la distancia entre niveles no es necesariamente igual.
- **Discreta**: recuentos de números enteros, como los errores cometidos.
- **Continua**: medidas que admiten decimales, como la nota o las horas.

Debajo de cada tarjeta aparece el resumen y el gráfico adecuados. Esta clasificación volverá a aparecer cuando tengamos que elegir una prueba estadística.

1.3 R y RStudio

R es el lenguaje: el motor

- Gratuito y de código abierto
- Miles de paquetes
- Reproducible: el código **documenta** el análisis

RStudio es el entorno: el salpicadero - ① **Script**: Escribimos y guardamos el código - ③ **Entorno**: Los objetos que vamos creando - ② **Consola**: Aquí se ejecuta y aparecen los resultados - ④ **Archivos · Gráficos · Ayuda**: Todo lo demás

`Ctrl + Enter` (`Cmd + Enter` en Mac) ejecuta la línea del script.

En pocas palabras

R es un lenguaje de programación pensado para el análisis de datos. Es gratuito y tiene miles de paquetes. Su gran ventaja es la **reproducibilidad**: el código queda escrito y cualquiera puede repetir el análisis exactamente.

RStudio es el programa en el que escribimos y ejecutamos R. Tiene cuatro paneles:

1. El **script**, donde escribimos y guardamos el código.
2. La **consola**, donde R ejecuta las órdenes y muestra los resultados.
3. El **entorno**, que lista los objetos que hemos creado.
4. Un panel para archivos, gráficos y ayuda.

El atajo `Ctrl + Enter` (`Cmd + Enter` en Mac) envía la línea del script a la consola.

1.4 R como calculadora y objetos

Paso 1 · Calcular

```
3 + 4
#> [1] 7
(85 + 72 + 90) / 3
#> [1] 82.33333
sqrt(144)
#> [1] 12
```

Paso 2 · Guardar en objetos

```
nota_ana <- 78
nota_li  <- 64
media_dos <- (nota_ana + nota_li) / 2
media_dos
#> [1] 71
```

! Idea clave

`<-` significa «guarda en». Todo lo que va tras `#` es un **comentario**: R no lo ejecuta.

En pocas palabras

Lo más básico que hace R es calcular: sumas, divisiones, raíces cuadradas. La salida aparece debajo, precedida de `[1]`, que simplemente indica que es el primer valor del resultado.

El paso importante es el segundo: **guardar resultados en objetos**. La flecha `<-` significa «guarda en». `nota_ana <- 78` crea un objeto llamado `nota_ana` que contiene el número 78. A partir de ese momento podemos usar el nombre en lugar del número.

Todo lo que aparece después del símbolo `#` es un **comentario**: R lo ignora. Sirve para dejar notas a nosotros mismos o a otros lectores del código.

1.5 Vectores y funciones

Paso 3 · Un vector es una lista de valores

```
notas <- c(78, 64, 90, 55, 71, 83)
L1s   <- c("inglés", "chino", "italiano", "árabe", "chino", "inglés")
length(notas)
#> [1] 6
notas[2]
#> [1] 64
notas > 70
#> [1]  TRUE FALSE  TRUE FALSE  TRUE  TRUE
mean(notas)        # una función: nombre(argumentos)
#> [1] 73.5
```

En pocas palabras

Un **vector** es una lista de valores del mismo tipo, creada con `c()` («combinar»). `notas` contiene seis números y `L1s`, seis palabras.

Con los vectores podemos hacer varias cosas:

- `length(notas)` cuenta cuántos elementos hay.
- `notas[2]` extrae el segundo elemento.
- `notas > 70` pregunta a cada valor si supera 70 y devuelve `TRUE` o `FALSE`.

Una **función** es una orden que recibe algo entre paréntesis y devuelve un resultado: `mean(notas)` calcula la media. Toda la estadística en R consiste en aplicar funciones a datos.

1.6 Cargar nuestros datos

Paso 4 · Leer el archivo

```
ele <- read_csv("datos/ele.csv")
```

Paso 5 · Echar un vistazo

```
glimpse(ele)
#> Rows: 240
#> Columns: 8
#> $ id            <chr> "A001", "A002", "A003", "A004", "A005", "A006", "A007", "A008", …
#> $ L1            <chr> "inglés", "inglés", "inglés", "italiano", "inglés", "chino", "ch…
#> $ nivel         <fct> A2, B2, A2, A2, B2, B2, A2, B2, B1, B1, C1, B2, B1, C1, B2, B1, …
#> $ estancia      <chr> "sí", "no", "no", "no", "sí", "sí", "sí", "no", "no", "sí", "no"…
#> $ horas         <dbl> 247, 174, 55, 103, 364, 354, 129, 303, 277, 269, 392, 312, 260, …
#> $ nota_dele     <dbl> 78, 57, 49, 64, 75, 69, 48, 62, 54, 75, 82, 61, 78, 76, 82, 46, …
#> $ err_ser_estar <dbl> 4, 6, 6, 2, 4, 0, 5, 3, 3, 5, 3, 1, 3, 5, 1, 3, 2, 10, 4, 1, 5, …
#> $ subjuntivo    <chr> "correcto", "incorrecto", "incorrecto", "incorrecto", "correcto"…
```

También: `View(ele)` (como una hoja de cálculo), `head(ele)`, `nrow(ele)`, `names(ele)`. Con `ele$nota_dele` seleccionamos una columna.

En pocas palabras

Hasta ahora escribíamos los datos a mano. En la práctica, los datos están en un archivo, normalmente una hoja de cálculo guardada como **.csv** (valores separados por comas).

`read_csv()` lee el archivo y lo guarda en un objeto llamado `ele`. El resultado es una **tabla de datos**: filas para las personas y columnas para las variables.

`glimpse()` ofrece un vistazo rápido: cuántas filas y columnas hay, el nombre de cada variable, su tipo (`chr` para texto, `dbl` para números) y los primeros valores. Para referirse a una sola columna se usa el símbolo `$`: `ele$nota_dele` es la columna de notas.

1.7 Tendencia central

Paso 6 · Media, mediana y moda

```
mean(ele$nota_dele)
#> [1] 68.07917
```

```r
median(ele$nota_dele)
#> [1] 68
table(ele$L1)   # la moda: la categoría más frecuente
#>
#>    árabe    chino   inglés italiano
#>       66       49       73       52
```

En pocas palabras

Las medidas de **tendencia central** responden a la pregunta «¿cuál es el valor típico?».

- La **media** suma todos los valores y divide entre cuántos hay. En nuestros datos, la nota media es de 68 puntos.
- La **mediana** es el valor que queda justo en medio cuando ordenamos los datos: la mitad está por debajo y la otra mitad por encima. Aquí también es 68.
- La **moda** es el valor o la categoría más frecuente. Para variables como la L1, que no son números, es la única medida posible: `table()` cuenta cuántos hay de cada lengua y la moda es el inglés.

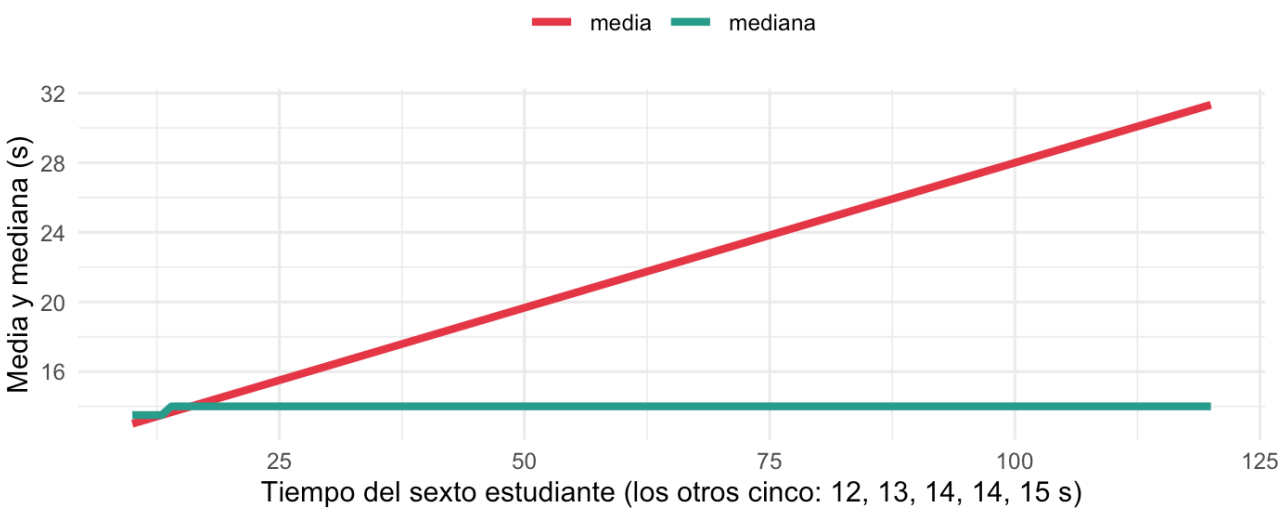
Que media y mediana coincidan indica que las notas están repartidas de forma bastante simétrica.

1.8 La media persigue a los extremos

Ejemplo inventado: `tiempos` son los **segundos** que tardan seis estudiantes en responder una misma pregunta de un test en línea. Cinco tardan entre 12 y 15 segundos; el sexto se distrajo y tardó 95.

Prueba esto en R

```r
tiempos <- c(12, 14, 13, 15, 14, 95)   # segundos que tarda cada estudiante en responder
mean(tiempos); median(tiempos)
```



En pocas palabras

Este apartado muestra la principal diferencia entre la media y la mediana.

La variable `tiempos` es inventada: recoge cuántos **segundos** tardan seis estudiantes en responder una misma pregunta de un test en línea. Cinco responden en 12, 13, 14, 14 y 15 segundos; el sexto se distrajo y tardó 95.

- La **media** es (12 + 14 + 13 + 15 + 14 + 95) / 6 = 163 / 6 ≈ **27,2 segundos**. Ningún estudiante tardó eso: cinco tardaron mucho menos y uno mucho más.
- La **mediana** es **14 segundos**: ordenados (12, 13, 14, 14, 15, 95), los dos valores centrales son 14 y 14.

La media se dispara porque suma ese valor extremo, mientras que la mediana apenas cambia, porque solo le importa qué valor queda en el centro.

En el gráfico interactivo se puede arrastrar el último valor: la media lo «persigue» y la mediana se queda quieta.

Por eso, cuando los datos tienen **valores extremos** o están muy desequilibrados hacia un lado (como los sueldos o los tiempos de reacción), la **mediana** representa mejor lo que es típico.

1.9 Cómo se calcula cada medida

Notas de 5 aprendientes: **55, 62, 70, 71, 92**

Medida	Cálculo	Resultado
Media	(55 + 62 + 70 + 71 + 92) / 5	70
Mediana	valor central de los datos **ordenados**	70
Moda	valor más repetido	— (no hay)
Rango	92 – 55	37

! Idea clave

Media → numéricas simétricas · **Mediana** → asimétricas u ordinales · **Moda** → nominales (L1)

En pocas palabras

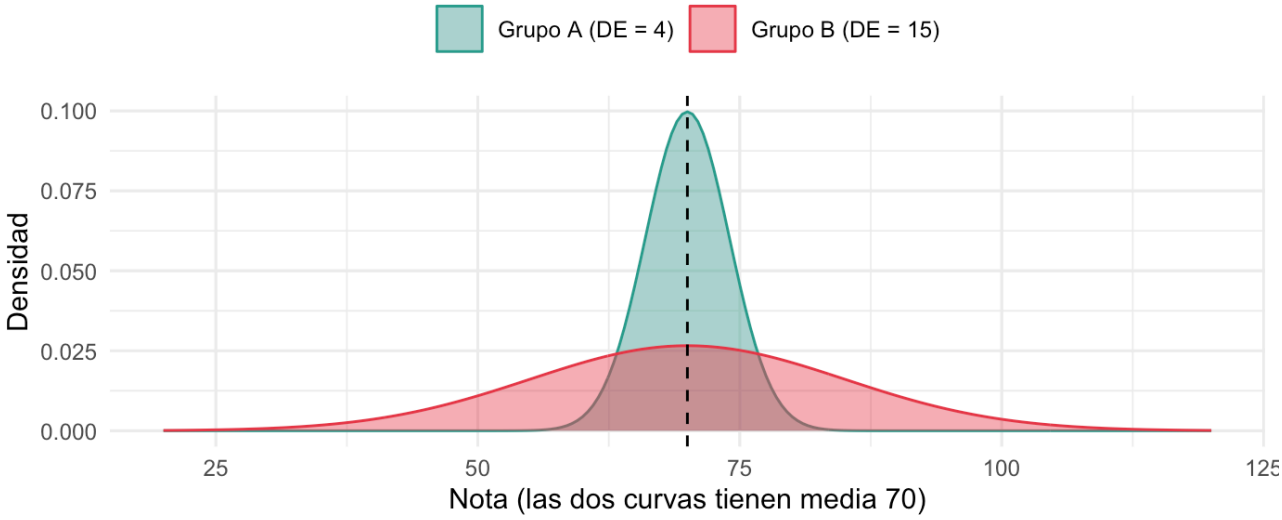
La tabla hace a mano, con solo cinco notas, lo que R hace con 240.

- **Media**: se suman las cinco notas (350) y se divide entre cinco: 70.
- **Mediana**: se ordenan las notas y se toma la del centro, la tercera: 70. Si hubiera un número par de valores, se haría la media de los dos centrales.
- **Moda**: no hay ninguna nota repetida, así que no existe.
- **Rango**: la distancia entre la nota más alta y la más baja: 37 puntos.

La regla práctica del recuadro resume cuándo usar cada una: media para números repartidos de forma simétrica, mediana para datos asimétricos u ordinales y moda para categorías.

1.10 Dispersión: ¿cuánto varían los datos?

DE = desviación típica (también llamada **desviación estándar**; en inglés, *SD*): cuánto se aleja, en promedio, cada nota de la media. **RIC** = rango intercuartílico: la anchura del 50 % central de las notas. Las dos se miden en **puntos de nota**.



En pocas palabras

Saber la media no basta. Dos grupos pueden tener la misma nota media y ser muy distintos: en uno, todos sacan alrededor de 70; en el otro, hay notas de 40 y de 100.

La **dispersión** mide cuánto se alejan los datos unos de otros. En el gráfico, dos grupos simulados tienen la misma nota media (70). Al aumentar la dispersión de un grupo, su curva se aplana y se ensancha: las personas se parecen menos entre sí.

Las dos medidas que aparecen a la derecha:

- **DE (desviación típica)**. *Desviación típica* y *desviación estándar* son **lo mismo**: dos nombres para la misma medida (en inglés, *standard deviation*, SD). Indica cuánto se aleja, en promedio, cada valor de la media. En el grupo A (DE = 4), unas dos de cada tres notas están entre 66 y 74 (70 ± 4); en el grupo B (DE = 15), entre 55 y 85.
- **RIC (rango intercuartílico)**. Es la distancia entre el primer cuartil (Q1) y el tercero (Q3): la anchura de la **mitad central** del grupo. Con DE = 4, el RIC es de unos 5,4 puntos: la mitad central del grupo A saca entre 67,3 y 72,7.

¿Y la **varianza**? Es la DE elevada al cuadrado ($4^2 = 16$). Contiene la misma información, pero en «puntos al cuadrado», una unidad difícil de interpretar; por eso para describir datos se usa la DE. La veremos paso a paso dos apartados más adelante.

Por eso un resumen completo siempre acompaña la media de una medida de dispersión. Decir «la nota media es 68» informa poco; decir «68, con una desviación típica de 13» ya describe el grupo.

1.11 Medidas de dispersión

Paso 7 · Rango, DE, RIC y cuartiles

```
range(ele$nota_dele)
#> [1]  32 100
sd(ele$nota_dele)        # desviación típica
#> [1] 12.89603
IQR(ele$nota_dele)       # rango intercuartílico
#> [1] 17
quantile(ele$nota_dele)
#>   0%  25%  50%  75% 100%
#>   32   60   68   77  100
```

En pocas palabras

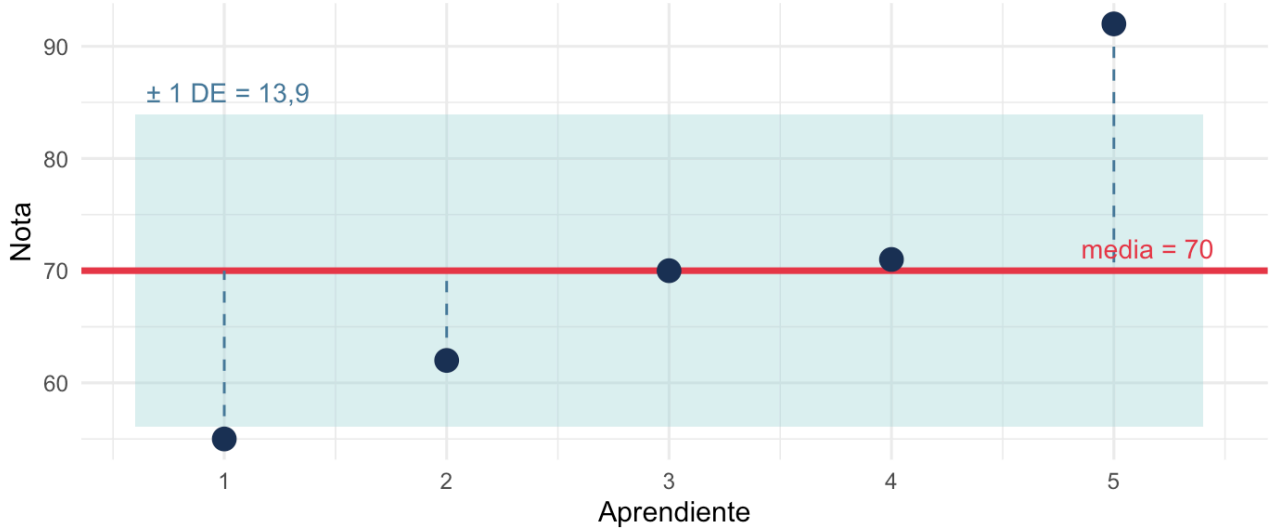
R calcula en una línea cada medida de dispersión.

- `range()`: la nota mínima y la máxima (32 y 100).
- `sd()`: la **desviación típica** (o desviación estándar, DE; en inglés, *standard deviation*), la distancia media de cada nota respecto a la media. Aquí es de 12,9 puntos: un aprendiente típico se aleja unos 13 puntos de la nota media (68).
- `IQR()`: el **rango intercuartílico** (RIC), la amplitud del 50 % central de los datos: Q3 – Q1 = 77 – 60 = 17 puntos. No le afectan los valores extremos.
- `quantile()`: los **cuartiles**, los valores que dividen los datos ordenados en cuatro partes iguales (0 %, 25 %, 50 %, 75 % y 100 %).

Si quisiéramos la **varianza**, la función sería `var(ele$nota_dele)`: 166,3, que es 12,9². La desviación típica va con la media; el rango intercuartílico, con la mediana.

1.12 La desviación típica, paso a paso

Nota	① Distancia a la media (x – 70)	② Al cuadrado
55	–15	225
62	–8	64
70	+0	0
71	+1	1
92	+22	484
Suma	0	**774**
Varianza		774 / 4 = **193,5**
DE		√193,5 ≈ **13,9**



En pocas palabras

La desviación típica parece una fórmula complicada, pero son cinco pasos sencillos que el gráfico recorre uno a uno con las notas de cinco aprendientes: **55, 62, 70, 71 y 92**.

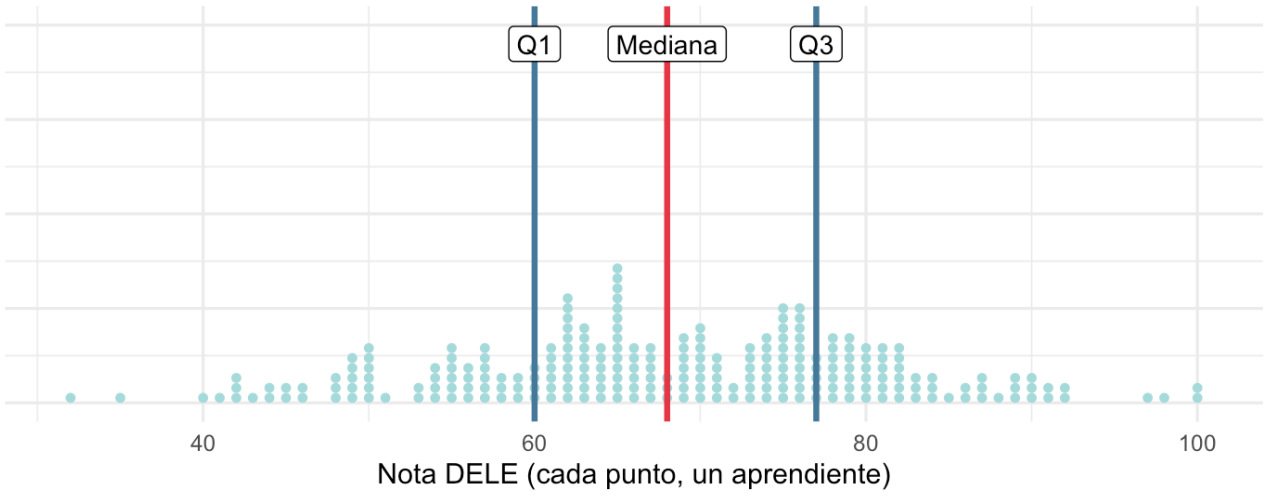
1. **Calcular la media**: 350 / 5 = 70.
2. **Restar la media a cada valor**, para ver cuánto se aleja cada persona: −15, −8, 0, +1 y +22. Si sumamos esas distancias, da 0 (las negativas compensan a las positivas), así que no sirven tal cual.
3. **Elevar cada distancia al cuadrado**, para que desaparezcan los signos: 225, 64, 0, 1 y 484. En el gráfico se ven literalmente como cuadrados.
4. **Sumar los cuadrados y promediarlos**. La suma es 225 + 64 + 0 + 1 + 484 = **774** (la *suma de cuadrados*). Al dividirla entre n − 1 = 4 obtenemos **193,5**: es la **varianza**. Se divide entre n − 1 y no entre n porque, una vez conocida la media, solo 4 de las 5 distancias son libres: la quinta queda fijada para que todas sumen 0.
5. **Hacer la raíz cuadrada** para volver a las unidades originales: √193,5 ≈ **13,9 puntos**. Es la **desviación típica** (DE).

La varianza (193,5) está en «puntos al cuadrado», una unidad que no se puede comparar con las notas. La DE (13,9) sí está en puntos: las notas de estos cinco aprendientes se alejan de la media, en promedio, unos 14 puntos.

En el paso 5, la **franja sombreada** marca la zona «media ± 1 DE»: de 70 − 13,9 = 56,1 a 70 + 13,9 = 83,9. Tres de las cinco notas (62, 70 y 71) caen dentro; 55 y 92 quedan fuera. En unos datos con forma de campana, esa franja contiene aproximadamente a dos de cada tres personas.

Como los valores alejados generan cuadrados enormes (el 92 aporta 484 de los 774), la desviación típica también es sensible a los extremos.

1.13 Cuartiles y percentiles



En pocas palabras

Si ordenamos a los 240 aprendientes de menor a mayor nota, un **percentil** indica qué nota deja por debajo un determinado porcentaje de personas. El percentil 90 es la nota que solo supera el 10 % del grupo.

Tres percentiles tienen nombre propio:

- **Q1** (percentil 25): deja por debajo a una cuarta parte.
- **Mediana** (percentil 50): deja por debajo a la mitad.
- **Q3** (percentil 75): deja por debajo a tres cuartas partes.

Entre Q1 y Q3 está la mitad central del grupo. Esa franja es la «caja» del diagrama de caja (*boxplot*), que veremos en la sesión 2.

1.14 `summary()` y frecuencias

Paso 8 · Todo de una vez

```
summary(ele[, c("horas", "nota_dele",
                "err_ser_estar")])
#>      horas         nota_dele      err_ser_estar
#>  Min.   : 29.0   Min.   : 32.00   Min.   : 0.000
#>  1st Qu.:221.8   1st Qu.: 60.00   1st Qu.: 2.000
#>  Median :290.0   Median : 68.00   Median : 3.000
#>  Mean   :283.7   Mean   : 68.08   Mean   : 3.175
#>  3rd Qu.:348.2   3rd Qu.: 77.00   3rd Qu.: 4.000
#>  Max.   :549.0   Max.   :100.00   Max.   :10.000
```

Paso 9 · Frecuencias

```
table(ele$nivel)
#>
#> A2 B1 B2 C1
#> 42 66 70 62
prop.table(table(ele$nivel)) |>
  round(2)
#>
#>   A2   B1   B2   C1
#> 0.17 0.28 0.29 0.26
```

En pocas palabras

`summary()` resume varias variables numéricas de una vez: mínimo, Q1, mediana, media, Q3 y máximo. Es la primera orden que conviene ejecutar con cualquier conjunto de datos nuevo, porque permite detectar enseguida valores imposibles o errores de introducción.

Para las variables categóricas, el resumen adecuado es una **tabla de frecuencias**:

- `table()` cuenta cuántas personas hay en cada nivel.
- `prop.table()` convierte esos recuentos en proporciones, y `round(2)` las redondea a dos decimales. Una proporción de 0,29 equivale a un 29 %.

1.15 Resumir por grupos con `dplyr`

Paso 10 · group_by() + summarise()

```r
ele |>
  group_by(L1) |>
  summarise(n = n(),
            media = mean(nota_dele),
            de = sd(nota_dele))
#> # A tibble: 4 × 4
#>   L1           n media    de
#>   <chr>    <int> <dbl> <dbl>
#> 1 chino       49  61.6  13.2
#> 2 inglés      73  69.7  10.3
#> 3 italiano    52  76.3  11.4
#> 4 árabe       66  64.6  12.8
```

`|>` es la **tubería**: «y después…».

En pocas palabras

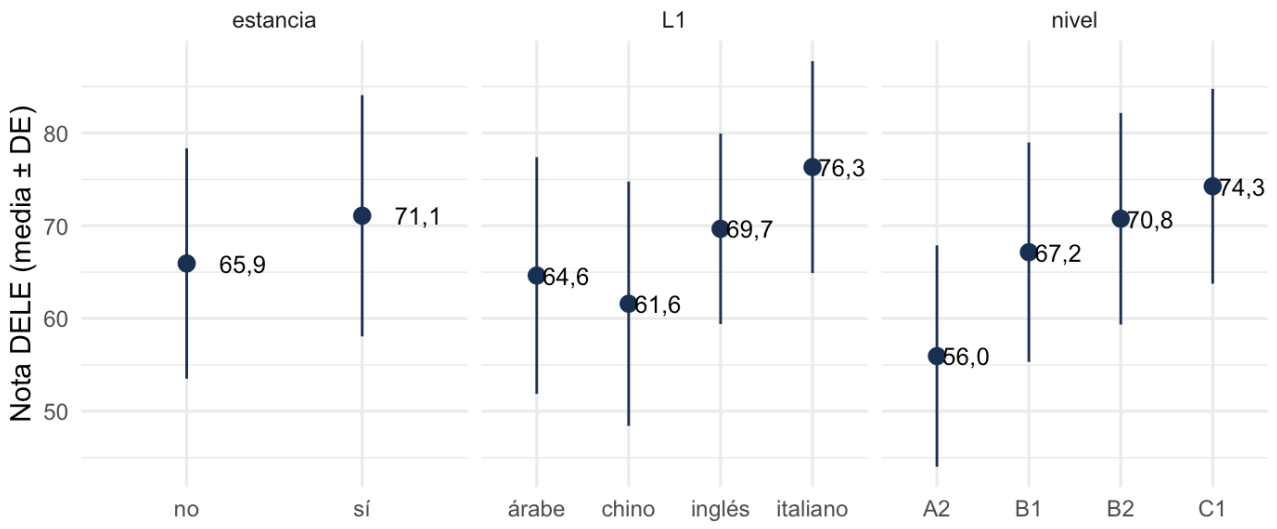
Casi siempre queremos comparar grupos: ¿sacan la misma nota los aprendientes de distintas lenguas maternas?

El paquete `dplyr` lo hace con dos órdenes encadenadas:

- `group_by(L1)` separa los datos por lengua materna.
- `summarise()` calcula, dentro de cada grupo, lo que le pidamos: cuántos hay (`n()`), la media y la desviación típica.

El símbolo `|>` se llama **tubería** y se lee «y después». Permite escribir los pasos en el orden en que se piensan: toma los datos, agrúpalos y después resúmelos. El resultado muestra que los italohablantes tienen la media más alta (76) y los sinohablantes, la más baja (62).

1.16 Explora cualquier resumen por grupos



En pocas palabras

Esta herramienta permite hacer lo mismo que el apartado anterior con cualquier combinación de variables, sin escribir código.

En los menús se elige por qué variable agrupar (L1, nivel, estancia), qué variable resumir y qué estadístico calcular. El gráfico y la tabla se actualizan al instante.

Lo más útil es el **código** que aparece a la derecha: se reescribe automáticamente con cada cambio y es exactamente lo que habría que escribir en R para obtener ese resumen. Es una forma de aprender la sintaxis viendo su efecto.

Algunas preguntas para explorar: ¿qué grupo acumula más horas? ¿Mejora la nota con el nivel? ¿Quién comete menos errores de *ser/estar*?

1.17 Filtrar, seleccionar y crear columnas

Paso 11 · filter() · select() · mutate()

```
ele |>
  filter(nivel == "C1", estancia == "sí") |>
  select(id, L1, nota_dele) |>
  mutate(aprobado = nota_dele >= 60) |>
  head(4)
#> # A tibble: 4 × 4
#>   id    L1       nota_dele aprobado
#>   <chr> <chr>        <dbl> <lgl>
#> 1 A014  italiano        76 TRUE
#> 2 A017  chino           75 TRUE
#> 3 A020  italiano        90 TRUE
#> 4 A029  inglés          79 TRUE
```

En pocas palabras

Tres funciones más completan el trabajo básico con tablas:

- `filter()` se queda con las **filas** que cumplen una condición. Aquí, los aprendientes de nivel C1 que han hecho una estancia. El doble igual (`==`) significa «es igual a».
- `select()` se queda con las **columnas** que nos interesan.
- `mutate()` **crea una columna nueva** a partir de las existentes. Aquí, `aprobado` vale `TRUE` si la nota es de 60 o más.

`head(4)` muestra solo las cuatro primeras filas. Encadenadas con la tubería, estas funciones permiten preparar los datos paso a paso, como una receta.

1.18 Práctica 1 (20 min)

1. Crea un proyecto en RStudio o en posit.cloud y carga `ele.csv`.
2. ¿Cuántos aprendientes hay de cada L1? ¿Y de cada nivel?
3. Calcula la media, la mediana y la DE de `horas`.
4. Calcula la nota media del DELE **por nivel**. ¿Aumenta con el nivel?
5. ¿Qué porcentaje usa bien el subjuntivo en cada L1?

Pista para la 5: `count(L1, subjuntivo)` y después `group_by(L1) |> mutate(pct = 100 * n / sum(n))`

1.19 Práctica 1: soluciones

Soluciones de la práctica 1

```
ele <- read_csv("datos/ele.csv")
table(ele$L1); table(ele$nivel)
mean(ele$horas); median(ele$horas); sd(ele$horas)
ele |> group_by(nivel) |> summarise(media = mean(nota_dele))
ele |> count(L1, subjuntivo) |>
  group_by(L1) |> mutate(pct = 100 * n / sum(n))
```

En pocas palabras

Este bloque reúne una posible solución para cada ejercicio de la práctica 1:

1. Cargar los datos con `read_csv()`.
2. Contar aprendientes por L1 y por nivel con `table()`.
3. Calcular media, mediana y desviación típica de las horas.
4. Agrupar por nivel y calcular la nota media: se comprueba que la nota **aumenta con el nivel**.
5. Contar cuántos aciertan y fallan el subjuntivo en cada L1 y convertirlo en porcentaje dentro de cada lengua.

El último ejercicio introduce `count()`, un atajo que agrupa y cuenta a la vez. No hay una única forma correcta de escribir el código: si el resultado coincide, la solución es válida.

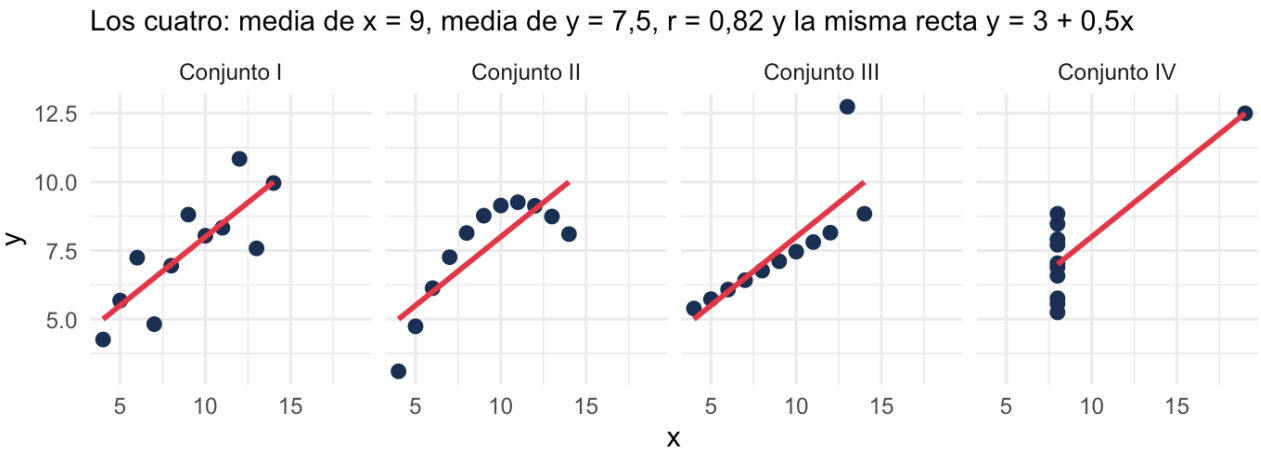
2 Visualización de datos

Sesión 2 · 60 minutos

> **La pregunta de esta sesión**
>
> ¿Qué nos cuentan los datos antes de hacer ninguna prueba?

Contenidos: Anscombe · Gramática de gráficos · Histogramas, cajas y violines · Motion charts · Gráficos interactivos · Mapas de calor.

2.1 ¿Por qué visualizar? El cuarteto de Anscombe

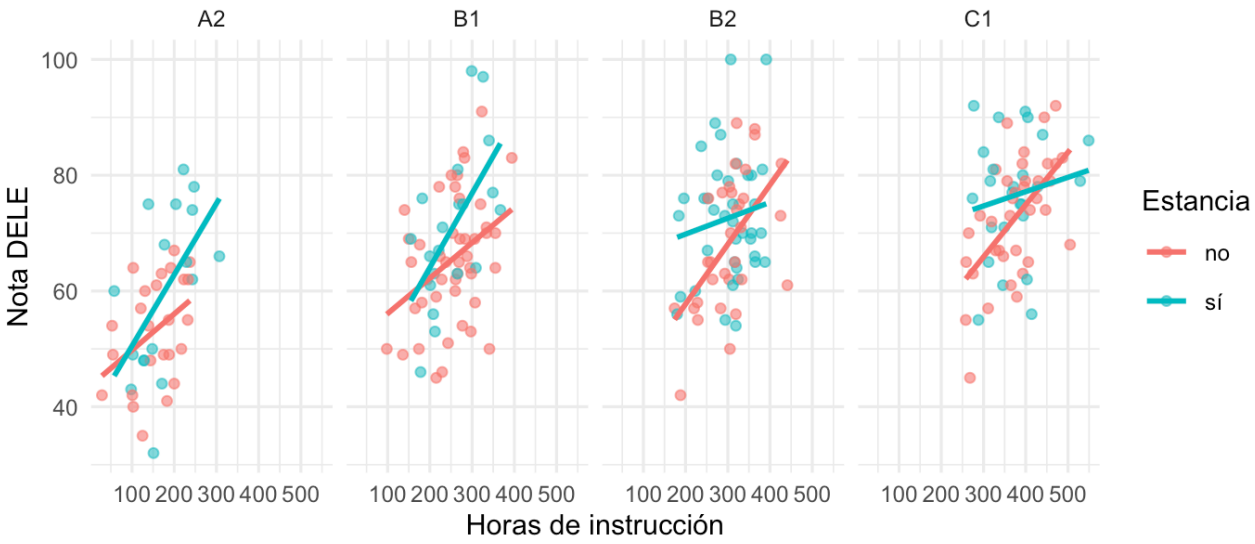


> **En pocas palabras**
>
> El estadístico Francis Anscombe construyó en 1973 cuatro conjuntos de datos con una propiedad sorprendente: tienen **exactamente** la misma media, la misma varianza, la misma correlación y la misma recta de regresión. Si solo miramos los números, son idénticos.
>
> Al dibujarlos, en cambio, son completamente distintos: uno sigue una recta, otro una curva, otro tiene un valor extremo que lo distorsiona todo, y en el último una sola persona crea la relación.
>
> La lección es sencilla: **los resúmenes numéricos pueden engañar**. Antes de calcular cualquier prueba hay que mirar los datos.

2.2 La gramática de gráficos, capa a capa

El gráfico completo, capa a capa

```
ggplot(ele, aes(x = horas, y = nota_dele,      # datos + estéticas x e y
                colour = estancia)) +          # tercera estética: el color
  geom_point(alpha = .6) +                     # capa de puntos
  geom_smooth(method = "lm", se = FALSE) +     # capa de rectas, una por grupo
  facet_wrap(~ nivel, nrow = 1) +              # un panel por nivel
  labs(x = "Horas de instrucción", y = "Nota DELE", colour = "Estancia")
```



En pocas palabras

`ggplot2`, el paquete de gráficos más usado en R, construye cada gráfico como una suma de **capas**, unidas con el signo `+`. El ejemplo relaciona las **horas de instrucción** (eje x) con la **nota DELE** (eje y) de los 240 aprendientes:

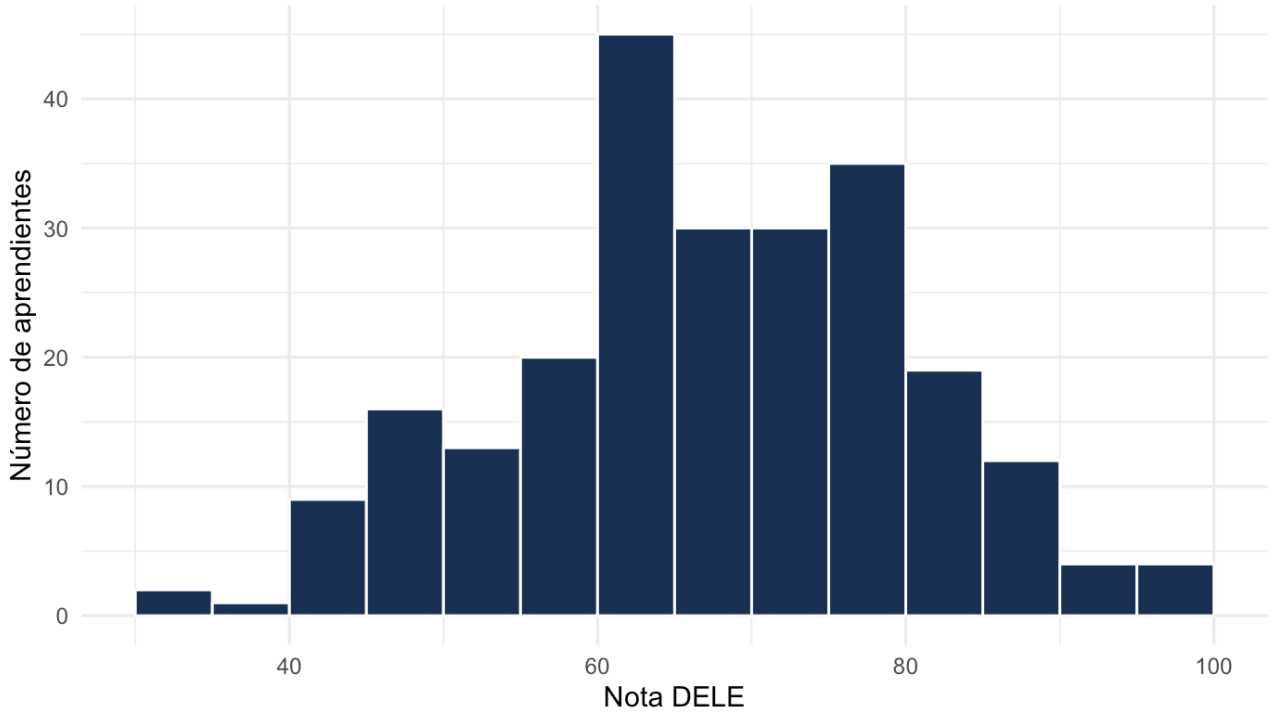
- **Datos**: la tabla con la que trabajamos (`ele`).
- **Estética** (`aes`): qué variable va en cada eje y cuál decide el color o el tamaño. Aquí, `colour = estancia` pinta de un color a quienes han hecho una estancia y de otro a quienes no.
- **Geometría** (`geom_`): la forma de representar los datos: puntos (`geom_point`), barras, líneas…
- **Recta de tendencia**: `geom_smooth(method = "lm")` dibuja la recta que mejor resume la relación. `lm` significa *linear model* (modelo lineal): R busca la recta que pasa lo más cerca posible de todos los puntos. La veremos con detalle en el material Plus. `se = FALSE` quita la banda gris de incertidumbre que la acompaña por defecto.
- **Facetas** (`facet_wrap`): dividir el gráfico en paneles, uno por nivel (A2, B1, B2, C1).
- **Etiquetas** (`labs`): títulos y nombres de los ejes.

Cada clic en la herramienta añade una capa y resalta la línea de código correspondiente. El botón **Copiar** del recuadro de código copia la versión de la capa en la que estés, lista para pegar en RStudio. Entender esta lógica permite crear cualquier gráfico combinando piezas.

2.3 Histograma: una variable numérica

Paso 1 · Histograma

```
ggplot(ele, aes(x = nota_dele)) +
  geom_histogram(binwidth = 5,     # cada barra: 5 puntos de nota
                 boundary = 0,     # tramos 60–65, 65–70…
                 fill = "#1D3557",
                 colour = "white") +
  labs(x = "Nota DELE",
       y = "Número de aprendientes")
```



En pocas palabras

Un **histograma** muestra cómo se reparten los valores de una variable numérica. Divide las notas en tramos y dibuja una barra por tramo: cuanto más alta, más aprendientes tienen una nota en ese intervalo.

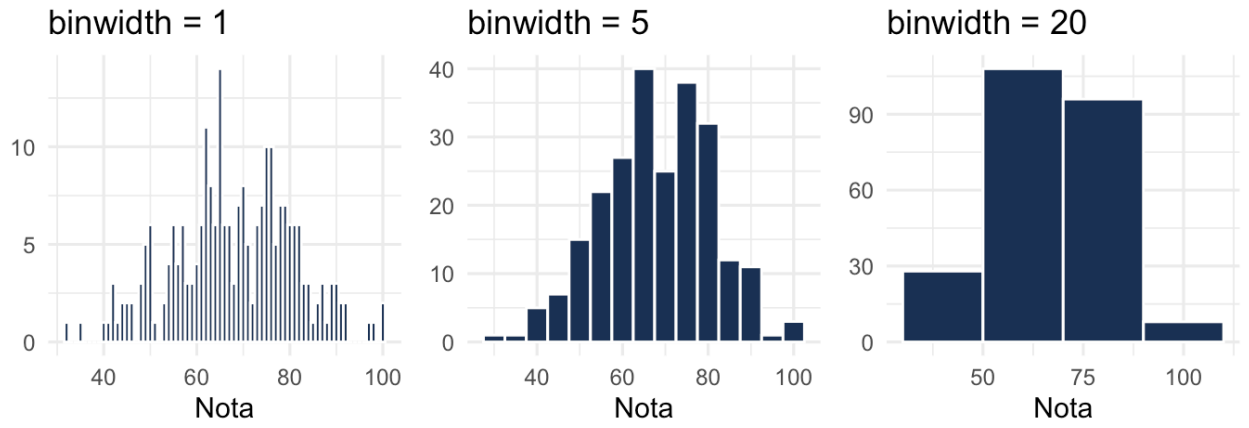
`binwidth` (anchura del tramo, en inglés *bin*) indica cuántas unidades de la variable abarca cada barra. Como la variable es `nota_dele` y se mide en puntos de 0 a 100, `binwidth = 5` significa que cada barra agrupa **5 puntos de nota**: una barra para las notas de 60 a 65, otra de 65 a 70, otra de 70 a 75… `boundary = 0` hace que los tramos empiecen en números redondos (múltiplos de 5). Por ejemplo, la barra de 75 a 80 mide 35: hay 35 aprendientes con una nota en ese tramo.

En nuestro caso, la mayoría se concentra entre 60 y 80 puntos, con menos personas en los extremos. Es la típica forma de campana.

En el código:

- `aes(x = nota_dele)`: la nota va en el eje horizontal.
- `geom_histogram(binwidth = 5, boundary = 0)`: tramos de 5 puntos que empiezan en múltiplos de 5.
- `fill` y `colour`: color de relleno y del borde de las barras.
- `labs()`: nombres de los ejes.

2.4 ¿Qué anchura de barra?



En pocas palabras

La forma de un histograma depende mucho de la **anchura de las barras** (`binwidth`), y no hay un valor correcto universal. Recordemos que la anchura se expresa en las unidades de la variable: con `nota_dele`, en puntos de nota; con `horas`, en horas de clase.

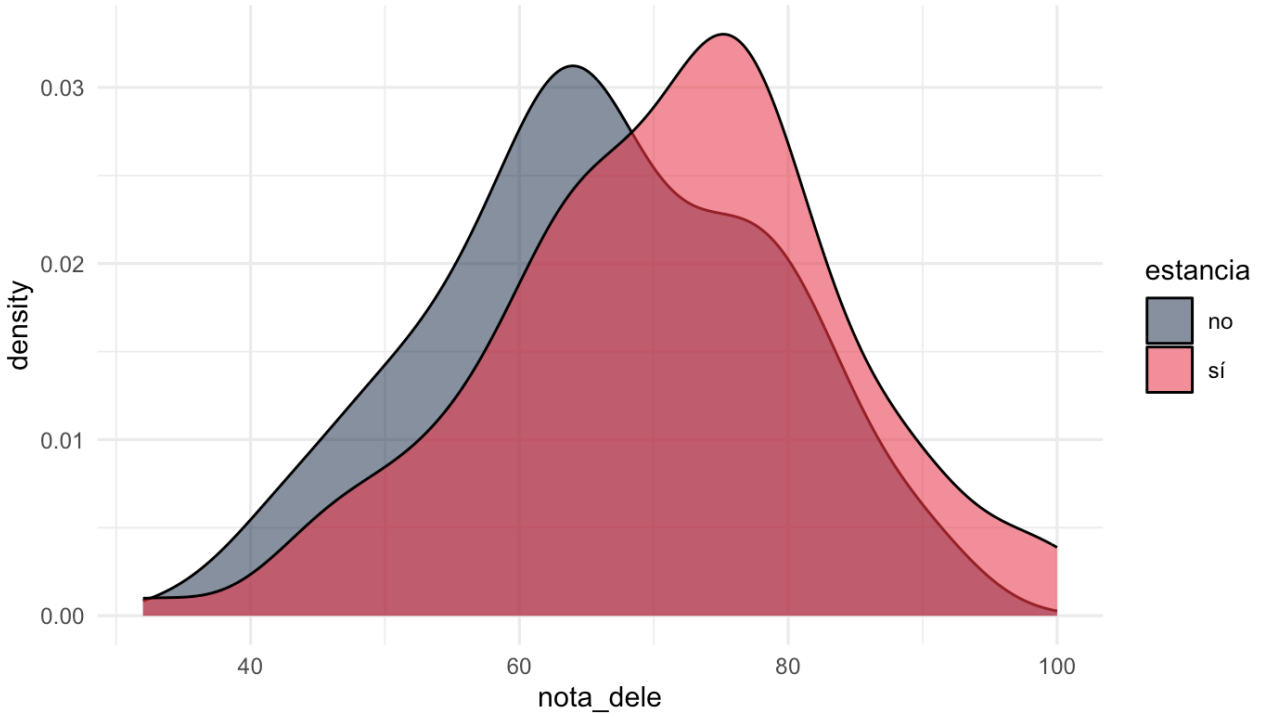
- Con barras **muy estrechas** (`binwidth = 1`, una barra por cada punto de nota), cada barra tiene solo dos o tres aprendientes y el gráfico se llena de picos y huecos que no significan nada: es ruido.
- Con barras **muy anchas** (`binwidth = 20`, tramos de 20 puntos), todo se agrupa en cuatro o cinco bloques y se pierde la forma.

El deslizador permite probar distintos valores, también con la variable «horas». La regla práctica es buscar una anchura que muestre la forma general sin detalles caprichosos y, si hay dudas, probar varias. El código de la izquierda se actualiza para copiar la anchura elegida.

2.5 Distribuciones por grupos

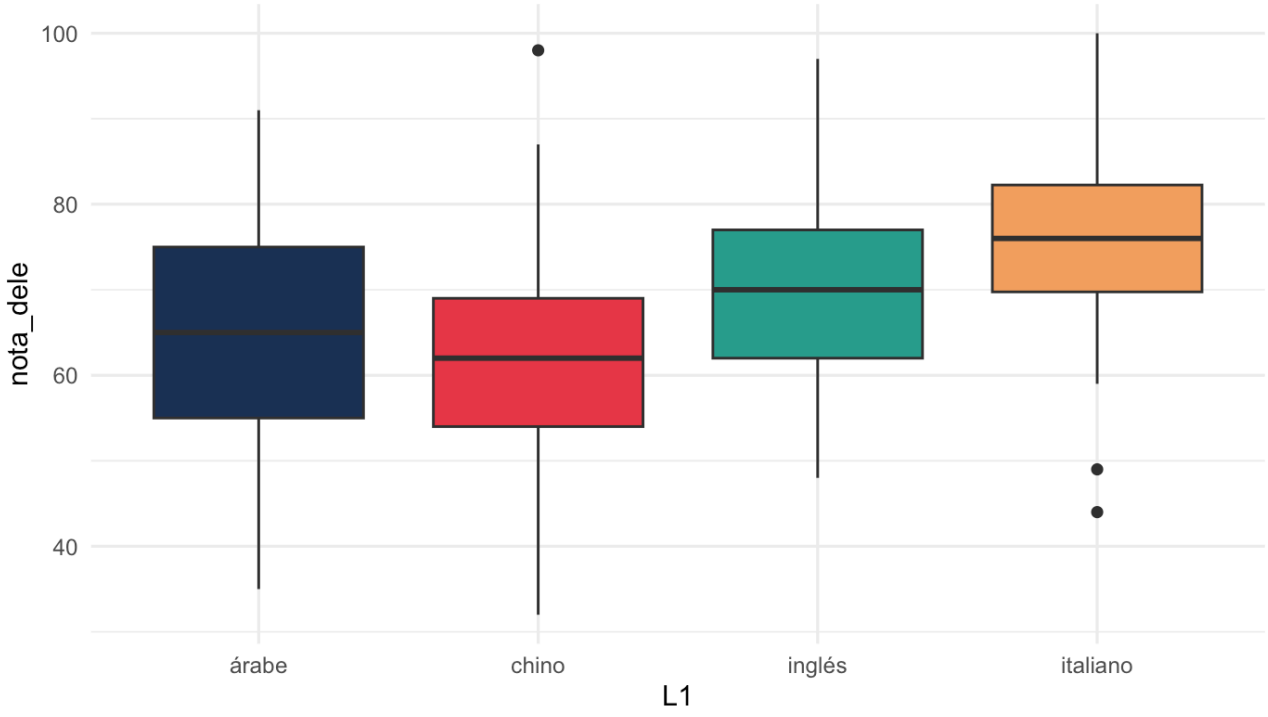
Densidad

```
ggplot(ele, aes(x = nota_dele,
                fill = estancia)) +
  geom_density(alpha = .6) +  # alpha: transparencia
  scale_fill_manual(
    values = c("#1D3557", "#E63946"))
```



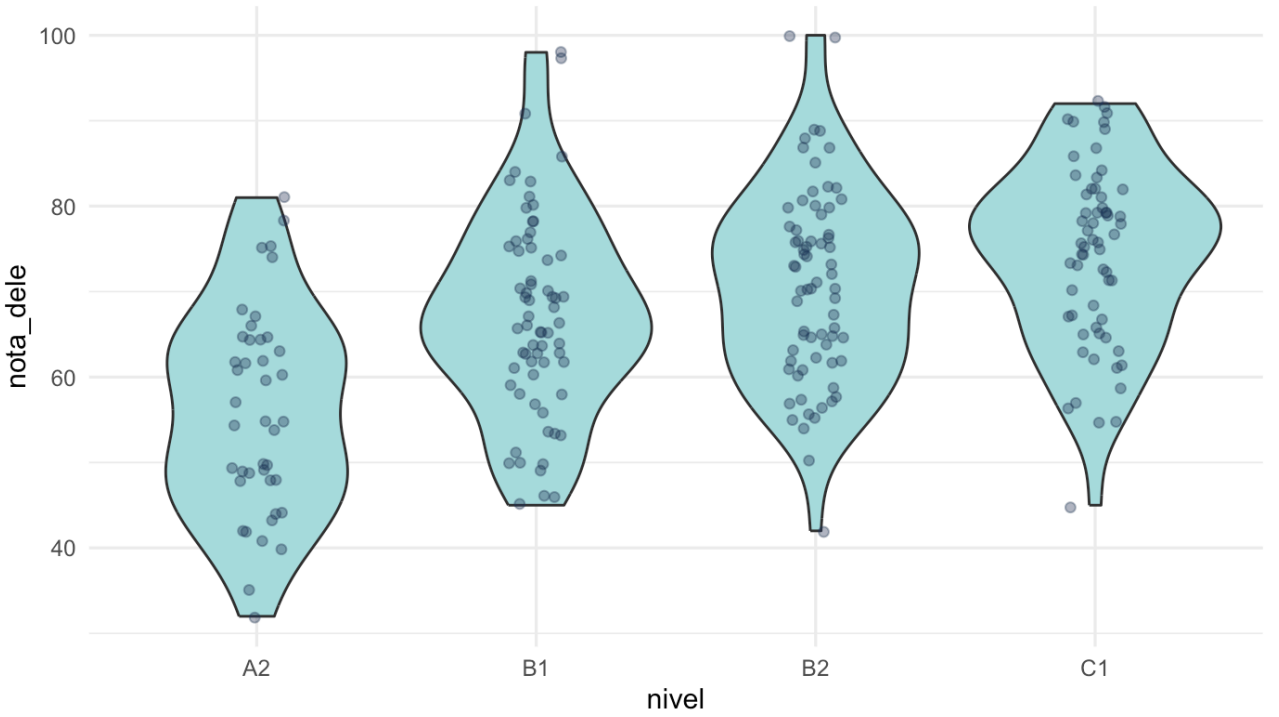
Boxplot

```
ggplot(ele, aes(x = L1, y = nota_dele,
                fill = L1)) +
  geom_boxplot(show.legend = FALSE) +
  scale_fill_manual(values = c(
    "#1D3557", "#E63946",
    "#2A9D8F", "#F4A261"))
```



Violín + puntos

```r
ggplot(ele, aes(x = nivel,
                y = nota_dele)) +
  geom_violin(fill = "#A8DADC") +
  geom_jitter(width = .1,  # desplazamiento lateral
              alpha = .4,  # transparencia
              colour = "#1D3557")
```



En pocas palabras

Las tres pestañas muestran formas distintas de comparar la **nota DELE** entre grupos:

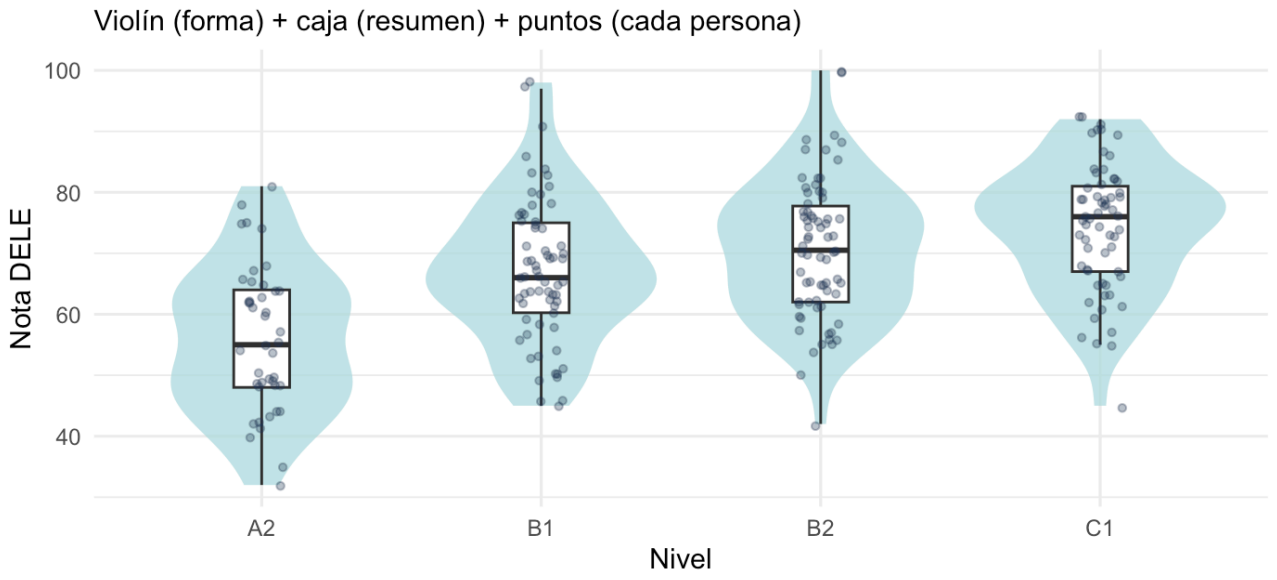
- **Densidad** (`geom_density`): una versión suavizada del histograma. En lugar de barras, dibuja una curva continua que sube donde hay muchas notas y baja donde hay pocas. El área bajo cada curva vale siempre 1, así que se pueden comparar grupos de distinto tamaño (140 aprendientes sin estancia y 100 con estancia): lo que se compara es la **forma**, no el número de personas. Aquí se ve que la curva roja (con estancia) está algo desplazada hacia notas más altas.
- **Boxplot** (diagrama de caja): resume cada L1 con su mediana (la línea), su mitad central (la caja, del Q1 al Q3) y su recorrido (los bigotes).
- **Violín con puntos**: el contorno muestra la forma de la distribución en cada nivel y cada punto es un aprendiente.

Dos argumentos que aparecen en el código:

- `alpha` es la **transparencia**, entre 0 (invisible) y 1 (totalmente opaco). Con `alpha = .6`, cada curva es opaca al 60 %, de modo que donde se superponen se ven las dos. Con `alpha = 1`, la curva roja taparía a la azul.
- `geom_jitter(width = .1)` dibuja los puntos con un pequeño desplazamiento horizontal al azar (*jitter*, «temblor»), para que los aprendientes con la misma nota no queden uno encima de otro. `width = .1` limita ese desplazamiento a 0,1 a cada lado (la distancia entre dos niveles del eje es 1), así que los puntos de B1 no se mezclan con los de B2. `alpha = .4` hace los puntos opacos al 40 %: donde se acumulan muchos, el color se oscurece.

Cada gráfico destaca algo distinto. Elegir uno u otro depende de lo que queramos que vea el lector.

2.6 Puntos, cajas y violines



En pocas palabras

Aquí se pueden activar y desactivar tres formas de dibujar los mismos datos, la **nota DELE por nivel** (o por L1), para ver qué aporta cada una.

- Los **puntos** muestran a cada persona. Son la representación más honesta, sobre todo con pocos datos.
- La **caja** resume: la línea central es la mediana, la caja abarca del Q1 al Q3 y los bigotes llegan hasta los valores no extremos (como mucho, 1,5 veces el RIC más allá de la caja). Los puntos aislados más allá son valores atípicos.
- El **violín** muestra la forma: es más ancho donde se acumulan más personas.

El código de la derecha se reescribe con las capas activas. En la capa de puntos aparece `geom_jitter(width = .1, alpha = .4)`:

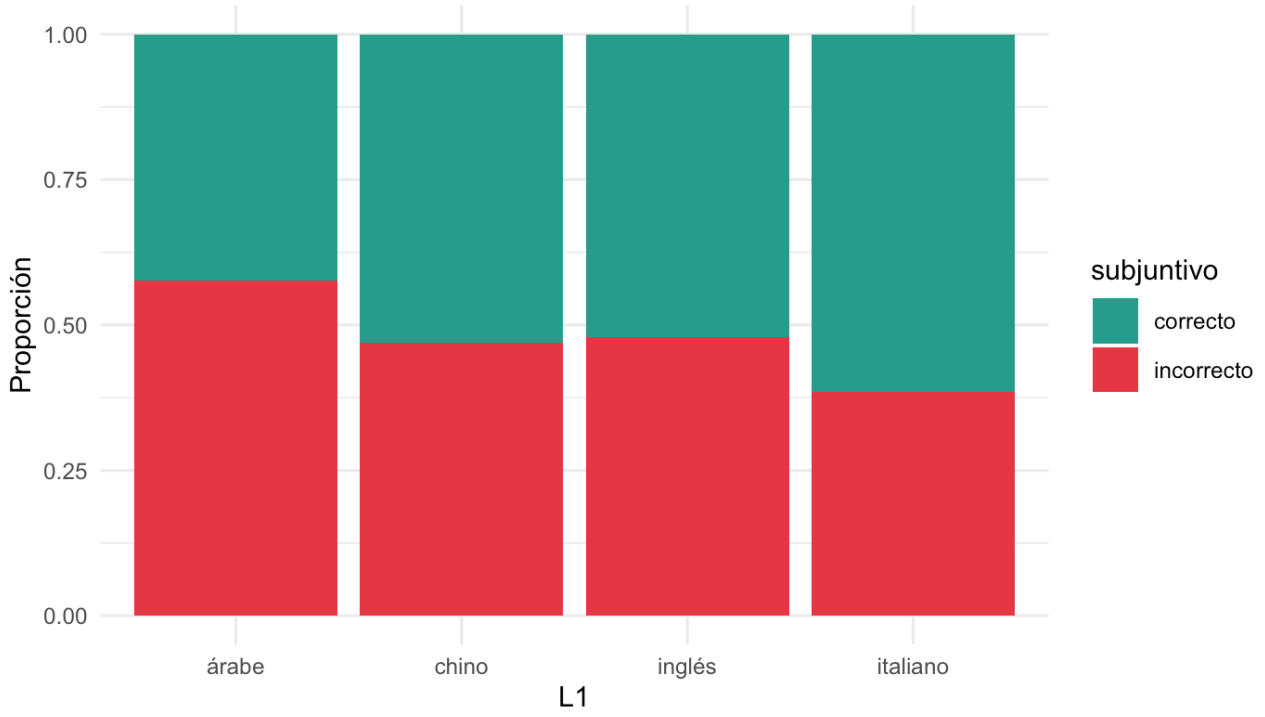
- `width = .1`: cada punto se desplaza al azar, como mucho, 0,1 unidades a izquierda o derecha de su nivel. Así no se apilan en una sola línea vertical los aprendientes con la misma nota, pero tampoco invaden el nivel vecino (que está a 1 unidad de distancia).
- `alpha = .4`: transparencia. Cada punto es opaco al 40 %; donde coinciden muchos aprendientes, el color se acumula y se ve más oscuro.

Combinarlos suele ser la mejor opción: la caja para el resumen y los puntos para no ocultar los datos reales.

2.7 Barras, dispersión y facetas

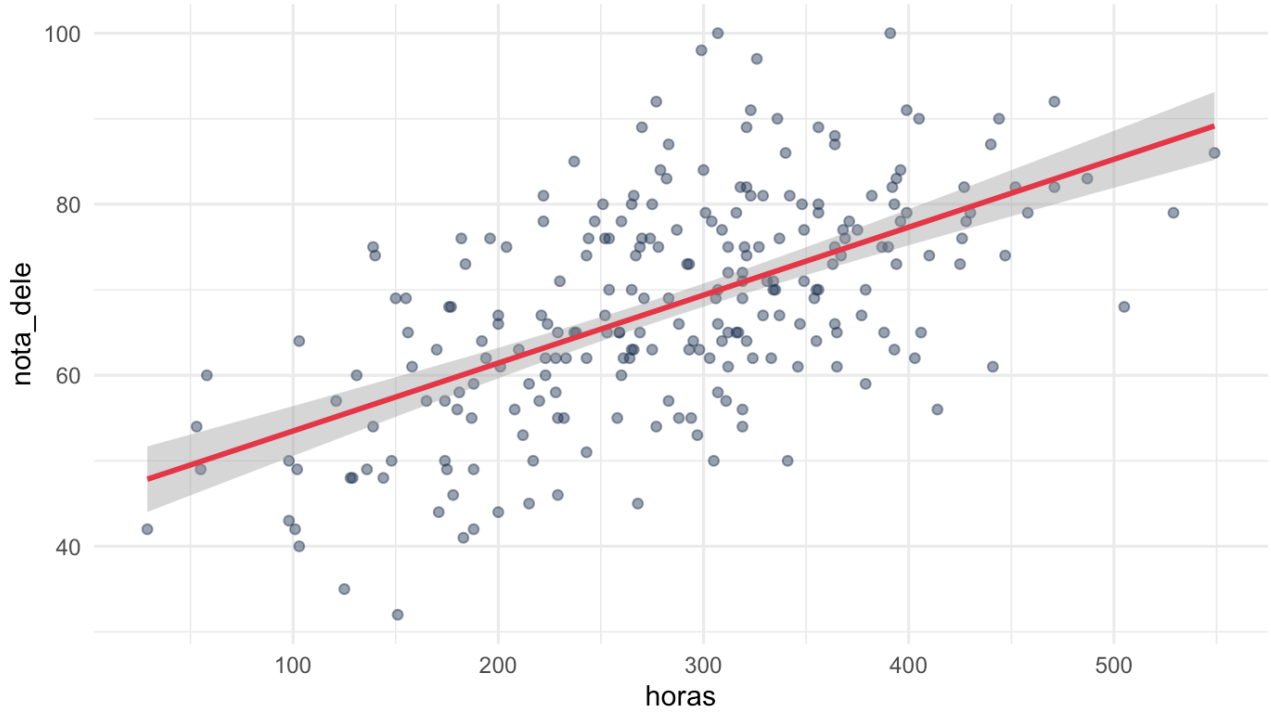
Barras

```
ggplot(ele, aes(x = L1,
                fill = subjuntivo)) +
  geom_bar(position = "fill") + # sin esto: recuentos
  labs(y = "Proporción") +
  scale_fill_manual(
    values = c("#2A9D8F", "#E63946"))
```



Dispersión

```
ggplot(ele, aes(x = horas,
                y = nota_dele)) +
  geom_point(alpha = .5,
             colour = "#1D3557") +
  geom_smooth(method = "lm",   # recta
              colour = "#E63946") # banda gris: IC 95 %
```



Facetas

```r
ggplot(ele, aes(horas, nota_dele,
                colour = estancia)) +
  geom_point(alpha = .6) +
  facet_wrap(~ L1, nrow = 2)
```



En pocas palabras

Tres gráficos más para otras combinaciones de variables:

- **Barras apiladas al 100 %**: para dos variables categóricas. Cada barra es una L1, y los colores muestran qué proporción acierta y falla el subjuntivo. **No es lo que hace `geom_bar()` por defecto**: por defecto cuenta personas (recuentos) y apila los colores, de modo que la barra del inglés (73 aprendientes) sería más alta que la del chino (49). Es `position = "fill"` lo que estira cada barra hasta 1 (el 100 %) y convierte los recuentos en proporciones **dentro de cada L1**. Así se comparan las lenguas aunque tengan distinto número de aprendientes. Si se borra ese argumento, se ven los recuentos.
- **Diagrama de dispersión**: para dos variables numéricas. Cada punto es un aprendiente, situado según sus horas y su nota. La línea roja resume la tendencia (más horas, más nota): es la recta de un modelo lineal (`method = "lm"`). La **zona gris** que la rodea es su **intervalo de confianza del 95 %**: el margen en el que probablemente estaría la recta si repitiéramos el estudio con otros 240 aprendientes. Es más ancha en los extremos porque allí hay menos datos (pocos aprendientes con menos de 100 o más de 450 horas) y porque un pequeño error en la inclinación de la recta se nota más cuanto más nos alejamos del centro. Con `se = FALSE` desaparece.
- **Facetas**: el mismo diagrama de dispersión, repetido en un panel por cada L1. Permite ver si la relación se mantiene en todos los grupos.

`facet_wrap(~ L1)` es la instrucción que divide el gráfico en paneles.

2.8 Convierte tu gráfico en interactivo

Paso 1 · Guarda el gráfico en un objeto

```r
p <- ggplot(ele, aes(horas, nota_dele, colour = L1,
                     text = paste("Aprendiente", id))) +
  geom_point(alpha = .7) +
  scale_colour_manual(values = c("#1D3557", "#E63946", "#2A9D8F", "#F4A261"))
```

Paso 2 · Una sola función: ggplotly()

```r
ggplotly(p)
```

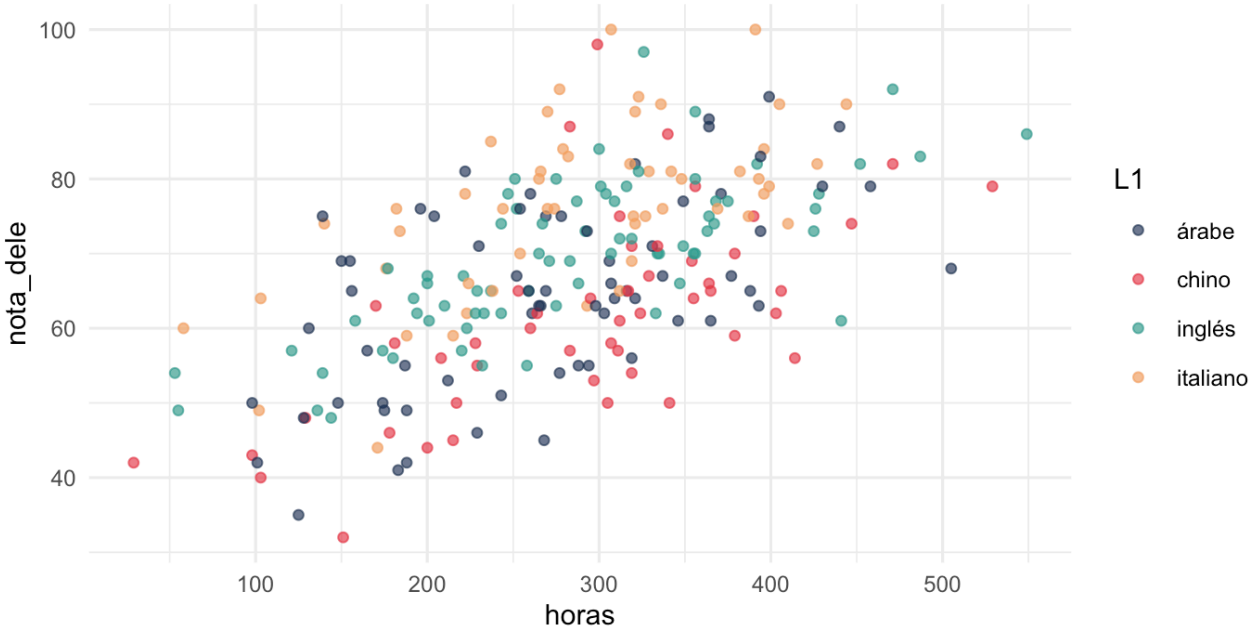
En pocas palabras

Cualquier gráfico de `ggplot2` se puede convertir en interactivo con una sola función.

1. Primero se guarda el gráfico en un objeto, aquí llamado `p`.
2. Después, `ggplotly(p)` lo transforma en un gráfico web interactivo.

El resultado se puede explorar en el navegador o en el panel de RStudio y se puede incluir en informes HTML o presentaciones como esta. Es muy útil para explorar los datos y localizar casos concretos.

2.9 Tu resultado: un gráfico que se puede explorar



En pocas palabras

Este es el gráfico interactivo que genera `ggplotly()`. Cada punto es un aprendiente, situado según sus horas de instrucción (eje horizontal) y su nota (eje vertical), con un color por lengua materna.

Se puede:

- **Pasar el ratón** por un punto para ver sus datos.
- **Arrastrar** para ampliar una zona y hacer **doble clic** para volver.
- **Pulsar en la leyenda** para ocultar o mostrar una L1.

Ocultar grupos ayuda a comparar: por ejemplo, dejar solo italiano y chino muestra que, con horas parecidas, los italohablantes tienden a estar más arriba.

2.10 Motion chart: los datos en movimiento

Paso 1 · Medias por nivel y L1

```
resumen <- ele |>
  group_by(nivel, L1) |>
  summarise(horas = mean(horas),
            nota = mean(nota_dele),
            n = n(), .groups = "drop")
head(resumen, 4)
#> # A tibble: 4 × 5
#>   nivel L1       horas  nota     n
#>   <fct> <chr>    <dbl> <dbl> <int>
#> 1 A2    chino     137.  45.2     8
#> 2 A2    inglés    181.  61.4    16
#> 3 A2    italiano  131.  55.8     5
#> 4 A2    árabe     172.  55.8    13
```

Paso 2 · Una burbuja por L1; el «tiempo» es el nivel

```
motion <- plot_ly(resumen, x = ~horas, y = ~nota,
    size = ~n, color = ~L1, frame = ~nivel, ids = ~L1,
    type = "scatter", mode = "markers",
    colors = c("#1D3557", "#E63946", "#2A9D8F", "#F4A261"),
    sizes = c(20, 70), marker = list(sizemode = "diameter")) |>
  layout(xaxis = list(range = c(80, 420), title = "Horas medias"),
         yaxis = list(range = c(40, 85), title = "Nota media")) |>
  animation_opts(frame = 1300, transition = 900, redraw = FALSE)
motion
```

En pocas palabras

Un ***motion chart*** (gráfico en movimiento) añade el tiempo, o cualquier otra secuencia, a un diagrama de dispersión. Lo popularizó Hans Rosling para mostrar la evolución de los países.

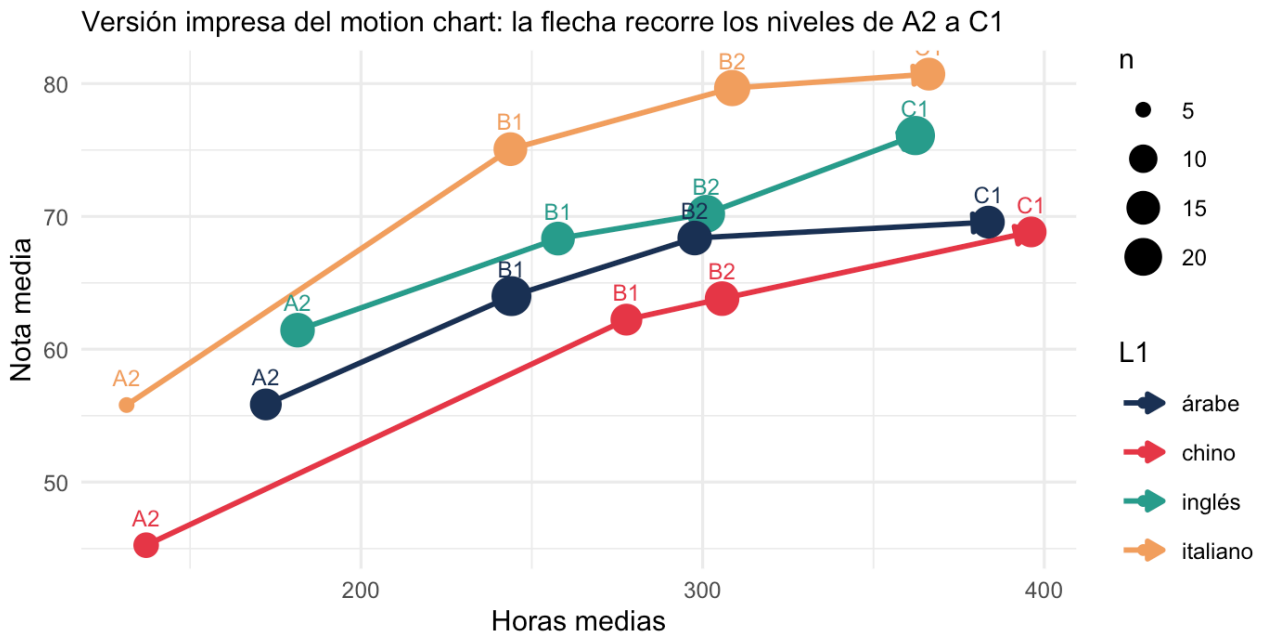
Aquí hay dos pasos:

1. Se resumen los datos: para cada combinación de nivel y L1 se calculan las horas medias, la nota media y cuántas personas hay.
2. Con `plot_ly()` se dibuja una burbuja por L1. La posición indica horas y nota; el tamaño, el número de personas; y `frame = ~nivel` hace que el gráfico avance de A2 a C1 como si fuera una animación.

Qué hace cada argumento del código:

- **`frame = ~nivel`** (dentro de `plot_ly`): la variable que define los **fotogramas** de la animación. Cada nivel (A2, B1, B2, C1) es un fotograma, como las páginas de un folioscopio.
- **`ids = ~L1`**: indica que cada burbuja es la misma lengua en todos los fotogramas, para que se desplace en lugar de aparecer y desaparecer.
- **`size = ~n`** y **`sizes = c(20, 70)`**: el tamaño de la burbuja depende del número de aprendientes del grupo; la más pequeña medirá 20 píxeles y la más grande, 70.
- **`marker = list(sizemode = "diameter")`**: ese tamaño se interpreta como el **diámetro** de la burbuja. La alternativa, `"area"`, haría proporcional el área.
- **`layout(xaxis = list(range = c(80, 420)))`**: fija los límites de los ejes para que no cambien de un fotograma a otro.
- **`animation_opts(frame = 1300, transition = 900, redraw = FALSE)`**: controla el ritmo. Aquí `frame` (¡otro sentido de la misma palabra!) es el tiempo que dura cada fotograma en milisegundos: 1300 ms = 1,3 segundos por nivel. `transition` es lo que tarda el movimiento de un fotograma al siguiente: 900 ms. `redraw = FALSE` evita redibujar el gráfico entero en cada paso; como solo se mueven las burbujas, la animación es más fluida.

2.11 Tu resultado: el motion chart



En pocas palabras

Al pulsar **Play**, el gráfico recorre los niveles de A2 a C1 y cada burbuja se desplaza. También se puede arrastrar el deslizador inferior para detenerse en un nivel concreto.

Cada burbuja es una lengua materna. Su posición horizontal indica cuántas horas de clase acumula ese grupo, y la vertical, qué nota media obtiene.

Al avanzar de nivel, todas las burbujas se mueven hacia arriba y hacia la derecha: más horas y más nota. Lo interesante es comparar trayectorias: unas lenguas avanzan por delante de otras durante todo el recorrido.

2.12 Cómo leer el motion chart

- **Cada burbuja**: Una lengua materna
- **Posición**: Horas medias (x) y nota media (y) de ese grupo
- **Tamaño**: Cuántos aprendientes hay en ese grupo
- **Movimiento**: Qué cambia al pasar de A2 a C1

Para recordar

- Todas las burbujas avanzan **hacia arriba y a la derecha**: más nivel → más horas y más nota.
- El **orden** se mantiene: inglés e italiano van por delante; chino y árabe, por detrás, con las mismas horas.
- Un motion chart responde a la pregunta **«¿cómo cambia la relación?»**, no solo «¿cuál es la relación?».

En pocas palabras

Este apartado resume cómo interpretar el gráfico anterior.

Cada elemento codifica una variable: la **burbuja** es una lengua materna, su **posición** indica horas y nota medias, su **tamaño** indica cuántas personas hay en el grupo y el **movimiento** muestra el paso de un nivel a otro.

Las conclusiones principales son tres:

- Todas las lenguas progresan en la misma dirección: más nivel implica más horas y más nota.
- El orden se mantiene: inglés e italiano van por delante; chino y árabe, por detrás, incluso con un número similar de horas.
- Este tipo de gráfico no solo muestra una relación, sino **cómo cambia** esa relación.

2.13 ¿Qué gráfico elijo?

Variable 1	Variable 2	Gráfico	Código de ggplot2
Numérica	—	Histograma o densidad	`geom_histogram()`, `geom_density()`
Categórica	—	Barras	`geom_bar()`
Numérica	Categórica	Boxplot o violín (+ puntos)	`geom_boxplot()`, `geom_violin()`, `geom_jitter()`
Numérica	Numérica	Dispersión	`geom_point()` + `geom_smooth()`
Categórica	Categórica	Barras apiladas al 100 % o mosaico	`geom_bar(position = "fill")`

Variable 1	Variable 2	Gráfico	Código de ggplot2
Categórica	Categórica con muchas celdas	Mapa de calor	`geom_tile()`

En pocas palabras

Elegir el gráfico adecuado depende, sobre todo, del **tipo de las variables** que queremos representar.

La herramienta lo resuelve en tres pasos: se indica si hay una o dos variables y de qué tipo son (numérica o categórica), y propone el gráfico recomendado, lo dibuja con nuestros datos y muestra el código de `ggplot2` listo para copiar.

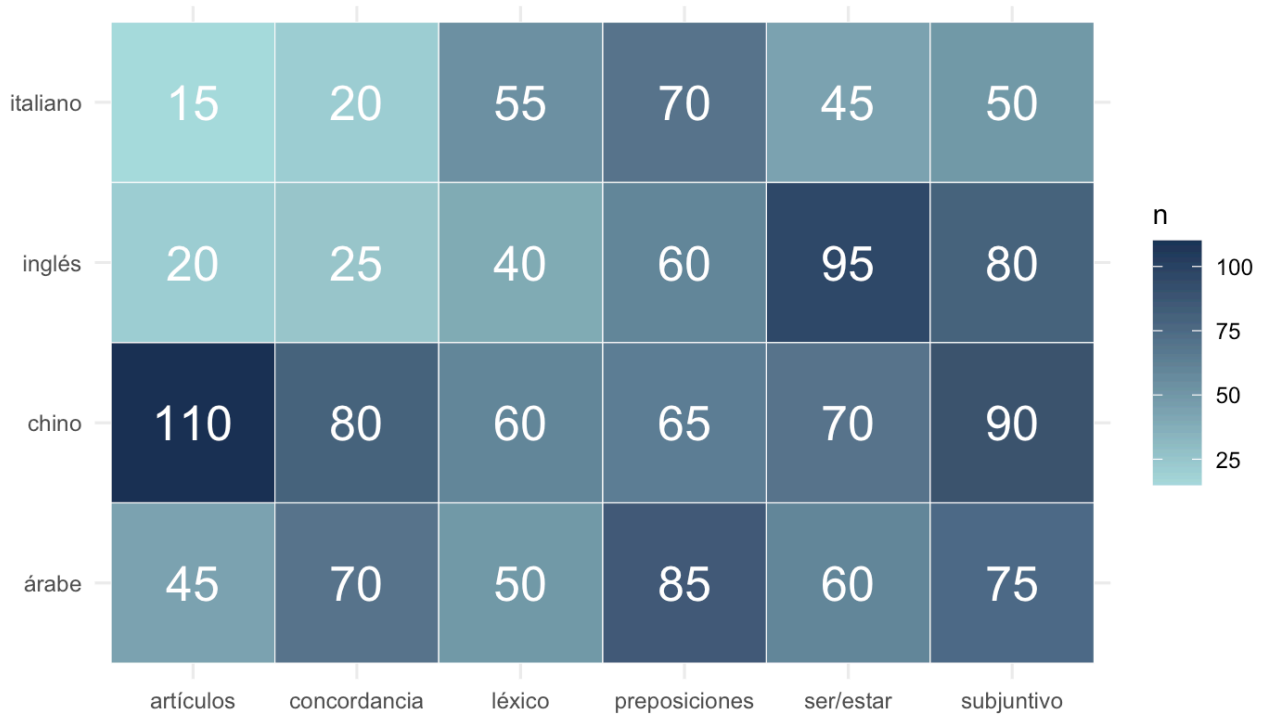
Algunas reglas generales:

- Una numérica: histograma o densidad.
- Una numérica por grupos: boxplot o violín.
- Dos numéricas: diagrama de dispersión.
- Dos categóricas: barras o mapa de calor.

2.14 Mapas de calor

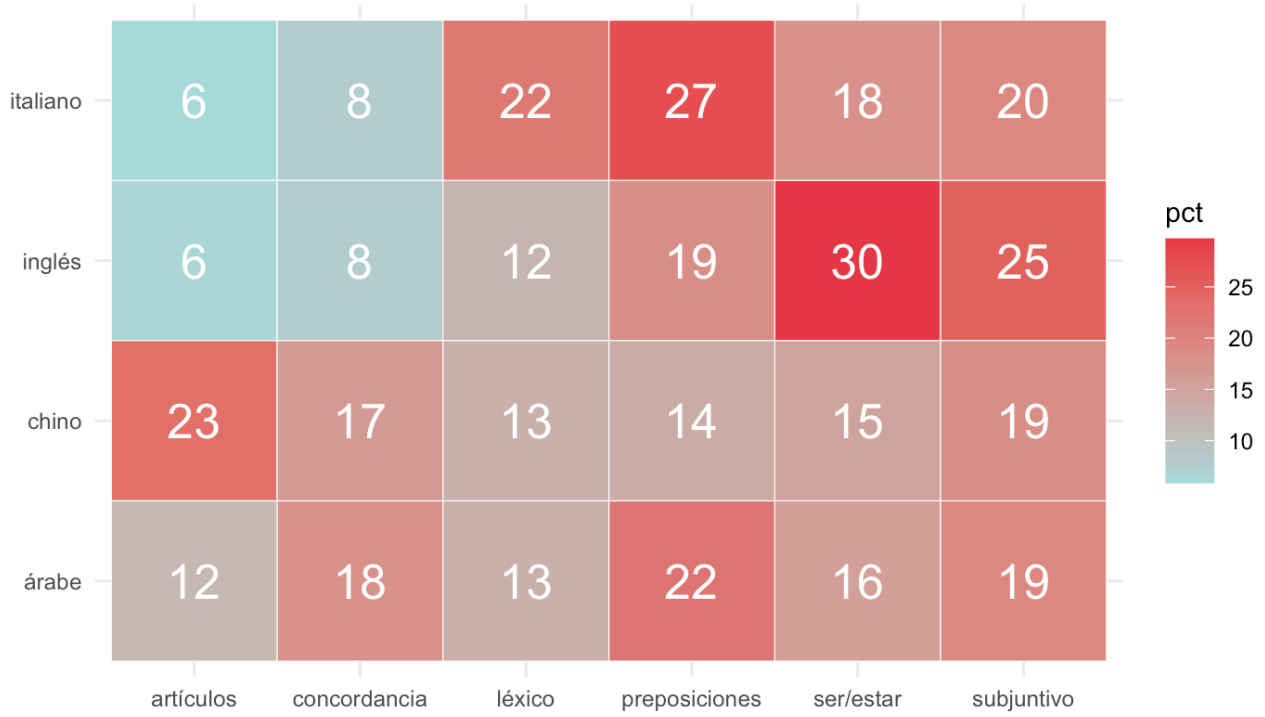
Recuentos

```
ggplot(errores, aes(x = tipo, y = L1,
                    fill = n)) +
  geom_tile(colour = "white") +
  geom_text(aes(label = n),
            colour = "white", size = 6) +
  scale_fill_gradient(low = "#A8DADC",
                      high = "#1D3557") +
  labs(x = NULL, y = NULL)
```



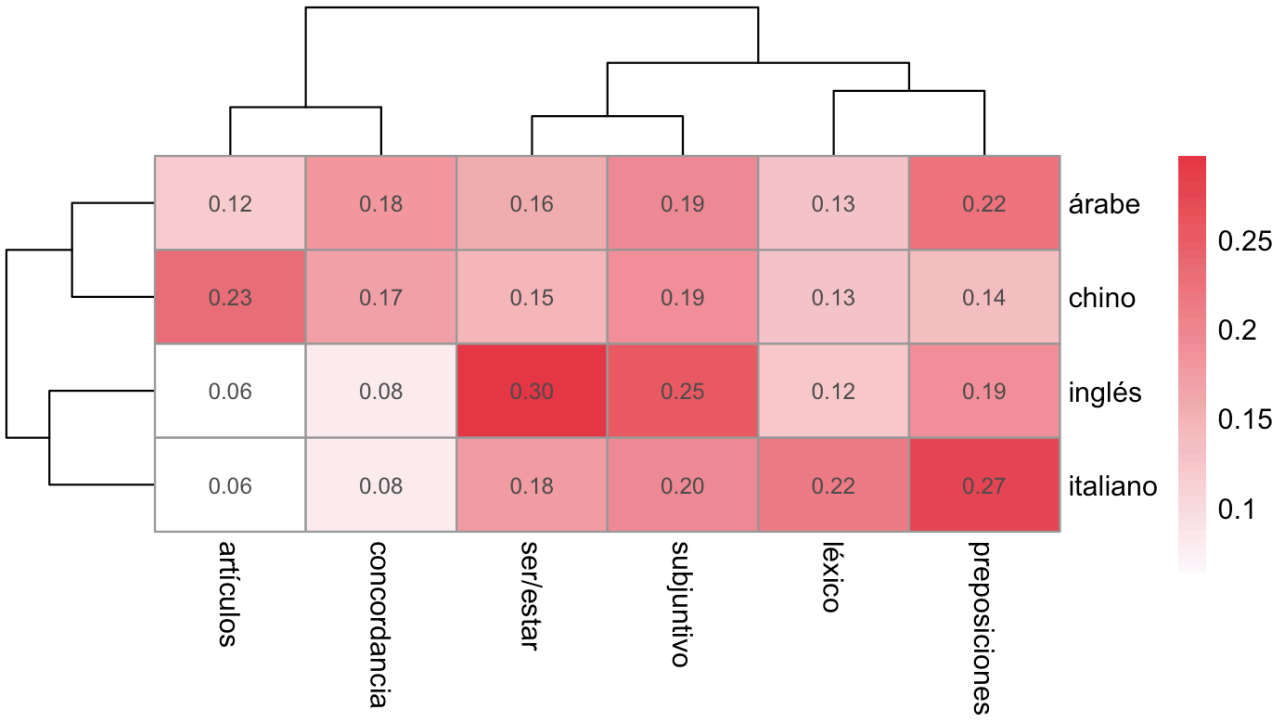
Porcentaje por fila

```
errores |>
  group_by(L1) |>
  mutate(pct = 100 * n / sum(n)) |>
  ggplot(aes(tipo, L1, fill = pct)) +
  geom_tile(colour = "white") +
  geom_text(aes(label = round(pct)),
            colour = "white", size = 6) +
  scale_fill_gradient(low = "#A8DADC",
                      high = "#E63946") +
  labs(x = NULL, y = NULL)
```



Con agrupación (pheatmap)

```
m <- xtabs(n ~ L1 + tipo, errores)
pheatmap::pheatmap(
  prop.table(m, 1),
  display_numbers = TRUE,
  color = colorRampPalette(
    c("white", "#E63946"))(50))
```



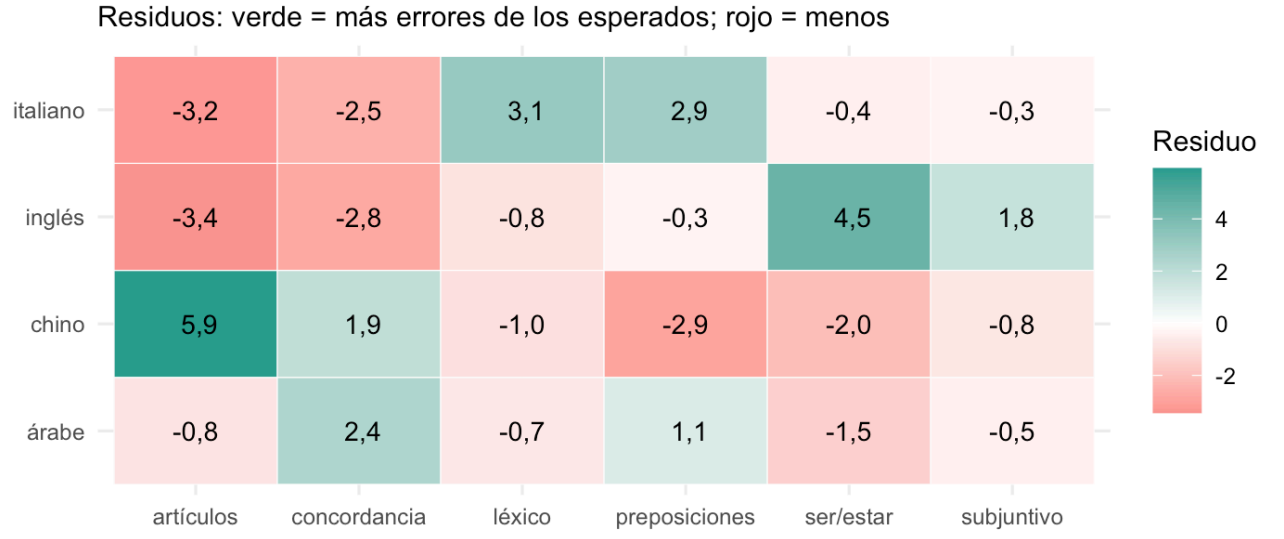
En pocas palabras

Un **mapa de calor** es una tabla en la que cada celda se colorea según su valor: cuanto más intenso el color, mayor el número. Es ideal para cruzar dos variables categóricas, como la L1 y el tipo de error.

Las tres pestañas muestran tres versiones:

- **Recuentos**: el número bruto de errores de cada tipo en cada lengua.
- **Porcentaje por fila**: qué parte de los errores de cada lengua corresponde a cada tipo. Así se comparan perfiles aunque unas lenguas tengan más textos que otras.
- **Con agrupación** (`pheatmap`): reordena filas y columnas para acercar las que se parecen, y dibuja a los lados unos árboles que muestran esas semejanzas.

2.15 Leer un mapa de calor



En pocas palabras

El mismo mapa de calor puede contar historias distintas según lo que se represente en cada celda.

- Con **recuentos**, el chino parece tener más errores de todo tipo, pero puede deberse simplemente a que hay más textos de sinohablantes.
- Con **porcentajes por fila**, se compara el perfil de cada lengua: qué tipos de error predominan.
- Con **residuos**, se ve dónde hay más errores de lo esperado (verde) o menos (rojo) si la L1 no influyera. Este concepto se explica en la sesión 4.

Lo más llamativo es la acumulación de errores de **artículos** en los sinohablantes, coherente con que el chino no tiene artículos.

2.16 Buenas prácticas visuales

- **Un mensaje**: Un gráfico = una idea. Ponla en el título.
- **Etiquetas**: Ejes con nombre y unidades: `labs()`.
- **Color accesible**: Evita rojo y verde juntos como único código.
- **Muestra los datos**: Con pocos datos, puntos mejor que barras.
- **Nada de tartas ni 3D**: Comparamos peor ángulos y volúmenes que longitudes.
- **Guarda en alta resolución**: `ggsave("fig.png", width = 8, height = 5, dpi = 300)`

En pocas palabras

Seis consejos para que un gráfico comunique bien:

- **Un mensaje**: cada gráfico debe transmitir una idea, y es útil ponerla en el título.
- **Etiquetas**: los ejes deben tener nombre y unidades. Por defecto R usa el nombre de la variable, que no siempre se entiende.
- **Color accesible**: alrededor del 8 % de los hombres no distingue bien el rojo del verde. No conviene usarlos como único código.
- **Mostrar los datos**: con pocos casos, los puntos informan más que una barra con la media.
- **Evitar tartas y 3D**: el ojo compara mal ángulos y volúmenes, y bien longitudes.
- **Alta resolución**: `ggsave()` guarda el gráfico con la calidad necesaria para una publicación.

2.17 Práctica 2 (20 min)

1. Haz un histograma de `horas`.
2. Haz un boxplot de `err_ser_estar` por `L1`. ¿Qué grupo comete más errores?
3. Relaciona `horas` y `nota_dele` en un gráfico de dispersión coloreado por `nivel`.
4. Conviértelo en interactivo con `ggplotly()`.
5. Haz un motion chart de `err_ser_estar` medio frente a `horas` medias, con `frame = ~nivel`.
6. Construye el mapa de calor de `errores.csv`, con el número de errores escrito en cada celda, y guárdalo con `ggsave()`.

2.18 Práctica 2: soluciones

Soluciones de la práctica 2

```
ggplot(ele, aes(horas)) + geom_histogram(binwidth = 25)
ggplot(ele, aes(L1, err_ser_estar)) + geom_boxplot()
p2 <- ggplot(ele, aes(horas, nota_dele, colour = nivel)) + geom_point()
ggplotly(p2)
```

```r
ele |> group_by(nivel, L1) |>
  summarise(h = mean(horas), e = mean(err_ser_estar), n = n()) |>
  plot_ly(x = ~h, y = ~e, size = ~n, color = ~L1, frame = ~nivel, ids = ~L1,
          type = "scatter", mode = "markers")
p3 <- ggplot(errores, aes(tipo, L1, fill = n)) +
  geom_tile(colour = "white") +
  geom_text(aes(label = n), colour = "white", size = 6)   # el número en cada celda
ggsave("mapa_calor.png", p3, width = 8, height = 4)
```

En pocas palabras

Una posible solución para cada ejercicio de la práctica 2:

1. Histograma de las horas, con tramos de 25.
2. Boxplot de los errores de *ser/estar* por L1: los sinohablantes cometen más errores y los italohablantes, menos.
3. Diagrama de dispersión de horas y nota, coloreado por nivel, guardado en `p2`.
4. `ggplotly(p2)` lo vuelve interactivo.
5. El *motion chart* de errores frente a horas se construye igual que el de la sesión: primero se resume por nivel y L1 y después se anima con `frame = ~nivel`.
6. El mapa de calor: `geom_tile()` dibuja las celdas y `geom_text(aes(label = n))` escribe en cada una su número de errores, como en el mapa de calor de la sesión. Se guarda como imagen con `ggsave()`, indicando el ancho y el alto en pulgadas.

3 Estadística inferencial

Sesión 3 · 60 minutos

> ℹ La pregunta de esta sesión
>
> ¿Influye la estancia en el extranjero en la nota? ¿Y cómo lo sabemos con solo 240 personas?

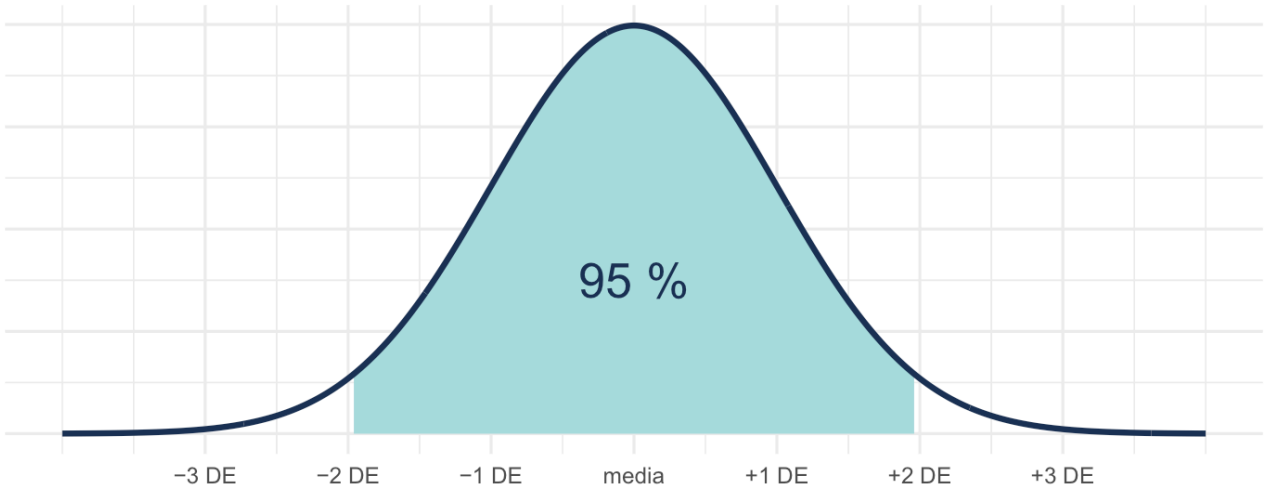
Contenidos: Distribución normal · Distribución muestral · Intervalos de confianza · Valor p y potencia · Árbol para elegir la prueba · t · Wilcoxon · correlación.

3.1 Descriptiva frente a inferencial

- **Descriptiva**: ¿Qué pasa **en mi muestra**? *La nota media de mis 240 aprendientes es 68,1.*
- **Inferencial**: ¿Qué puedo decir **de la población**? *La nota media de los aprendientes está, con un 95 % de confianza, entre 66,4 y 69,7.*

> En pocas palabras
>
> Hasta ahora hemos hecho **estadística descriptiva**: describir lo que ocurre en nuestros 240 aprendientes. La nota media de este grupo concreto es 68,1 y eso es un hecho.
>
> La **estadística inferencial** da un paso más. Usa la muestra para decir algo sobre la **población**: todos los aprendientes parecidos a los nuestros. Como no los hemos medido a todos, la respuesta no puede ser exacta. Por eso se expresa con un margen de incertidumbre: «la nota media de la población está, con un 95 % de confianza, entre 66,4 y 69,7».
>
> Toda esta sesión trata de cómo calcular y entender ese margen.

3.2 La distribución normal



En pocas palabras

La **distribución normal** es la famosa curva en forma de campana. Describe muchas variables reales: la mayoría de los valores se concentra en el centro y, cuanto más nos alejamos, menos casos hay, de forma simétrica a ambos lados.

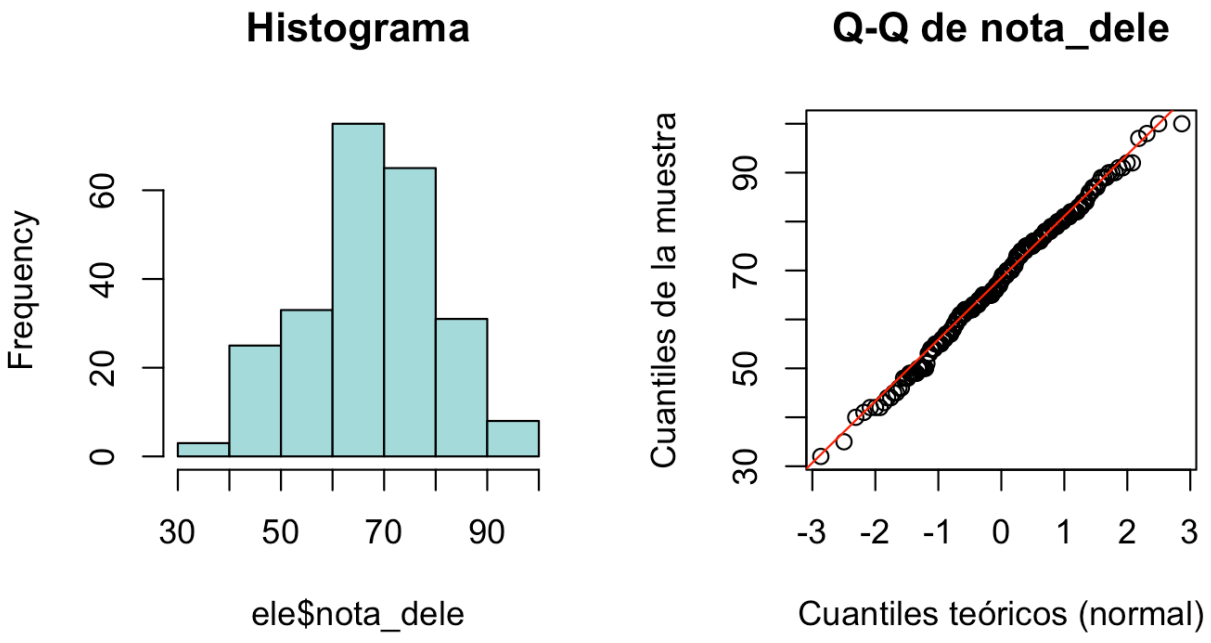
Queda definida por dos números: la **media**, que la sitúa, y la **desviación típica**, que la hace más estrecha o más ancha.

Tiene una propiedad muy útil: en cualquier curva normal, el **95 %** de los valores está a menos de 1,96 desviaciones típicas de la media. Este número reaparecerá en los intervalos de confianza. Al superponer nuestras notas reales se ve que se parecen bastante a una normal.

3.3 ¿Son normales mis datos?

Gráficos

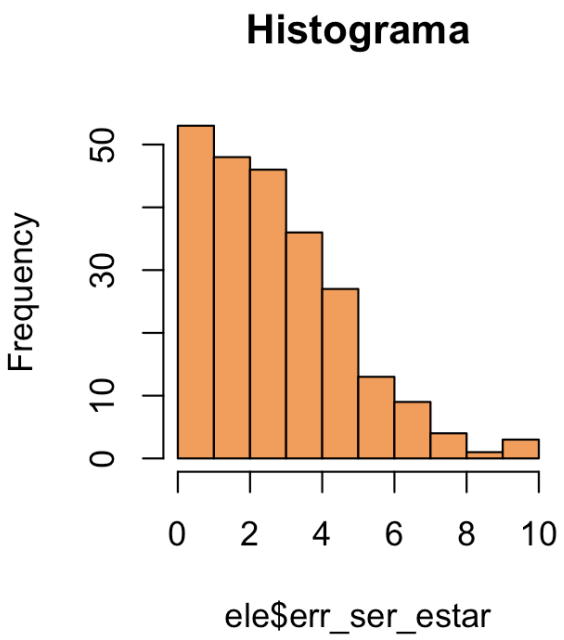
```
par(mfrow = c(1, 2))
hist(ele$nota_dele, col = "#A8DADC",
     main = "Histograma")
qqnorm(ele$nota_dele,
  main = "Q-Q de nota_dele",
  xlab = "Cuantiles teóricos (normal)",
  ylab = "Cuantiles de la muestra")
qqline(ele$nota_dele, col = "red")
```

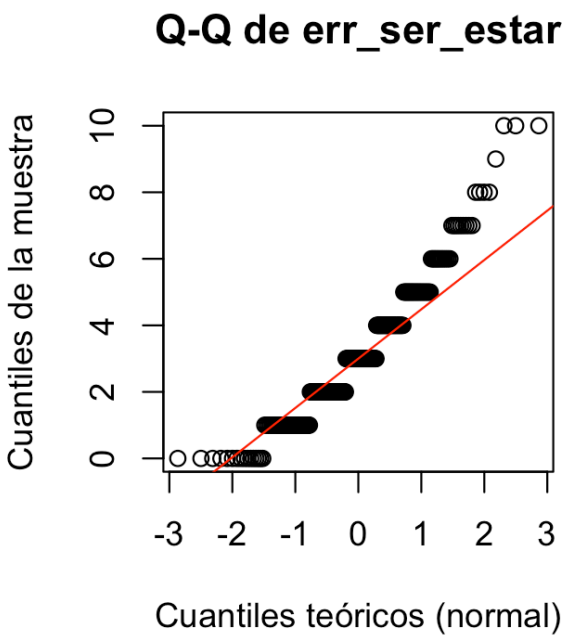


Q-Q de un recuento

```
par(mfrow = c(1, 2))
hist(ele$err_ser_estar, col = "#F4A261",
     main = "Histograma")
qqnorm(ele$err_ser_estar,
  main = "Q-Q de err_ser_estar",
```

```
  xlab = "Cuantiles teóricos (normal)",
  ylab = "Cuantiles de la muestra")
qqline(ele$err_ser_estar, col = "red")
```





Shapiro-Wilk

```
shapiro.test(ele$nota_dele)
#>
#>  Shapiro-Wilk normality test
#>
#> data:  ele$nota_dele
#> W = 0.99408, p-value = 0.4669
shapiro.test(ele$err_ser_estar)
#>
#>  Shapiro-Wilk normality test
#>
#> data:  ele$err_ser_estar
#> W = 0.93937, p-value = 2.083e-08
```

- p > 0,05 → no hay evidencia contra la normalidad
- p < 0,05 → los datos **no** son normales (los recuentos rara vez lo son)

En pocas palabras

Muchas pruebas estadísticas suponen que los datos siguen una distribución normal, así que conviene comprobarlo. Hay dos formas:

- **Gráficos**: el histograma debe tener forma de campana. En el **gráfico Q-Q**, si los datos son normales, los puntos se alinean sobre la recta roja.
- **Prueba de Shapiro-Wilk**: si su valor p es mayor que 0,05, no hay motivos para pensar que los datos no sean normales.

Cómo se construye y se lee un gráfico Q-Q («cuantil-cuantil»). Cada punto es un aprendiente:

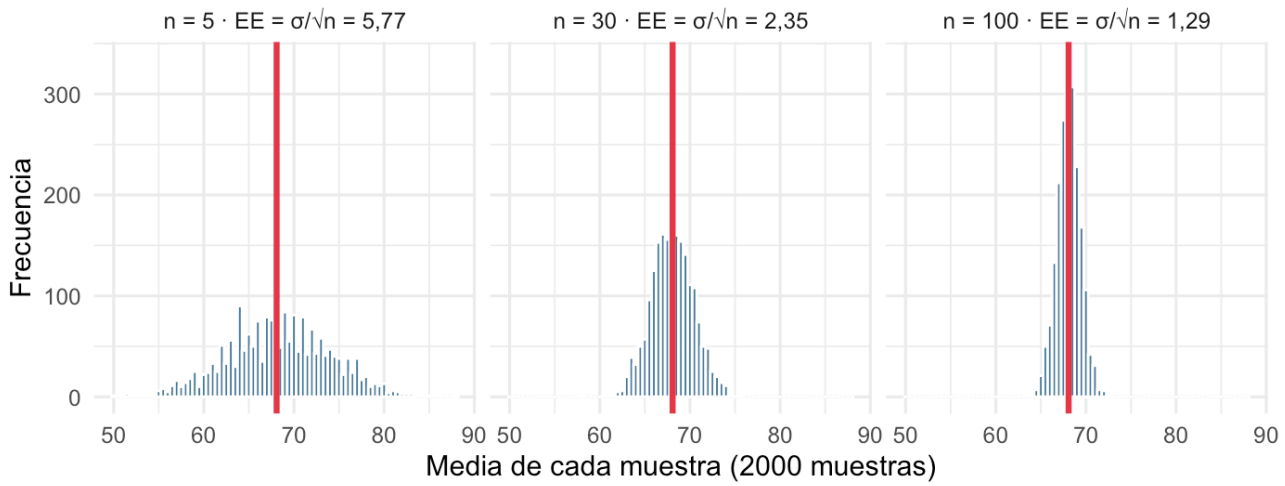
1. Se ordenan las 240 notas de menor a mayor. Esos valores son los **cuantiles de la muestra** (*sample quantiles*) y van en el eje vertical: la nota más baja es 32, la central es 68 y la más alta, 100.
2. Para cada posición se calcula qué valor le correspondería si los datos siguieran una **normal perfecta**. Son los **cuantiles teóricos** (*theoretical quantiles*), en el eje horizontal, y se expresan en desviaciones típicas desde el centro: el aprendiente que ocupa el puesto 1 de 240 debería estar unas 2,9 DE por debajo de la media (–2,9); el del centro, en 0; el último, en +2,9.
3. Si cada nota real está donde la normal predice, los puntos forman una línea recta (la roja). Así, el punto de abajo a la izquierda es el aprendiente con 32 puntos, situado en x = –2,9.

Cuando los puntos se apartan de la recta, la forma en que lo hacen tiene una lectura concreta: si los extremos se curvan hacia fuera, hay más valores extremos de los que tendría una normal; si los puntos forman escalones, la variable solo toma unos pocos valores enteros.

La pestaña **Q-Q de un recuento** muestra un caso no normal: los errores de *ser/estar* solo toman valores enteros (0, 1, 2…), así que los puntos forman **escalones**, y el histograma está inclinado hacia la izquierda.

En nuestro caso, las notas parecen normales (p = 0,47). Los errores de *ser/estar*, en cambio, no lo son (p < 0,001), algo habitual en los recuentos: muchos aprendientes cometen pocos errores y unos pocos cometen muchos.

3.4 De la muestra a la población



En pocas palabras

Esta simulación muestra la idea central de la inferencia.

Cada vez que se extrae una muestra, su media cae en la parte inferior del gráfico. Si se repite mil veces, esas medias forman una campana: la **distribución muestral**. Su centro coincide con la media real de la población.

La anchura de esa campana se llama **error estándar** (EE) e indica cuánto suele equivocarse la media de una muestra. Depende del tamaño de la muestra: con 4 personas la campana es ancha (las medias varían mucho); con 100, estrecha.

El panel muestra dos cifras que deberían parecerse:

- **DE de las medias**: la desviación típica de todas las medias simuladas. Es el error estándar *medido* con la simulación.
- **Error estándar teórico σ/√n**: el mismo valor, *calculado* con una fórmula sin necesidad de simular. **σ** (sigma) es la desviación típica de la población: aquí, las 240 notas, con σ = 12,9 puntos. **n** es el tamaño de cada muestra.

Ejemplo con n = 30: EE = 12,9 / √30 = 12,9 / 5,48 ≈ **2,35 puntos**. Significa que la media de una muestra de 30 aprendientes suele alejarse unos 2,4 puntos de la media real (68,1), y casi nunca más de unos 4,7 (el doble). Con n = 120: 12,9 / √120 ≈ 1,17 puntos, la mitad.

Como n está dentro de una raíz cuadrada, para reducir el error a la mitad hay que **cuadruplicar** la muestra (de 30 a 120). En la vida real no conocemos σ y usamos en su lugar la DE de nuestra muestra, como veremos en el apartado siguiente.

3.5 Error estándar e intervalo de confianza

Paso 1 · El IC del 95 % a mano

```
x <- ele$nota_dele
ee <- sd(x) / sqrt(length(x))       # error estándar: 12,9 / √240
ee
#> [1] 0.832435
c(media = mean(x), inf = mean(x) - 1.96 * ee, sup = mean(x) + 1.96 * ee)
#>    media      inf      sup
#> 68.07917 66.44759 69.71074
```

Paso 2 · Lo mismo con t.test()

```
t.test(ele$nota_dele)$conf.int      # $conf.int: solo el intervalo
#> [1] 66.43932 69.71901
#> attr(,"conf.level")
#> [1] 0.95
```

En pocas palabras

Un **intervalo de confianza** (IC) es un rango de valores en el que, con cierta seguridad, se encuentra el valor real de la población. Aquí queremos saber la nota media de **todos** los aprendientes parecidos a los nuestros, no solo de estos 240.

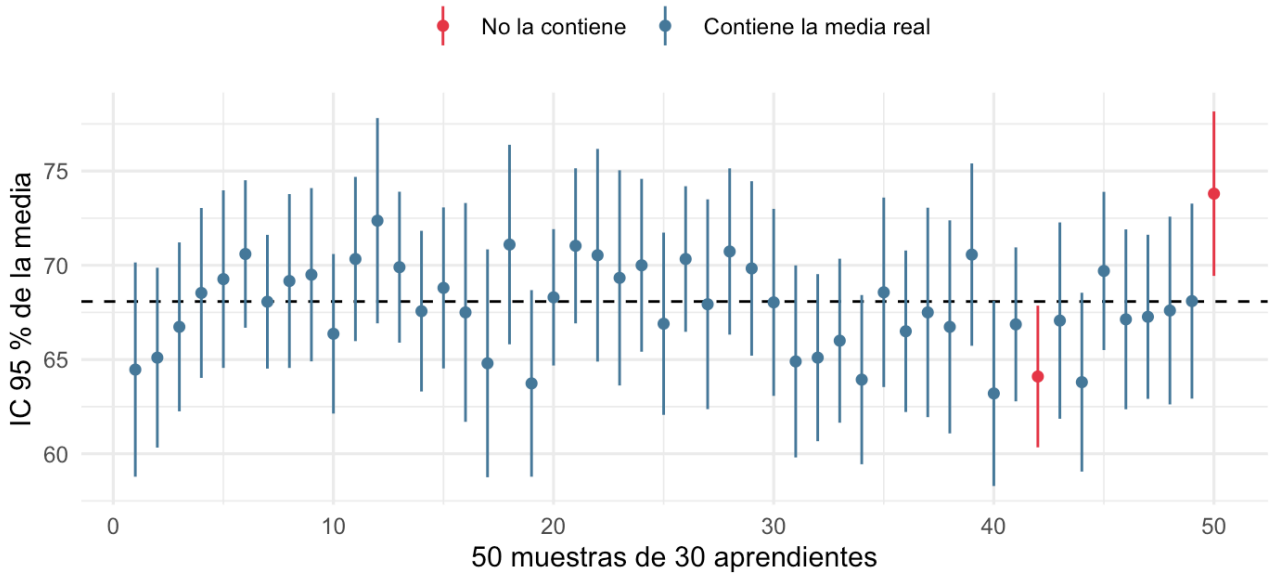
Paso 1, a mano, con nuestras notas:

1. **Error estándar**: la desviación típica dividida entre la raíz del número de personas. EE = 12,9 / √240 = 12,9 / 15,5 ≈ **0,83 puntos**. Es el margen típico de error de una media calculada con 240 personas: mucho menor que la DE (12,9), que mide cuánto difieren las *personas* entre sí.
2. **Intervalo del 95 %**: la media ± 1,96 errores estándar. 1,96 × 0,83 ≈ 1,63, así que el intervalo va de 68,1 − 1,6 = **66,4** a 68,1 + 1,6 = **69,7** puntos. El 1,96 sale de la distribución normal: el 95 % de sus valores queda a menos de 1,96 DE del centro.

Paso 2, con `t.test()`. La función `t.test()` sirve para muchas cosas: si le damos una sola variable, contrasta si su media es 0 (algo que aquí no nos interesa) y, de paso, calcula el intervalo de confianza de la media. Con `$conf.int` extraemos solo ese intervalo. El resultado (66,44 a 69,72) es casi idéntico al nuestro, con una diferencia: en lugar de 1,96 usa 1,97, un multiplicador de la **distribución t** con 239 grados de libertad. La t es una versión algo más prudente de la normal, porque tiene en cuenta que la DE también se ha estimado con la muestra. Con muestras pequeñas la diferencia importa más: con 30 personas el multiplicador sería 2,05.

En la práctica se usa esta segunda forma, que es más exacta y más corta.

3.6 Qué significa un IC del 95 %



En pocas palabras

Esta simulación responde a la pregunta más difícil del tema: ¿qué quiere decir exactamente «95 % de confianza»?

Qué se simula. De nuevo tratamos a los 240 aprendientes como la población, cuya nota media real (68,1) conocemos: es la línea discontinua horizontal. Cada barra vertical es un investigador distinto que toma su propia muestra de 30 aprendientes, calcula su media (el punto) y su intervalo del 95 % (la barra), igual que en el apartado anterior. Por ejemplo, un investigador obtiene una media de 66,0 y un intervalo de 61,2 a 70,8: su intervalo **contiene** 68,1 y acierta. Otro obtiene 73,5, con un intervalo de 69,0 a 78,0: deja fuera el 68,1 y **falla** (barra roja).

Qué se observa. La mayoría de los intervalos contienen la media real. Unos pocos, en rojo, la dejan fuera. Si se repite muchas veces, fallan aproximadamente **5 de cada 100**: con 50 muestras, unas 2 o 3.

Qué significa, entonces, el 95 %. No que haya un 95 % de probabilidad de que el valor esté en *este* intervalo concreto: una vez calculado, el intervalo contiene o no contiene la media real. El 95 % describe **el método**: si muchos investigadores lo aplican, el 95 % de sus intervalos acertarán. Por eso, cuando decimos que la nota media está «entre 66,4 y 69,7 con un 95 % de confianza», queremos decir que hemos usado un procedimiento que acierta 19 de cada 20 veces.

Qué cambia la anchura del intervalo. El panel muestra la semianchura media (el «±»):

- **La confianza**: al bajarla al 80 %, los intervalos se estrechan, pero fallan más (unos 10 de cada 50). Al subirla al 99 %, se ensanchan y casi nunca fallan.
- **El tamaño de la muestra**: con n = 30 el intervalo es de unos ±4,8 puntos; con n = 120, de unos ±2,3. Aumentar la muestra los estrecha **sin** fallar más. Es la mejor forma de ganar precisión.

3.7 La lógica del contraste de hipótesis

- **H_0 · hipótesis nula**: No hay diferencia o no hay relación *La estancia no influye en la nota.*
- **H_1 · hipótesis alternativa**: Sí la hay *Quienes han hecho una estancia obtienen otra nota.*

p = P(datos tan extremos | H_0 cierta)

! Idea clave

El valor p **no** es la probabilidad de que H_0 sea cierta, ni mide la **importancia** del efecto.

⚠ En una frase

El valor p responde a esta pregunta: **si H_0 fuera cierta, ¿qué probabilidad habría de obtener unos datos tan extremos como los nuestros?** Si esa probabilidad es muy baja (normalmente, menos de 0,05), rechazamos H_0.

En pocas palabras

Un **contraste de hipótesis** funciona como un juicio en el que se presume la inocencia.

- La **hipótesis nula (H_0)** es la postura por defecto: no hay diferencia ni relación. Por ejemplo, la estancia no influye en la nota.
- La **hipótesis alternativa (H_1)** es lo que queremos demostrar: sí la hay.

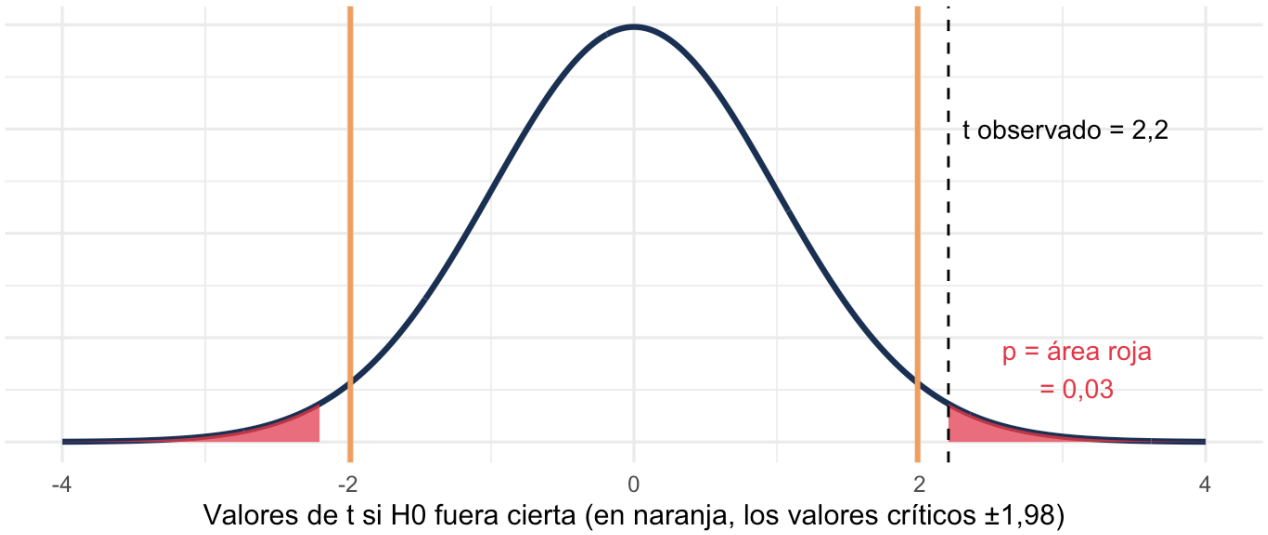
El **valor p** responde a esta pregunta: si H_0 fuera cierta, ¿qué probabilidad habría de obtener unos datos tan extremos como los nuestros? Si esa probabilidad es muy baja (normalmente, menos de 0,05), rechazamos H_0.

En la fórmula, la barra vertical «|» se lee «suponiendo que»: p es la probabilidad de unos datos tan extremos **suponiendo que** H_0 es cierta.

Un ejemplo con nuestros datos, por simulación. Quienes han hecho una estancia sacan de media 5,1 puntos más (71,1 frente a 65,9). Para saber si eso puede ser casualidad, podemos fabricar un mundo en el que H_0 es cierta: barajamos al azar las etiquetas «sí» y «no» de la estancia entre los 240 aprendientes (así la estancia ya no puede tener ninguna relación con la nota) y volvemos a calcular la diferencia. Repetido 10 000 veces, una diferencia de 5,1 puntos o más (en cualquier dirección) solo aparece unas **35 veces**: $p \approx 0{,}0035$. Es tan raro que concluimos que la diferencia real no se debe al azar. La prueba t de la sesión llega a la misma conclusión con una fórmula ($p = 0{,}002$).

El valor p **no** es la probabilidad de que H_0 sea cierta, ni indica si la diferencia es grande o importante.

3.8 El valor p, visualmente



En pocas palabras

La curva muestra cómo se distribuiría el estadístico **t** si **no hubiera ninguna diferencia** real entre quienes han hecho estancia y quienes no: casi siempre cerca de 0 y raramente lejos. (El estadístico t mide la diferencia entre dos medias en unidades de error estándar; lo veremos en detalle con la prueba t.)

El valor p es el **área roja**: la proporción de esa curva que queda más lejos que nuestro resultado. Cuanto más extremo es el t obtenido, más pequeña es el área y menor es p.

Las líneas naranjas marcan los **valores críticos**: el punto a partir del cual el área que queda más allá es exactamente del 5 %. Si nuestro t los supera, p es menor que 0,05 y el resultado se considera significativo.

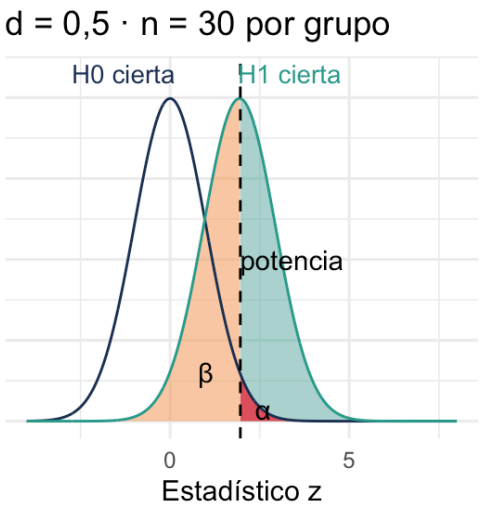
Bilateral o unilateral depende de cómo se formule la hipótesis alternativa *antes* de mirar los datos:

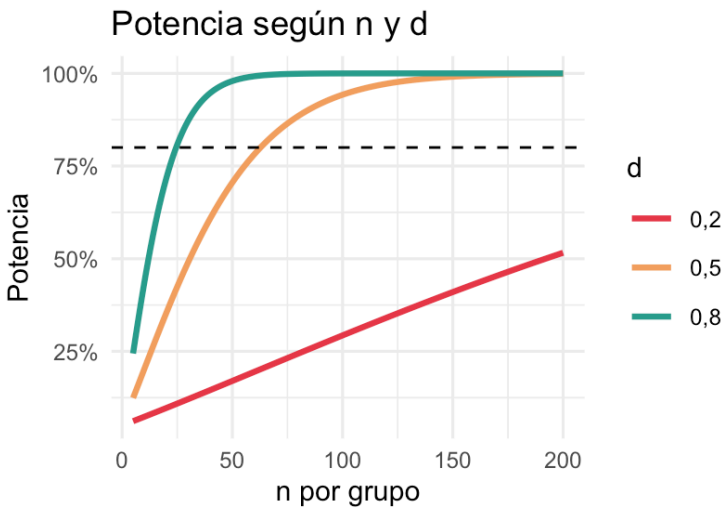
- **Bilateral** (la opción por defecto en R). H_1: «la nota es **distinta** con estancia», sin decir si mayor o menor. Una diferencia sorprendente puede aparecer en cualquiera de las dos direcciones, así que se suma el área roja de **los dos extremos** de la curva. Con nuestros datos, t = 3,08 y p = 0,0024 (0,0012 en cada cola). Los valores críticos son −1,97 y +1,97.
- **Unilateral**. H_1: «la nota es **mayor** con estancia». Solo nos interesa una dirección, así que se cuenta el área de **un solo extremo**: p = 0,0012, la mitad. El valor crítico baja a 1,65, así que es más fácil obtener un resultado significativo… pero a cambio renunciamos a detectar un efecto en la dirección contraria. Solo es legítimo si la dirección se decidió antes de ver los datos. En R: `t.test(nota_dele ~ estancia, data = ele, alternative = "less")` («less» porque R resta no – sí y esperamos que salga negativo).

El botón «Nuestro t» muestra el valor que obtendremos al comparar la nota según la estancia.

3.9 Errores tipo I y II, y potencia

	H_0 es verdadera	H_0 es falsa
Rechazo H_0	Error tipo I (α) «falsa alarma»	Potencia (1 − β)
No rechazo H_0	Acierto	Error tipo II (β) «se me escapa»





En pocas palabras

Al decidir con datos podemos equivocarnos de dos maneras, como muestra la tabla:

- **Error tipo I**, una falsa alarma: concluir que hay un efecto cuando no lo hay. Su probabilidad es α, normalmente un 5 %.
- **Error tipo II**, que algo se nos escape: no detectar un efecto que sí existe. Su probabilidad es β.

La **potencia** (1 − β) es la probabilidad de detectar un efecto real. Se suele buscar al menos un 80 %.

Qué es el tamaño del efecto (d) del deslizador. Es lo grande que es la diferencia real entre los dos grupos, medida en desviaciones típicas: *d* = diferencia de medias / DE. Como la DE de la nota DELE es de unos 13 puntos, d = 0,5 equivale a una diferencia real de unos 6,5 puntos entre grupos; d = 1,5, a unos 20 puntos. Nuestro efecto de la estancia es d ≈ 0,41 (5,1 puntos).

Cómo leer el gráfico. La curva azul es lo que saldría si H_0 fuera cierta (no hay diferencia); la verde, lo que saldría si la diferencia real fuera d. La línea discontinua es el valor crítico: a su derecha, rechazamos H_0.

- **Rojo (α)**: la parte de la curva azul más allá del valor crítico: falsas alarmas.
- **Naranja (β)**: la parte de la curva verde que no llega al valor crítico: efectos reales que se nos escapan.
- **Verde (potencia)**: la parte de la curva verde que sí lo supera.

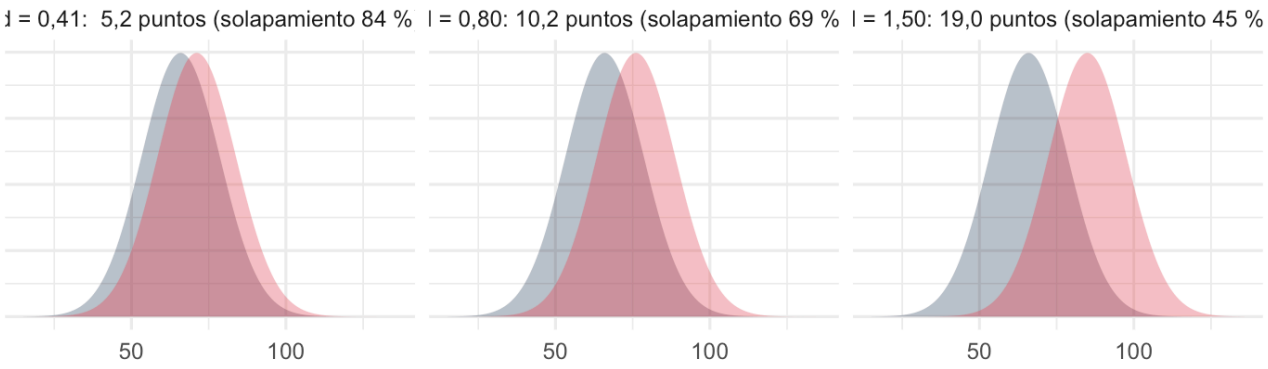
Dos dudas frecuentes:

- *¿Por qué β no es el 95 % cuando α es el 5 %?* Porque no son complementarios: α se calcula en el mundo sin efecto (curva azul) y β en el mundo con efecto (curva verde). β depende del tamaño del efecto y de n. Solo cuando el efecto se acerca a 0 las dos curvas se superponen, la potencia baja hasta α (5 %) y β sube hasta el 95 %. Con d = 0,1 y 30 por grupo, por ejemplo, β ≈ 93 %: un efecto tan pequeño casi nunca se detecta.
- *¿Qué significa «hacen falta unos 7 aprendientes por grupo»?* El deslizador «n por grupo» indica cuántas personas **tienes**; la frase calcula cuántas **necesitarías** como mínimo para alcanzar un 80 % de potencia con ese tamaño del efecto. Con d = 1,5 (unos 20 puntos de diferencia), el efecto es tan grande que bastan 7 aprendientes por grupo para detectarlo 8 de cada 10 veces; con tus 30 por grupo, la potencia es prácticamente del 100 %. Con d = 0,5 harían falta unos 63 por grupo, y con d = 0,2, unos 393.

En el gráfico, al reducir α hay menos falsas alarmas, pero aumenta β. La forma de mejorar ambas a la vez es **aumentar la muestra**: las dos curvas se separan.

3.10 Tamaño del efecto: la nota DELE con y sin estancia

Medida	Prueba	Pequeño	Mediano	Grande	En L2 (Plonsky y Oswald, 2014)
d de Cohen	t	0,2	0,5	0,8	0,4 · 0,7 · 1,0
r	correlación	0,1	0,3	0,5	0,25 · 0,40 · 0,60
V de Cramér	χ^2	0,1	0,3	0,5	—
η^2	ANOVA	0,01	0,06	0,14	—



Nota DELE (azul: sin estancia; rojo: con estancia, desplazada d desviaciones típicas)

En pocas palabras

Un resultado puede ser estadísticamente significativo y, aun así, irrelevante en la práctica. Con muestras muy grandes, diferencias minúsculas producen valores p pequeños. Por eso, además del valor p, hay que informar del **tamaño del efecto**: cuánto difieren los grupos o qué fuerza tiene la relación. A diferencia del valor p, el tamaño del efecto **no depende de cuántas personas tengamos**.

Cada prueba tiene su medida, y la tabla ofrece las referencias habituales de efecto pequeño, mediano y grande. Las de Cohen son generales; la última columna recoge las que propusieron Plonsky y Oswald (2014) tras revisar cientos de estudios de adquisición de segundas lenguas, donde los efectos suelen ser algo mayores. Son **convenciones orientativas**, no leyes: un efecto «pequeño» puede ser muy relevante si es barato de conseguir.

El caso del gráfico. La variable es la **nota DELE** y los grupos son quienes **no** han hecho una estancia (curva azul) y quienes **sí** (curva roja). El eje está en puntos de nota.

La ***d* de Cohen** expresa la diferencia entre dos medias en desviaciones típicas: d = (71,1 − 65,9) / 12,7 ≈ **0,41**. Es decir, la diferencia de 5,1 puntos equivale a 0,41 veces lo que varían las personas dentro de cada grupo. Según Cohen, es un efecto entre pequeño (0,2) y mediano (0,5); según las referencias de L2, pequeño (0,4). Sería **grande** a partir de 0,8 (Cohen) o de 1,0 (L2): una diferencia de unos 10 a 13 puntos de nota.

El panel traduce d a tres cifras más intuitivas (con d = 0,41):

- **Solapamiento del 84 %**: la proporción de área que comparten las dos curvas. La mayor parte de las notas de ambos grupos está en la misma zona: muchos aprendientes sin estancia sacan más nota que muchos con estancia. La diferencia es real, pero no separa a los grupos.
- **El 66 % del grupo con estancia supera la media del grupo sin estancia**. Si no hubiera ningún efecto, sería el 50 %.
- **Probabilidad de superioridad del 61 %**: si elegimos al azar a un aprendiente con estancia y a otro sin estancia, el primero saca más nota 61 de cada 100 veces (50 de cada 100 si no hubiera efecto).

Con d = 0,2 las dos curvas casi se solapan (92 %); con d = 0,8 la separación ya es evidente (69 % de solapamiento y un 79 % del grupo rojo por encima de la media del azul).

3.11 Árbol de decisión: ¿qué prueba uso?

Variable respuesta y situación	Prueba	En R	Efecto
Numérica: 2 grupos distintos, datos normales (o n > 30)	t de Welch	`t.test(y ~ g)`	d de Cohen
Numérica: 2 grupos distintos, no normales	Wilcoxon (Mann-Whitney)	`wilcox.test(y ~ g)`	r

Variable respuesta y situación	Prueba	En R	Efecto
Numérica: Mismas personas dos veces, normal	t para muestras relacionadas	`t.test(post, pre, paired = TRUE)`	d
Numérica: Mismas personas dos veces, no normal	Wilcoxon de rangos con signo	`wilcox.test(post, pre, paired = TRUE)`	r
Numérica: 3 o más grupos, normal y varianzas parecidas	ANOVA + Tukey	`aov()`, `TukeyHSD()`	η^2
Numérica: 3 o más grupos, no normal	Kruskal-Wallis	`kruskal.test()`	ε^2
Numérica: Relación con otra numérica, lineal y normal	Correlación de Pearson	`cor.test(x, y)`	r, r^2
Numérica: Relación con otra numérica, ordinal o con extremos	Correlación de Spearman	`cor.test(x, y, method = "spearman")`	ρ
Numérica: Predecir a partir de uno o varios predictores	Regresión lineal (o árbol de regresión)	`lm()`	R^2
Categórica: Asociación entre dos categóricas, esperadas ≥ 5	Chi cuadrado	`chisq.test()`	V de Cramér
Categórica: Asociación con celdas pequeñas	Prueba exacta de Fisher	`fisher.test()`	odds ratio
Categórica: Predecir con varios predictores y cuantificar efectos	Regresión logística	`glm(family = binomial)`	odds ratios
Categórica: Predecir con reglas fáciles de explicar	Árbol de decisión	`rpart()`	acierto
Frecuencias: Una palabra en dos corpus	Log-likelihood + log-ratio	`ll()`, `log2()`	log--ratio

En pocas palabras

Este diagrama ayuda a elegir la prueba estadística respondiendo a una serie de preguntas sobre los datos:

1. ¿La variable que quiero explicar es numérica o categórica?
2. ¿Comparo grupos o relaciono variables?
3. ¿Cuántos grupos hay?
4. ¿Son las mismas personas (muestras relacionadas) o personas distintas (independientes)?
5. ¿Los datos son aproximadamente normales?

Por ejemplo: «¿difiere la nota según la estancia?» → numérica → comparar grupos → dos → independientes → normal → **prueba t**.

Se puede recorrer con los botones o pulsando en cualquier punto del diagrama. Al final del curso veremos árboles que construye R a partir de los datos.

3.12 Prueba t: ficha

- **¿Qué pregunta responde?**: ¿Difieren las **medias** de una variable numérica entre **2 grupos**? *¿Sacan más nota quienes han hecho una estancia?*
- **Hipótesis**: H_0: $\mu_1 = \mu_2$ H_1: $\mu_1 \neq \mu_2$
- **Supuestos**: · Variable numérica · Aproximadamente **normal** en cada grupo (o n > 30) · Observaciones **independientes** · R usa por defecto la versión de **Welch**
- **Estadístico**: $t = (\bar{x}_1 - \bar{x}_2) / EE$ = **señal / ruido** |t| grande → p pequeño

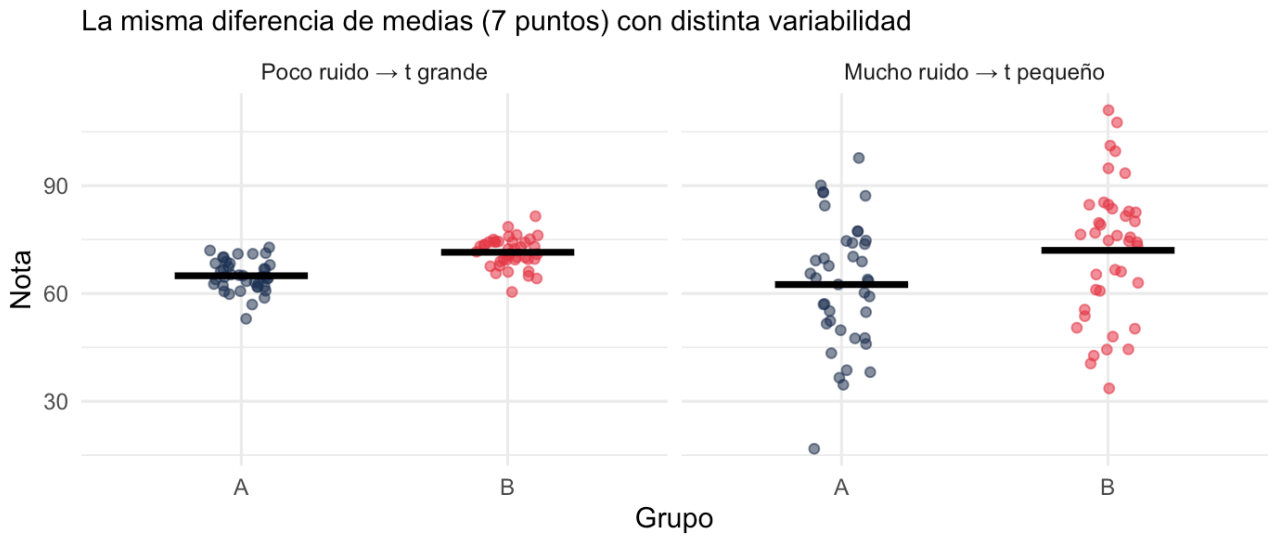
En pocas palabras

La **prueba t** compara las **medias de dos grupos** en una variable numérica. Nuestra pregunta: ¿sacan más nota quienes han hecho una estancia?

- **Hipótesis nula**: las dos medias son iguales en la población.
- **Supuestos**: la variable es numérica, los datos de cada grupo son aproximadamente normales (o hay más de 30 personas por grupo) y cada persona aparece una sola vez.
- **Estadístico**: t divide la diferencia entre medias por su error estándar. Es una relación entre **señal** (la diferencia) y **ruido** (la variabilidad).

R usa por defecto la versión de **Welch**, que no exige que los dos grupos tengan la misma dispersión.

3.13 La idea de t: señal frente a ruido



En pocas palabras

La simulación genera la **nota** de dos grupos de aprendientes, A y B (piensa, por ejemplo, en sin estancia y con estancia). Los deslizadores controlan tres cosas: cuánto difieren de verdad los grupos, cuánto varían las personas dentro de cada grupo y cuántas personas hay.

El valor t compara dos cosas: la **señal**, que es la diferencia entre las medias, y el **ruido**, que es la incertidumbre con que medimos esa diferencia (su error estándar, que depende de la variabilidad dentro de los grupos y del número de personas):

t = señal / ruido = diferencia de medias / EE de la diferencia

Ejemplo con los valores iniciales: diferencia de 7 puntos, DE de 10 dentro de cada grupo y 40 personas por grupo. El error estándar de la diferencia es $10 \times \sqrt{(1/40 + 1/40)} \approx 2{,}24$ puntos, así que $t \approx 7 / 2{,}24 \approx$ **3,1**. La diferencia es unas tres veces mayor que su margen de error típico. (El panel muestra los valores exactos de la simulación, que se apartan un poco porque las notas se generan al azar.)

La barra de la derecha representa |t| (el valor de t sin signo). La línea discontinua es el **valor crítico**: el valor de t a partir del cual $p < 0{,}05$. Con 40 personas por grupo está en torno a 1,99. Es un listón:

- Si la barra **supera** el listón (se vuelve verde), $p < 0{,}05$: la diferencia es significativa.
- Si **no llega** (roja), la diferencia observada podría deberse al azar.

Prueba estos cambios:

- Con **poco ruido** (DE = 3), los grupos apenas se solapan y la diferencia es clara: t es enorme.
- Con **mucho ruido** (DE = 25), los grupos se superponen y esa misma diferencia de 7 puntos podría deberse al azar: t baja por debajo del listón.
- **Aumentar la muestra** también hace crecer t, porque reduce el error estándar. Si la diferencia es 0, t es cercano a 0 y p es grande.

3.14 Prueba t: ¿influye la estancia en la nota?

Paso 1 · Descriptivos por grupo

```
ele |> group_by(estancia) |>
  summarise(n = n(), media = mean(nota_dele), de = sd(nota_dele))
#> # A tibble: 2 × 4
#>   estancia     n media    de
#>   <chr>    <int> <dbl> <dbl>
#> 1 no         140  65.9  12.4
#> 2 sí         100  71.1  13.0
```

Paso 2 · La prueba

```
t.test(nota_dele ~ estancia, data = ele)
```

En pocas palabras

Aplicamos la prueba t a nuestra pregunta en dos pasos.

1. **Descriptivos**: antes de cualquier prueba hay que mirar los datos. Quienes no han hecho estancia (140 personas) tienen una media de 65,9; quienes sí la han hecho (100), de 71,1. La dispersión de ambos grupos es parecida.
2. **La prueba**: `t.test(nota_dele ~ estancia, data = ele)`. La tilde (~) se lee «en función de»: la nota en función de la estancia.

La pregunta que responde la prueba es si esa diferencia de unos 5 puntos es lo bastante grande como para no atribuirla al azar. La respuesta está en el apartado siguiente.

3.15 Tu resultado, pieza a pieza

```
    Welch Two Sample t-test

data:  nota_dele by estancia
t = -3.0772, df = 207.3, p-value = 0.002372
alternative hypothesis: true difference in means between group no and group sí is not equal to 0
95 percent confidence interval:
 -8.440092 -1.848479
sample estimates:
mean in group no mean in group sí
        65.93571         71.08000
```

Cómo se lee cada parte de la salida:

- `Welch Two Sample t-test` **Welch**: la versión de la prueba t que no exige varianzas iguales en los dos grupos. Es la que R usa por defecto.
- t = −3.0772 **t**: la diferencia entre las medias medida en unidades de error estándar. Es negativa porque R resta «no − sí». Lo que importa es su tamaño: |t| ≈ 3.
- df = 207.3 **Grados de libertad** (gl, en inglés *df*): cuánta información independiente hay para estimar la variabilidad. Con 140 + 100 personas y dos medias estimadas serían 240 − 2 = 238; Welch los rebaja a 207,3 porque los grupos tienen tamaños y dispersiones distintos. Deciden qué curva t se usa para calcular p. Se informan entre paréntesis: t(207,3).
- p-value = 0.002372 **Valor p**: si la estancia no influyera, obtener una diferencia así de grande ocurriría solo 2 veces de cada 1000. Es menor que 0,05 → rechazamos H_0.
- 95 percent confidence interval: −8.440092 −1.848479 **IC 95 % de la diferencia** (no − sí): entre 1,8 y 8,4 puntos a favor de la estancia. **No incluye el 0**: coherente con p < 0,05.
- mean in group no mean in group sí 65.93571 71.08 **Medias de cada grupo**: sin estancia y con estancia. La diferencia es de unos 5 puntos.

En pocas palabras

Esta es la salida de `t.test()`.

- **t = –3,08**: la diferencia equivale a unos tres errores estándar. El signo negativo solo indica que R resta «no – sí».
- **df = 207,3**: los **grados de libertad** (gl). Indican cuánta información independiente tenemos para estimar la variabilidad de los datos. Un ejemplo: si sabemos que la media de cinco notas es 70 y conocemos cuatro de ellas (55, 62, 70 y 71), la quinta ya no es libre: tiene que ser 92 para que la media sea 70. De cinco datos, solo cuatro «se mueven libremente»: 5 – 1 = 4 grados de libertad (por eso dividíamos entre n – 1 al calcular la varianza). En la prueba t con dos grupos se estiman dos medias, así que serían 140 + 100 – 2 = 238. La versión de **Welch** los ajusta a la baja, a 207,3, porque los grupos tienen tamaños distintos y no exactamente la misma dispersión; por eso no salen enteros. ¿Para qué sirven? Para elegir qué curva t se usa al calcular p: con pocos grados de libertad la curva es más ancha y hace falta un t más extremo para que sea significativo; con muchos, como aquí, se parece a la normal y el valor crítico se acerca a 1,96 (aquí, 1,97). Se informan entre paréntesis junto al t: *t*(207,3) = 3,08.
- **p = 0,002**: si la estancia no influyera, una diferencia así aparecería solo 2 de cada 1000 veces. Como es menor que 0,05, rechazamos la hipótesis nula.
- **Intervalo de confianza**: la diferencia real está probablemente entre 1,8 y 8,4 puntos. No incluye el 0.
- **Medias**: 65,9 sin estancia y 71,1 con estancia.

Conclusión: la estancia se asocia con una nota más alta.

3.16 Muestras relacionadas y *d* de Cohen

Paso 3 · Pretest y postest

```
set.seed(2)
pre  <- round(rnorm(20, 60, 10))
post <- pre + round(rnorm(20, 5, 6))
t.test(post, pre, paired = TRUE)$p.value
#> [1] 0.00527797
```

- **Independientes**: personas distintas (estancia sí/no)
- **Relacionadas**: las mismas personas, dos veces

Paso 4 · Tamaño del efecto

```
d_cohen <- function(x, y) {
  sp <- sqrt(((length(x) - 1) * var(x) +
              (length(y) - 1) * var(y)) /
             (length(x) + length(y) - 2))
  (mean(x) - mean(y)) / sp
}
with(ele, d_cohen(nota_dele[estancia == "sí"],
                  nota_dele[estancia == "no"]))
#> [1] 0.4060321
```

En pocas palabras

Dos ideas complementarias:

Muestras relacionadas. Si medimos a las **mismas personas** dos veces, por ejemplo antes y después de un curso, las dos medidas no son independientes: quien saca buena nota antes suele sacarla después. La prueba t para muestras relacionadas (`paired = TRUE`) analiza la mejora de cada persona y es más sensible. En el ejemplo simulado, la mejora media de 5 puntos es significativa (p = 0,005).

Tamaño del efecto. La función `d_cohen()` calcula la *d* de Cohen: la diferencia entre medias dividida por la desviación típica conjunta. Para la estancia, d ≈ 0,41, un efecto entre pequeño y mediano. Es el dato que indica si la diferencia es relevante, no solo si es real.

3.17 Wilcoxon: comparar rangos

Grupo	Errores de ser/estar	Rangos (todos juntos, de menor a mayor)	Suma de rangos
Sin estancia	3, 5, 2, 6, 4, 8, 3, 12	9,0, 13,0, 6,0, 14,0, 11,5, 15,0, 9,0, 16,0	93,5
Con estancia	1, 2, 0, 3, 1, 4, 2, 1	3,0, 6,0, 1,0, 9,0, 3,0, 11,5, 6,0, 3,0	42,5

En pocas palabras

La prueba de **Wilcoxon** (también llamada de Mann-Whitney) es la alternativa a la prueba t cuando los datos no son normales, son ordinales o tienen valores extremos. La pregunta que responde es: **¿tienden los valores de un grupo a ser más altos que los del otro?**

El ejemplo de la animación (inventado, para que se pueda seguir a mano): el número de errores de *ser/estar* de 8 aprendientes sin estancia (azul: 3, 5, 2, 6, 4, 8, 3, 12) y 8 con estancia (rojo: 1, 2, 0, 3, 1, 4, 2, 1). Hay un valor extremo: un aprendiente con 12 errores, que inflaría la media del grupo azul.

En lugar de usar los valores, Wilcoxon usa su **posición**:

1. Se juntan las 16 personas en una sola fila y se ordenan de menor a mayor.
2. Cada una recibe un **rango** (su puesto): el 0 es el puesto 1; los tres 1 ocupan los puestos 2, 3 y 4, así que comparten el rango medio, 3; los tres 2 comparten el 6; y así hasta el 12, que recibe el rango 16.
3. Se suman los rangos de cada grupo. Sin estancia: 93,5; con estancia: 42,5. Si no hubiera diferencias, cada grupo sumaría más o menos lo mismo, unos 68.

El estadístico **W** que da R es la suma de rangos del primer grupo menos la mínima posible (1 + 2 + … + 8 = 36): 93,5 − 36 = **57,5**. Tiene una lectura sencilla: de las 8 × 8 = 64 parejas posibles formadas por un aprendiente de cada grupo, en unas 57,5 el aprendiente sin estancia comete más errores (los empates cuentan medio). Si no hubiera diferencias, serían unas 32. Aquí p = 0,008: los aprendientes sin estancia tienden a cometer más errores.

Así, el valor extremo 12 pesa lo mismo que si fuera 9: solo importa que es el más alto. Por eso la prueba es **robusta** ante valores atípicos.

3.18 Wilcoxon en R

Paso 5 · Datos no normales: errores de ser/estar

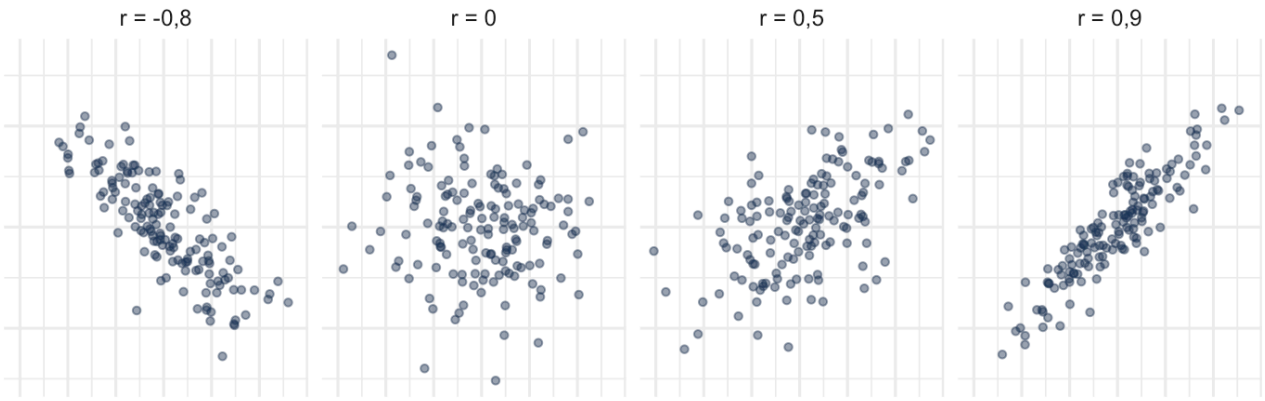
```
wilcox.test(err_ser_estar ~ estancia, data = ele)
#>
#>  Wilcoxon rank sum test with continuity correction
```

```
#>
#> data:  err_ser_estar by estancia
#> W = 7500.5, p-value = 0.3398
#> alternative hypothesis: true location shift is not equal to 0
```

- **Cuándo usarla**: Datos no normales, ordinales o recuentos; muestras pequeñas; valores extremos.
- **Qué compara**: Si un grupo tiende a tener valores más altos (los rangos), no las medias.

> En pocas palabras
>
> Los errores de *ser/estar* son recuentos y, como vimos, no siguen una distribución normal. Para comparar quienes han hecho estancia y quienes no, la prueba adecuada es la de Wilcoxon.
>
> La sintaxis es la misma que la de la prueba t: la variable, la tilde y el grupo.
>
> En nuestros datos, la mediana es de 3 errores en los dos grupos (las medias, 3,3 y 3,0) y el valor p es de 0,34, bastante mayor que 0,05. Por tanto, **no hay evidencia** de que la estancia se asocie con un número distinto de errores de *ser/estar*. W = 7500,5 se lee como en el ejemplo anterior: de las 140 × 100 = 14 000 parejas posibles, en unas 7500 (el 54 %) el aprendiente sin estancia comete más errores; muy cerca del 50 % que esperaríamos sin diferencias.
>
> Que una prueba no sea significativa también es un resultado: no demuestra que no haya ninguna diferencia, pero indica que, con estos datos, no podemos afirmarla.

3.19 Correlación



En pocas palabras

La **correlación** mide si dos variables numéricas varían juntas. Su coeficiente, **r**, va de –1 a 1:

- **r positivo**: cuando una sube, la otra también.
- **r negativo**: cuando una sube, la otra baja.
- **r cercano a 0**: no hay relación lineal.

r^2 (r al cuadrado) indica qué proporción de la variación de una variable se puede predecir a partir de la otra. Con r = 0,5, r^2 = 0,25: conocer X permite explicar un 25 % de lo que varía Y; el 75 % restante depende de otras cosas.

Un ejemplo con nuestros datos: entre horas de instrucción y nota DELE, r = 0,58, así que r^2 = 0,34. Si tuviéramos que adivinar la nota de un aprendiente sin saber nada de él, diríamos la media (68) y nos equivocaríamos, de media, unos 12,9 puntos (la DE). Si conocemos sus horas, podemos afinar la predicción y el error típico baja a unos 10,5 puntos. Hemos reducido un 34 % la variación (medida como varianza, es decir, en puntos al cuadrado: de 166 a 110). El 66 % restante depende de la L1, la estancia, la aptitud de cada persona…

Un aviso: r^2 parece pequeño enseguida. r = 0,3 (una correlación «mediana») solo explica un 9 %.

Al activar el valor atípico se ve que un solo punto puede alterar mucho la correlación de **Pearson**. La de **Spearman**, basada en rangos como Wilcoxon, resiste mejor. Y, sobre todo, **correlación no implica causalidad**.

3.20 Correlación en R

Paso 6 · Horas y nota

```
cor.test(ele$horas, ele$nota_dele)
#>
#>  Pearson's product-moment correlation
#>
#> data:  ele$horas and ele$nota_dele
#> t = 11.105, df = 238, p-value < 2.2e-16
#> alternative hypothesis: true correlation is not equal to 0
#> 95 percent confidence interval:
#>  0.4941355 0.6618754
#> sample estimates:
#>       cor
#> 0.5842101
```

Para datos no normales u ordinales: `method = "spearman"`.

En pocas palabras

`cor.test()` calcula la correlación entre horas de instrucción y nota, y además comprueba si es significativa.

En nuestros datos, r = 0,58: una relación positiva y moderadamente fuerte. A más horas, más nota. El valor p es minúsculo, así que la relación no es casual. La salida incluye también el intervalo de confianza de r (de 0,49 a 0,66).

¿Cuándo es fuerte una correlación? Las referencias más citadas son las de Cohen (1988), para las ciencias sociales en general: |r| ≈ 0,1 es débil, 0,3 moderada y **0,5 o más, fuerte**. En investigación sobre segundas lenguas, Plonsky y Oswald (2014) revisaron cientos de estudios y propusieron 0,25, 0,40 y 0,60. ¿Por qué estos números? No salen de ninguna fórmula: son **convenciones** basadas en lo que suele encontrarse en cada campo. El comportamiento humano depende de muchos factores a la vez, así que es raro que una sola variable explique más de un tercio de otra; en física, en cambio, r = 0,6 sería una relación pobre. Por eso, con r = 0,58 decimos que la relación es fuerte según Cohen y entre moderada y fuerte según las referencias de L2.

Con r = 0,58, las horas comparten aproximadamente un tercio de la variación de la nota ($0{,}58^2 \approx 0{,}34$). El resto depende de otros factores. El signo no influye en la fuerza: r = −0,58 sería igual de fuerte, pero en sentido contrario.

Si los datos no son normales o son ordinales, basta con añadir `method = "spearman"`.

3.21 Práctica 3 (20 min)

1. Comprueba con un gráfico Q-Q y con `shapiro.test()` si `horas` es normal.
2. Calcula el IC 95 % de la nota media del DELE.
3. Recorre el árbol para elegir la prueba: ¿difiere la nota entre L1 **inglés** y **chino**? Filtra y usa `t.test()`.
4. Calcula la *d* de Cohen de esa diferencia.
5. ¿Se relacionan `horas` y `err_ser_estar`? Usa Spearman.

3.22 Práctica 3: soluciones

Soluciones de la práctica 3

```
qqnorm(ele$horas); qqline(ele$horas); shapiro.test(ele$horas)
t.test(ele$nota_dele)$conf.int
dos <- ele |> filter(L1 %in% c("inglés", "chino"))
t.test(nota_dele ~ L1, data = dos)
with(dos, d_cohen(nota_dele[L1 == "inglés"], nota_dele[L1 == "chino"]))
cor.test(ele$horas, ele$err_ser_estar, method = "spearman")
```

En pocas palabras

Una posible solución para cada ejercicio de la práctica 3:

1. Normalidad de las horas: gráfico Q-Q y prueba de Shapiro-Wilk.
2. Intervalo de confianza de la nota media con `t.test()`.
3. Comparar inglés y chino: primero se filtran esas dos lenguas con `%in%` («está entre») y después se aplica `t.test()`. La diferencia es significativa.
4. *d* de Cohen de esa diferencia, con la función creada en la sesión.
5. Relación entre horas y errores con Spearman, porque los errores no son normales. La correlación es negativa: a más horas, menos errores.

4 Chi cuadrado y ANOVA

Sesión 4 · 60 minutos

> ℹ La pregunta de esta sesión
>
> ¿Depende el uso del subjuntivo de la estancia? ¿Difiere la nota según la lengua materna?

Contenidos: Tablas de contingencia · Observadas y esperadas · Residuos · V de Cramér · ANOVA y F · Tukey · Interacciones.

4.1 Tabla de contingencia

Paso 1 · Cruzar dos categóricas

```
tab <- table(ele$estancia, ele$subjuntivo)
tab
#>
#>      correcto incorrecto
#>   no       62         78
#>   sí       62         38
prop.table(tab, 1) |> round(2)
#>
#>      correcto incorrecto
#>   no     0.44       0.56
#>   sí     0.62       0.38
```

> En pocas palabras
>
> Cuando las dos variables son **categóricas** no podemos calcular medias. Lo que hacemos es contar cuántas personas hay en cada combinación: es una **tabla de contingencia**.
>
> Aquí cruzamos la estancia con el uso del subjuntivo:
>
> - Sin estancia: 62 aciertan y 78 fallan.
> - Con estancia: 62 aciertan y 38 fallan.
>
> Como los grupos tienen tamaños distintos, los recuentos engañan. `prop.table(tab, 1)` calcula porcentajes **por fila**: sin estancia acierta un 44 %; con estancia, un 62 %. La diferencia parece clara. La prueba de chi cuadrado dirá si es lo bastante grande como para no atribuirla al azar.

4.2 Chi cuadrado: ficha

- **¿Qué pregunta responde?**: ¿Están **asociadas** dos variables **categóricas**? *¿Depende el uso del subjuntivo de la estancia?*
- **Hipótesis**: H_0: las variables son independientes H_1: están asociadas
- **Supuestos**: · Recuentos (no porcentajes ni medias) · Cada persona en **una sola** celda · Esperadas ≥ 5 (si no, **Fisher**)
- **Estadístico**: $\chi^2 = \Sigma\ (O - E)^2 / E$ esperada = total fila × total columna / total Efecto: **V de Cramér**

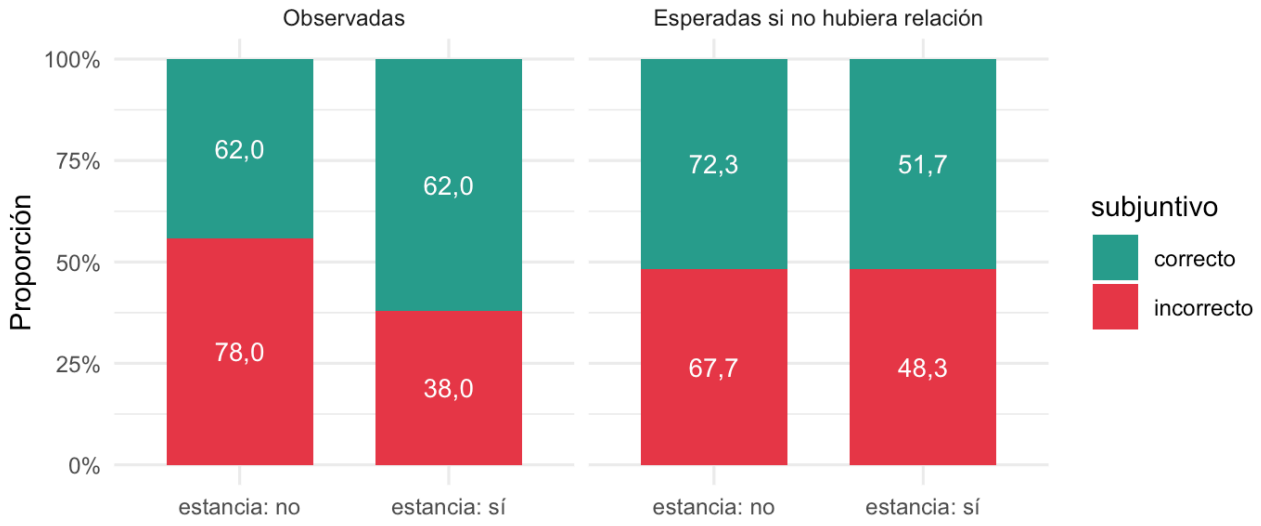
En pocas palabras

La prueba de **chi cuadrado (χ^2)** comprueba si dos variables categóricas están **asociadas**.

- **Hipótesis nula**: son independientes. Saber si alguien ha hecho una estancia no dice nada sobre si usará bien el subjuntivo.
- **Supuestos**: se trabaja con recuentos (no con porcentajes), cada persona aparece en una sola celda y las frecuencias esperadas son al menos 5. Si no se cumple esto último, se usa la prueba de **Fisher**.
- **Estadístico**: compara lo **observado** (O) con lo **esperado** (E) si no hubiera asociación. Cuanto mayor es la diferencia, mayor es χ^2.

Para medir la fuerza de la asociación se usa la **V de Cramér**.

4.3 Observadas, esperadas y residuos



En pocas palabras

La visualización muestra la lógica del chi cuadrado en tres vistas:

- **Observadas** (O): lo que hay en los datos. La anchura de cada columna representa cuántas personas hay en ese grupo (140 sin estancia, 100 con estancia); la altura de cada color, el reparto entre acierto y fallo.
- **Esperadas** (E): lo que habría si la estancia no influyera. En total acierta el 51,7 % (124 de 240), así que esperaríamos que acertara ese mismo 51,7 % en los dos grupos: 140 × 124 / 240 = **72,3** aciertos sin estancia y 100 × 124 / 240 = **51,7** con estancia. Las dos columnas tendrían el mismo reparto.
- **Residuos estandarizados**: cuánto se aleja cada celda de lo esperado, en una escala comparable entre celdas (como una puntuación z). Sin estancia hay 62 aciertos en lugar de 72,3: residuo de **–2,71**. Un residuo mayor que 1,96 o menor que –1,96 indica una celda que se aparta claramente de lo esperado.

La tabla del panel recoge, para cada celda, cuatro números: lo observado (O), lo esperado (E), su **contribución** al χ^2, $(O - E)^2 / E$, y el **residuo estandarizado**, el mismo que se dibuja en el gráfico. La contribución y el residuo no son lo mismo: la contribución siempre es positiva y mide cuánto aporta la celda al total; el residuo tiene signo y dice si hay más (+) o menos (–) casos de lo esperado.

¿Por qué la suma de contribuciones (7,33) no coincide con el χ^2 que da R (6,64)? Por la **corrección de Yates**. En las tablas de 2 × 2, `chisq.test()` resta 0,5 a cada diferencia $|O - E|$ antes de elevarla al cuadrado. Se hace porque los recuentos son números enteros y la fórmula del χ^2 supone una variable continua; la corrección compensa ese desajuste y hace la prueba algo más prudente (el χ^2 baja y p sube: 0,010 en lugar de 0,007). Si se quiere el valor sin corregir, que sí coincide con la suma: `chisq.test(tab, correct = FALSE)`. En tablas más grandes que 2 × 2 no se aplica.

χ^2 mide, en un solo número, cuánto se diferencian la imagen observada y la esperada. En nuestro caso, las cuatro celdas superan el umbral de ±1,96.

4.4 χ^2 paso a paso

Paso 2 · Esperadas y contribuciones

```
prueba <- chisq.test(tab)
prueba$expected |> round(1)   # esperadas
#>
#>      correcto incorrecto
#>   no     72.3       67.7
#>   sí     51.7       48.3
140 * 124 / 240               # una celda a mano
#> [1] 72.33333
round((tab - prueba$expected)^2 /
        prueba$expected, 2)   # contribuciones
#>
#>      correcto incorrecto
#>   no     1.48       1.58
#>   sí     2.07       2.21
```

Paso 3 · La prueba y los residuos

```
prueba                        # con corrección de Yates
#>
#>  Pearson's Chi-squared test with Yates' continuity correction
#>
#> data:  tab
#> X-squared = 6.6379, df = 1, p-value = 0.009983
```

```r
prueba$stdres |> round(2)     # residuos estandarizados
#>
#>      correcto incorrecto
#>   no    -2.71       2.71
#>   sí     2.71      -2.71
chisq.test(tab, correct = FALSE)$statistic
#> X-squared
#>  7.330049
```

En pocas palabras

El cálculo del chi cuadrado, paso a paso, con la función de R.

1. **Frecuencias esperadas**. No hace falta calcularlas a mano: `chisq.test()` las guarda y basta con pedirlas con `prueba$expected`. Para cada celda, R hace: total de su fila × total de su columna ÷ total general. Lo comprobamos con una celda: sin estancia hay 140 personas y en total aciertan 124, así que esperaríamos 140 × 124 / 240 = **72,3** aciertos sin estancia si no hubiera asociación.
2. **Contribución de cada celda**: (observada – esperada)² ÷ esperada. Para la celda anterior, (62 – 72,3)² / 72,3 = 1,48. La suma de las cuatro contribuciones es 7,33: el χ^2 sin corrección.
3. `chisq.test()` hace todo el proceso. El resultado es $\chi^2 = 6{,}64$ y $p = 0{,}010$: rechazamos la independencia. Es algo menor que 7,33 porque, en las tablas de 2 × 2, R aplica la **corrección de Yates** (resta 0,5 a cada |O – E|), que hace la prueba más prudente. Con `correct = FALSE` se obtiene el valor sin corregir, 7,33.

Los **residuos estandarizados** (`stdres`) indican dónde está la asociación: con estancia hay más aciertos de los esperados (+2,71) y sin estancia, más fallos (+2,71).

(Al restar `tab - prueba$expected`, R resta celda a celda las dos tablas, que tienen la misma forma.)

4.5 Tamaño del efecto y Fisher

Paso 4 · V de Cramér

```r
v_cramer <- function(t) {
  chi <- suppressWarnings(chisq.test(t, correct = FALSE))$statistic
  sqrt(chi / (sum(t) * (min(dim(t)) - 1))) |> unname()
}
v_cramer(tab)
#> [1] 0.1747623
fisher.test(tab)$p.value
#> [1] 0.008698842
```

Fisher: cuando alguna frecuencia esperada es menor que 5.

En pocas palabras

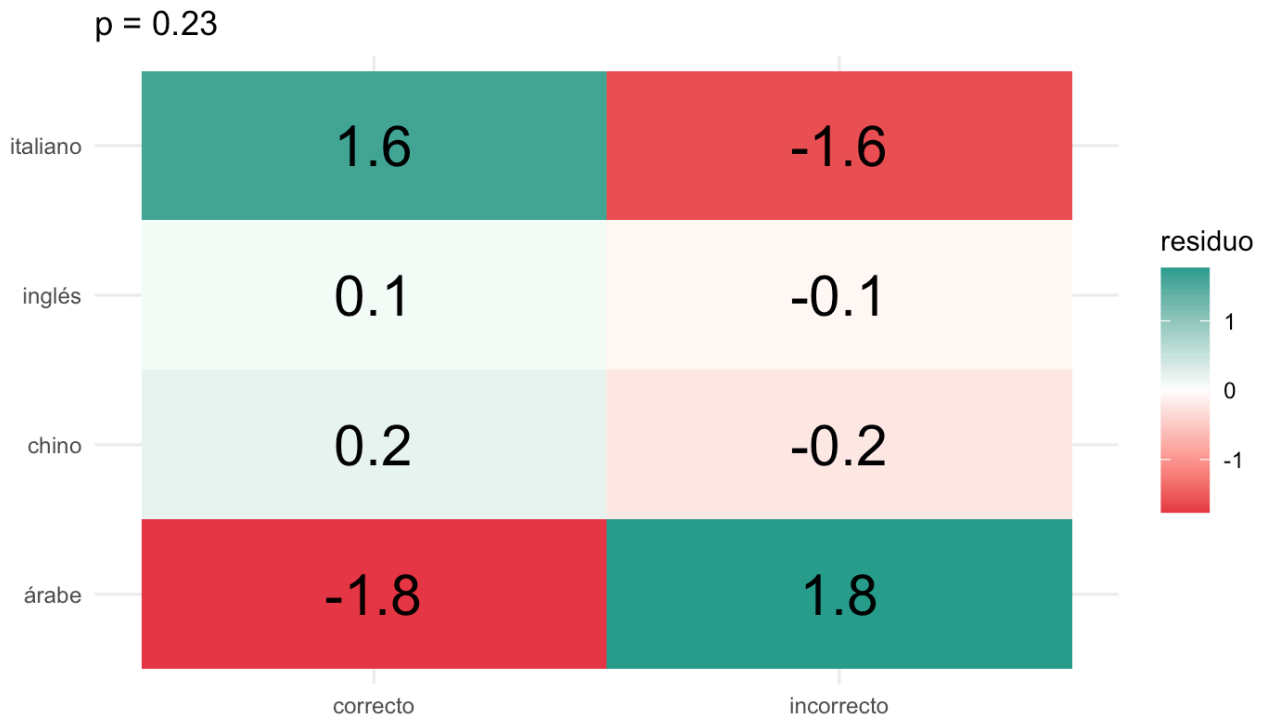
La **V de Cramér** mide la **fuerza** de la asociación entre dos variables categóricas. Va de 0 (ninguna asociación) a 1 (asociación perfecta). Con una tabla de 2 × 2 como esta, un valor en torno a 0,1 se considera pequeño, 0,3 mediano y 0,5 grande. Nuestro resultado indica un efecto pequeño: la estancia importa, pero no lo explica todo.

La **prueba exacta de Fisher** es la alternativa al chi cuadrado cuando hay pocas observaciones, en concreto cuando alguna frecuencia esperada es menor que 5. Calcula la probabilidad exacta en lugar de usar una aproximación. Aquí llega a la misma conclusión que el chi cuadrado.

4.6 Más categorías: residuos como mapa de calor

Paso 5 · L1 × subjuntivo

```
tab2 <- table(L1 = ele$L1,
              subjuntivo = ele$subjuntivo)
prueba2 <- chisq.test(tab2)    # p = 0,23
res <- as.data.frame(prueba2$stdres) |>
  rename(residuo = Freq)
ggplot(res, aes(subjuntivo, L1,
                fill = residuo)) +
  geom_tile() +
  geom_text(aes(label = round(residuo, 1)),
            size = 7) +
  scale_fill_gradient2(low = "#E63946",
    mid = "white", high = "#2A9D8F") +
  labs(x = NULL, y = NULL, title = paste(
    "p =", round(prueba2$p.value, 2)))
```



En pocas palabras

Cuando una de las variables tiene muchas categorías, como la L1 con cuatro lenguas, es difícil ver en una tabla dónde está la asociación. Una solución es dibujar los **residuos estandarizados** como mapa de calor. El código tiene tres pasos:

1. `table(L1 = ..., subjuntivo = ...)` cruza las dos variables. Al poner un nombre delante de cada una, la tabla recuerda cómo se llaman.
2. `as.data.frame()` convierte la tabla de residuos en una tabla de datos con una fila por celda, que es lo que necesita `ggplot()`. Esta función llama siempre **`Freq`** a la columna con el valor de cada celda, porque está pensada para tablas de frecuencias; aquí ese valor es un residuo, no una frecuencia, así que la renombramos con `rename(residuo = Freq)`. (Si no hubiéramos nombrado las variables en el paso 1, las otras dos columnas se llamarían `Var1` y `Var2`, nombres genéricos que no dicen nada.)
3. `ggplot()` dibuja una celda por combinación (`geom_tile`), escribe el residuo dentro (`geom_text`) y colorea con una escala de tres colores (`scale_fill_gradient2`).

Cómo se leen los colores:

- **Verde**: más casos de los esperados en esa combinación.
- **Rojo**: menos de los esperados.
- **Blanco**: lo esperado.

En nuestros datos, el valor p de la prueba (`prueba2$p.value`, en el título del gráfico) es 0,23: **no hay asociación significativa** entre la L1 y el uso del subjuntivo. Los colores son suaves y ningún residuo supera ±1,96: el más extremo es el del árabe (–1,8 en «correcto»: algo menos de aciertos de los esperados), y el del italiano va en sentido contrario (+1,6). Esta es otra forma de ver que un resultado no es significativo.

4.7 Cambiamos de pregunta: ¿difiere la nota según la L1?

- **Hasta ahora: χ^2**: Dos variables **categóricas** *estancia (sí/no)* × *subjuntivo (correcto/incorrecto)* Contamos personas en cada celda.
- **Ahora: ANOVA**: Una variable **numérica** y un factor con **3 o más grupos** *nota DELE* × *L1 (árabe, chino, inglés, italiano)* Comparamos medias.

Paso 6 · Las cuatro medias que vamos a comparar

```
ele |> group_by(L1) |>
  summarise(n = n(), media = mean(nota_dele), de = sd(nota_dele))
#> # A tibble: 4 × 4
#>   L1           n media    de
#>   <chr>    <int> <dbl> <dbl>
#> 1 chino       49  61.6  13.2
#> 2 inglés      73  69.7  10.3
#> 3 italiano    52  76.3  11.4
#> 4 árabe       66  64.6  12.8
```

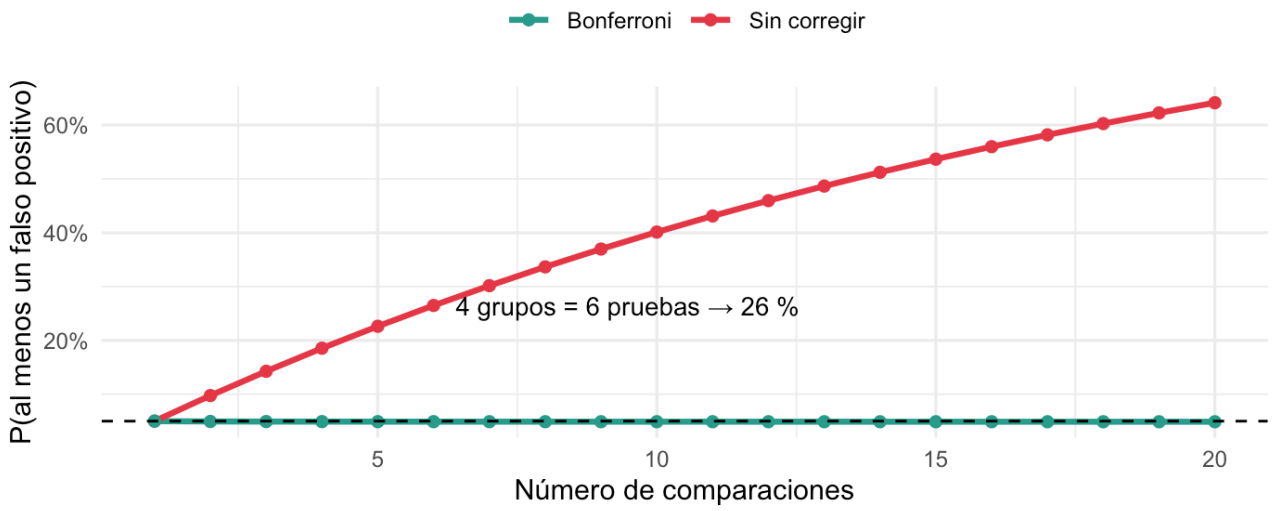
En pocas palabras

Este apartado marca el paso a la segunda parte de la sesión. Hasta ahora, con el chi cuadrado, las dos variables eran **categóricas** y lo que hacíamos era contar personas en cada combinación.

Ahora cambiamos de pregunta: **¿difiere la nota del DELE según la lengua materna?** La respuesta (la nota) es **numérica**, así que podemos calcular medias, y el factor (la L1) tiene **cuatro grupos**. Con dos grupos usaríamos la prueba t de la sesión 3; con tres o más, el **análisis de la varianza (ANOVA)**.

Los descriptivos muestran lo que vamos a contrastar: los italohablantes tienen la media más alta (76,3), seguidos de los anglohablantes (69,7), los arabófonos (64,6) y los sinohablantes (61,6). ¿Son diferencias reales o podrían deberse al azar? La primera tentación sería comparar las lenguas de dos en dos con pruebas t. El apartado siguiente explica por qué no es buena idea.

4.8 ¿Por qué no hacer muchas pruebas t?



En pocas palabras

Si queremos comparar la nota de cuatro lenguas maternas, podríamos hacer una prueba t para cada pareja: árabe-chino, árabe-inglés, árabe-italiano, chino-inglés, chino-italiano e inglés-italiano. Son **seis comparaciones**. El problema es que cada prueba tiene un 5 % de riesgo de dar una falsa alarma, y los riesgos se acumulan.

Con seis pruebas, la probabilidad de obtener **al menos una** falsa alarma sube a $1 - 0{,}95^6 \approx$ **26 %**. Con veinte pruebas, a más del 60 %. Es como tirar seis veces un dado de veinte caras: aunque la probabilidad de sacar un 20 en cada tirada sea pequeña, en seis tiradas es bastante probable que salga alguna vez.

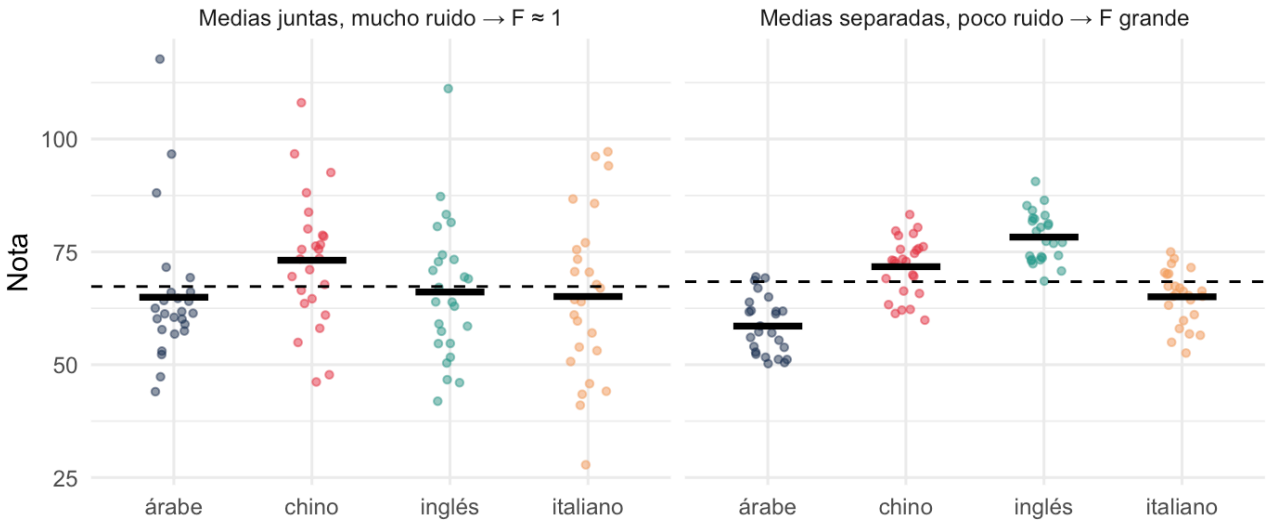
Hay dos soluciones:

- **Corregir el umbral con Bonferroni**: dividir 0,05 entre el número de pruebas. Con seis pruebas, cada una debe dar $p < 0{,}0083$ para considerarse significativa. Al activar la casilla, la curva baja y el riesgo total vuelve a quedar por debajo del 5 %.
- **Hacer una sola prueba global**, el ANOVA, y después comparar las parejas con un método diseñado para ello (Tukey).

El inconveniente de Bonferroni es que es **demasiado prudente**. Al exigir un umbral tan bajo a cada prueba, se pierde potencia: aumentan los errores de tipo II y se nos escapan diferencias que sí existen. Con veinte comparaciones, cada una tendría que dar $p < 0{,}0025$. Por ejemplo, una diferencia real con $p = 0{,}01$ dejaría de ser significativa. Por eso, cuando todas las comparaciones proceden de los mismos grupos, se prefiere el camino del **ANOVA**: primero una única pregunta global con $\alpha = 0{,}05$ («¿hay alguna diferencia entre las cuatro medias?») y, solo si la respuesta es sí, la prueba de **Tukey**, que corrige las comparaciones múltiples de forma menos drástica que Bonferroni.

4.9 La idea de F

$$F = \frac{\text{variación entre grupos}}{\text{variación dentro de los grupos}}$$



En pocas palabras

El **ANOVA** (análisis de la varianza) compara las medias de tres o más grupos con un único estadístico, **F**, que tiene la misma lógica que la prueba t: señal frente a ruido.

La simulación genera la **nota** de cuatro grupos de 25 aprendientes, uno por L1. Cada punto es un aprendiente, las rayas negras son las medias de cada grupo y la línea discontinua, la media general.

- **Variación entre grupos** (la señal): cuánto se separan las medias de los grupos de la media general.
- **Variación dentro de los grupos** (el ruido): cuánto varían las personas alrededor de la media de su propio grupo.

Qué miden las barras. Las dos barras de la derecha son **varianzas medias** (en la salida de R, *Mean Sq*), en puntos al cuadrado:

- La **gris (dentro)** es, en esencia, la varianza típica de las notas dentro de cada grupo. Con la variabilidad del deslizador en 10 puntos, vale unos 10^2 = **100**.
- La **roja (entre)** mide cuánto se apartan las medias de los grupos de la media general, multiplicado por el número de personas por grupo (25). Si los grupos no se diferenciaran en nada, valdría también unos 100, porque las medias variarían solo por azar.

F es el cociente de las dos. Por ejemplo, si la barra roja llega a **120** y la gris a **35**, F = 120 / 35 ≈ **3,4**: la separación entre grupos es 3,4 veces mayor de lo que cabría esperar por azar. Si F estuviera cerca de 1, la señal no se distinguiría del ruido.

- Al separar las medias, la barra roja crece y F sube.
- Al aumentar la variabilidad dentro de los grupos, la barra gris crece y F baja.

η^2 (eta cuadrado) responde a otra pregunta: de toda la variación de las notas, ¿qué parte se debe a la diferencia entre grupos? Se calcula con las **sumas de cuadrados** (las de la desviación típica, paso a paso): η^2 = SC entre / SC total. Un η^2 de 0,15 significa que el **15 %** de lo que varían las notas de estos 100 aprendientes se explica por el grupo al que pertenecen; el 85 % restante son diferencias entre personas del mismo grupo. Si supiéramos el grupo de cada persona y predijéramos su nota con la media de su grupo, reduciríamos en un 15 % la variación total.

4.10 ANOVA: ficha

- **¿Qué pregunta responde?**: ¿Difieren las **medias** entre **3 o más grupos**? *¿Varía la nota del DELE según la L1?*
- **Hipótesis**: H_0: $\mu_1 = \mu_2 = \mu_3 = \mu_4$ H_1: al menos una media es distinta
- **Supuestos**: · Residuos **normales** · **Varianzas similares** · Observaciones independientes¿No se cumplen? → **Kruskal-Wallis** (`kruskal.test()`), que compara rangos.
- **Después**: F grande → p pequeño → **post hoc** (Tukey) y **η^2** Con Kruskal-Wallis, el post hoc es `pairwise.wilcox.test()`

En pocas palabras

El **ANOVA de un factor** comprueba si la media de una variable numérica difiere entre **tres o más grupos**. Nuestra pregunta: ¿varía la nota del DELE según la lengua materna?

- **Hipótesis nula**: todas las medias son iguales.
- **Hipótesis alternativa**: al menos una es distinta. El ANOVA no dice cuál.
- **Supuestos**: los residuos son aproximadamente normales, los grupos tienen una dispersión parecida y las observaciones son independientes. Los **residuos** son lo que el modelo no explica: la distancia de cada persona a la media de su grupo. Una aprendiente italohablante con 85 puntos tiene un residuo de 85 − 76,3 = +8,7.
- **Si no se cumplen los supuestos** (por ejemplo, con recuentos como los errores de *ser/estar*, que no son normales), se usa la prueba de **Kruskal-Wallis**, la versión del ANOVA basada en rangos, igual que Wilcoxon lo es de la prueba t. La veremos al final de la sesión.

Si el resultado es significativo, se hace un análisis ***post hoc*** («a posteriori»: Tukey) para saber qué grupos difieren, y se calcula **η²** para medir el tamaño del efecto.

4.11 ANOVA de un factor

Paso 7 · Ajustar el ANOVA

```
modelo_aov <- aov(nota_dele ~ L1, data = ele)
summary(modelo_aov)
#>              Df Sum Sq Mean Sq F value   Pr(>F)
#> L1            3   6567  2188.9   15.57 2.84e-09 ***
#> Residuals   236  33181   140.6
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Paso 8 · Supuestos

```
shapiro.test(residuals(modelo_aov))$p.value   # normalidad de residuos
#> [1] 0.8432763
bartlett.test(nota_dele ~ L1, data = ele)$p.value  # igualdad de varianzas
#> [1] 0.1971892
```

En pocas palabras

`aov()` ajusta el ANOVA y `summary()` muestra la tabla de resultados. Las columnas más importantes:

- **Df**: grados de libertad: número de grupos menos uno (4 − 1 = 3) y número de personas menos el de grupos (240 − 4 = 236).
- **Sum Sq**: las sumas de cuadrados, la variación entre grupos (6567) y dentro de los grupos (33 181).
- **Mean Sq**: cada suma dividida entre sus grados de libertad: 6567 / 3 = 2188,9 y 33 181 / 236 = 140,6. Son las dos barras del apartado de la idea de F. La de dentro, 140,6, es la varianza típica dentro de cada L1 (√140,6 ≈ 11,9 puntos de DE).
- **F value**: el cociente de las dos: 2188,9 / 140,6 = **15,6**. La separación entre las medias de las L1 es 15,6 veces mayor de lo que cabría esperar por azar.
- **Pr(>F)**: el valor p, minúsculo (0,000000003). Las medias **no** son todas iguales.

El segundo bloque comprueba los supuestos:

- **Shapiro-Wilk** sobre los residuos: si p > 0,05, son aproximadamente normales.
- **Bartlett**: si p > 0,05, las varianzas de los grupos son similares.

Si los supuestos no se cumplieran, la alternativa sería Kruskal-Wallis, que veremos al final de la sesión.

4.12 ¿Qué grupos difieren? Tukey

Paso 9 · Comparaciones post hoc

```
TukeyHSD(modelo_aov)
#>   Tukey multiple comparisons of means
#>     95% family-wise confidence level
#>
#> Fit: aov(formula = nota_dele ~ L1, data = ele)
#>
#> $L1
#>                       diff        lwr      upr     p adj
#> chino-árabe     -3.044527 -8.8299337  2.74088 0.5247310
#> inglés-árabe     5.034869 -0.1762258 10.24596 0.0624970
#> italiano-árabe  11.690559  6.0017374 17.37938 0.0000015
#> inglés-chino     8.079396  2.4134110 13.74538 0.0015803
#> italiano-chino  14.735086  8.6268482 20.84332 0.0000000
#> italiano-inglés  6.655690  1.0883611 12.22302 0.0118235
```

En pocas palabras

El ANOVA nos ha dicho que no todas las medias son iguales, pero no cuáles difieren. La prueba de **Tukey** compara todas las parejas de grupos y corrige automáticamente el problema de las comparaciones múltiples.

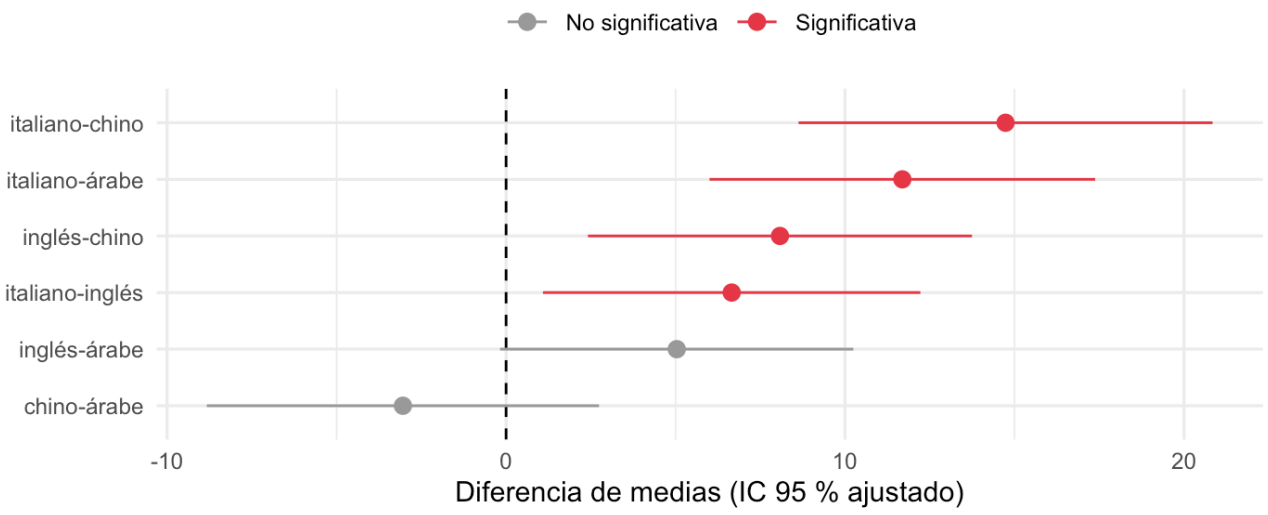
Cada fila es una pareja:

- **diff**: la diferencia entre las dos medias.
- **lwr** y **upr**: el intervalo de confianza de esa diferencia.
- **p adj**: el valor p ya corregido.

Las diferencias significativas en nuestros datos: el italiano supera a las otras tres lenguas y el inglés supera al chino. Chino y árabe no difieren, e inglés y árabe tampoco (p = 0,06, en el límite).

Un ejemplo de lectura, la fila `italiano–inglés`: diff = 6,66, lwr = 1,09, upr = 12,22, p adj = 0,012. Los aprendientes con L1 italiano sacan de media **6,7 puntos más** en el DELE que los de L1 inglés (76,3 frente a 69,7); la diferencia real está probablemente entre 1,1 y 12,2 puntos, y como el intervalo no incluye el 0 (y p < 0,05), es significativa. El orden de los nombres indica la resta: `italiano–inglés` es italiano menos inglés.

4.13 Tu resultado de Tukey, explicado



En pocas palabras

Este gráfico representa los resultados de Tukey para la **nota DELE según la L1**. Cada línea horizontal es el **intervalo de confianza** de la diferencia entre dos lenguas, y el punto central, la diferencia estimada en puntos de nota.

La clave está en la línea vertical del **cero**:

- Si el intervalo **no toca el cero** (en color), la diferencia es significativa: podemos afirmar que esas dos lenguas difieren.
- Si el intervalo **cruza el cero** (en gris), la diferencia podría ser nula.

Ejemplos concretos:

- `italiano–inglés`: el punto está en +6,7 y el intervalo va de +1,1 a +12,2. Traducido: **los aprendientes italohablantes sacan, de media, unos 7 puntos más que los anglohablantes** (entre 1 y 12 con un 95 % de confianza). Como todo el intervalo está a la derecha del cero, es una diferencia significativa.
- `italiano–chino`: +14,7 puntos, de +8,6 a +20,8. Es la mayor diferencia: casi 15 puntos de nota a favor de los italohablantes.
- `chino–árabe`: –3,0 puntos, de –8,8 a +2,7. El intervalo cruza el cero: con estos datos no podemos afirmar que sinohablantes y arabófonos saquen notas distintas.

Este tipo de gráfico suele comunicar los resultados mejor que la tabla.

4.14 Tamaño del efecto y dos factores

Paso 10 · η²

```
ss <- summary(modelo_aov)[[1]]$`Sum Sq`
eta2 <- ss[1] / sum(ss)
eta2
#> [1] 0.1652138
```

La L1 explica un 17 % de la variación de la nota: un efecto **grande** ($\eta^2 \geq 0{,}14$). El 83 % restante se reparte entre otros factores (horas, estancia, diferencias individuales…).

Paso 11 · ANOVA de dos factores con interacción

```
summary(aov(nota_dele ~ L1 * estancia, data = ele))
#>              Df Sum Sq Mean Sq F value   Pr(>F)
#> L1            3   6567  2188.9  16.260 1.26e-09 ***
#> estancia      1   1597  1596.8  11.861  0.00068 ***
#> L1:estancia   3    351   117.0   0.869  0.45790
#> Residuals   232  31233   134.6
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

En pocas palabras

η² (eta cuadrado) es el tamaño del efecto del ANOVA: la proporción de la variación total de la nota que se explica por la L1. Se calcula con las sumas de cuadrados de la tabla: 6567 / (6567 + 33 181) = 6567 / 39 748 ≈ **0,17**, el 17 %.

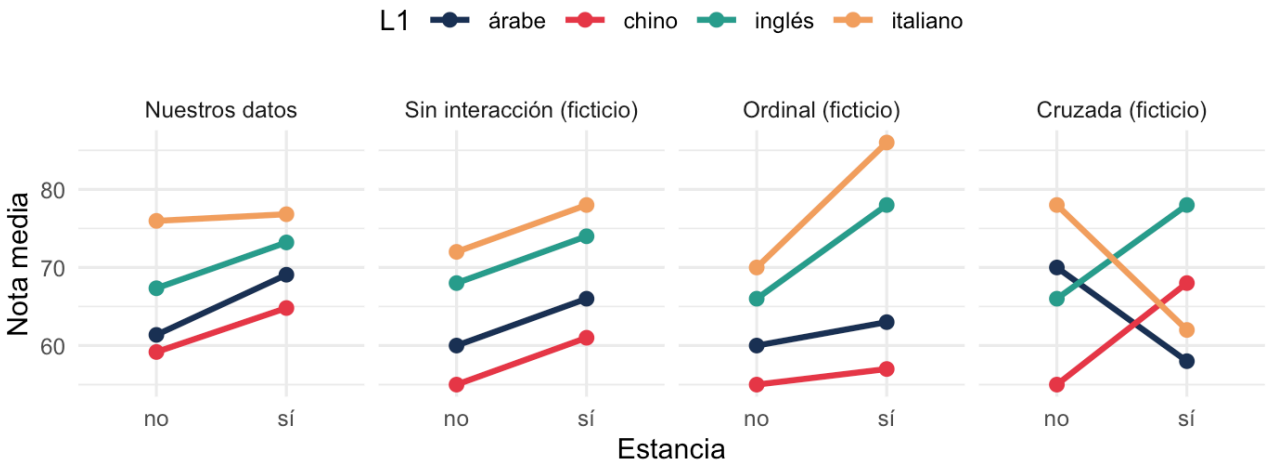
¿Por qué un 17 % es un efecto «grande»? Porque se compara con lo que suele encontrarse en ciencias humanas, no con el 100 %. La nota de un aprendiente depende de muchísimas cosas a la vez: horas de clase, estancias, motivación, aptitud, el día del examen… Que **una sola** característica, la lengua materna, explique casi una sexta parte de todas las diferencias es mucho. Las referencias de Cohen para η² lo reflejan: 0,01 es pequeño, 0,06 mediano y **0,14 o más, grande**.

¿Y el 83 % restante? ¿No importa? Claro que importa: es todo lo que la L1 **no** explica, repartido entre muchas fuentes que este modelo no incluye. Algunas las tenemos en los datos: si añadimos las horas y la estancia al modelo (`lm(nota_dele ~ L1 + horas + estancia)`), la parte explicada sube del 17 % al **53 %**. El resto son diferencias individuales que no hemos medido. η² no dice que lo demás sea irrelevante, sino cuánto pesa la variable que estamos estudiando.

El segundo bloque introduce el **ANOVA de dos factores**: analiza a la vez la L1 y la estancia. El asterisco (`L1 * estancia`) incluye ambos factores y su **interacción**. La tabla tiene tres filas:

- **L1**: efecto de la lengua materna (significativo).
- **estancia**: efecto de la estancia (significativo).
- **L1:estancia**: la interacción (no significativa, p = 0,46). El efecto de la estancia es parecido en todas las lenguas.

4.15 ¿Qué es una interacción?



En pocas palabras

Hay **interacción** cuando el efecto de un factor depende del otro. Por ejemplo, si la estancia ayudara mucho a los sinohablantes y nada a los italohablantes.

En el gráfico, cada línea es una L1, y su inclinación muestra el efecto de la estancia:

- **Nuestros datos** (el único escenario real): las líneas son casi paralelas. La estancia ayuda de forma parecida a todos y no hay interacción.
- **Sin interacción** (ficticio): líneas exactamente paralelas; la estancia suma 6 puntos a todas las L1.
- **Interacción ordinal** (ficticia): la estancia ayuda a todos, pero no por igual. Por ejemplo, suma 2 puntos a los sinohablantes y 16 a los italohablantes.
- **Interacción cruzada** (ficticia): ayuda a unos y perjudica a otros. Por ejemplo, los sinohablantes pasarían de 55 a 68 con estancia y los arabófonos bajarían de 70 a 58.

Los tres últimos escenarios son **inventados**: sirven para ver qué aspecto tendría una interacción, pero no salen de nuestros datos.

En el último caso, hablar del «efecto de la estancia» en general sería engañoso: hay que describirlo para cada grupo por separado.

4.16 Kruskal-Wallis y resumen

Paso 12 · ANOVA no paramétrico

```
kruskal.test(err_ser_estar ~ L1, data = ele)
#>
#>  Kruskal-Wallis rank sum test
#>
#> data:  err_ser_estar by L1
#> Kruskal-Wallis chi-squared = 22.722, df = 3, p-value = 4.615e-05
```

Paso 13 · Post hoc: el «Tukey» de Kruskal-Wallis

```
pairwise.wilcox.test(ele$err_ser_estar, ele$L1,
                     p.adjust.method = "holm",
                     exact = FALSE)
#>
#>  Pairwise comparisons using Wilcoxon rank sum test with continuity correction
#>
#> data:  ele$err_ser_estar and ele$L1
#>
#>          árabe  chino  inglés
#> chino    0.0863 -      -
#> inglés   0.6894 0.1552 -
#> italiano 0.0062 7e-05  0.0044
#>
#> P value adjustment method: holm
```

Pregunta	Prueba	Post hoc
2 categóricas	Chi cuadrado / Fisher (`chisq.test()`, `fisher.test()`)	Residuos (`$stdres`)
Numérica × ≥ 3 grupos	ANOVA (`aov()`)	Tukey (`TukeyHSD()`)
Ídem, sin normalidad	Kruskal-Wallis (`kruskal.test()`)	Wilcoxon por parejas (`pairwise.wilcox.test()`)

En pocas palabras

La prueba de **Kruskal-Wallis** es al ANOVA lo que Wilcoxon es a la prueba t: la alternativa cuando los datos no son normales. Junta a todos los aprendientes, los ordena, les asigna rangos y comprueba si algún grupo tiende a tener rangos más altos que los demás.

La aplicamos a los errores de *ser/estar* según la L1, que son recuentos y no siguen una distribución normal. El resultado es significativo ($\chi^2 = 22{,}7$, gl = 3, $p < 0{,}001$): el número de errores **sí** depende de la lengua materna. Las medianas son de 4 errores en chino, 3 en árabe e inglés y 2 en italiano.

¿Y qué grupos difieren? Igual que el ANOVA, Kruskal-Wallis solo dice que hay alguna diferencia. El equivalente de Tukey es comparar las parejas con **pruebas de Wilcoxon** y **corregir** los valores p por comparaciones múltiples. Eso hace `pairwise.wilcox.test()`:

- Cada celda es el valor p corregido de una pareja. Por ejemplo, chino frente a italiano: p = 0,00007.
- `p.adjust.method = "holm"` aplica la corrección de **Holm**, una versión de Bonferroni menos estricta (se puede usar también `"bonferroni"`).
- `exact = FALSE` evita un aviso de R cuando hay empates, muy habituales en los recuentos.

En nuestros datos, el **italiano** difiere de las otras tres lenguas (comete menos errores: p = 0,006 frente al árabe, 0,00007 frente al chino y 0,004 frente al inglés). Entre árabe, chino e inglés no hay diferencias significativas. Otra opción muy usada es la **prueba de Dunn** (paquete `FSA`, función `dunnTest()`).

La tabla resume la sesión:

- **Dos variables categóricas**: chi cuadrado o Fisher; los residuos indican dónde está la asociación.
- **Una numérica y tres o más grupos**: ANOVA y, después, Tukey.
- **Lo mismo, sin normalidad**: Kruskal-Wallis y, después, Wilcoxon por parejas con corrección.

4.17 Práctica 4 (20 min)

1. ¿Depende el uso del subjuntivo del **nivel**? Haz la tabla y el χ^2.
2. Revisa los residuos estandarizados: ¿qué niveles se desvían?
3. Dibuja el mosaico (`mosaicplot(t4, shade = TRUE)`) y calcula la V de Cramér.
4. ¿Difiere la nota del DELE según el **nivel**? Haz un ANOVA y un Tukey.
5. ¿Difieren los errores de *ser/estar* según el nivel? (Kruskal-Wallis)

4.18 Práctica 4: soluciones

Soluciones de la práctica 4

```
t4 <- table(ele$nivel, ele$subjuntivo)
chisq.test(t4); chisq.test(t4)$stdres
mosaicplot(t4, shade = TRUE); v_cramer(t4)
m4 <- aov(nota_dele ~ nivel, data = ele)
summary(m4); TukeyHSD(m4)
kruskal.test(err_ser_estar ~ nivel, data = ele)
```

En pocas palabras

Una posible solución para cada ejercicio de la práctica 4:

1. Tabla de nivel por subjuntivo y prueba de chi cuadrado.
2. Residuos estandarizados con `$stdres`: muestran qué niveles aciertan más o menos de lo esperado.
3. `mosaicplot(..., shade = TRUE)` dibuja la tabla coloreando las celdas según sus residuos, y `v_cramer()` mide la fuerza de la asociación.
4. ANOVA de la nota por nivel y Tukey para ver qué niveles difieren.
5. Kruskal-Wallis para los errores de *ser/estar* por nivel.

5 Árboles de decisión

Sesión 5 · 60 minutos

> **La pregunta de esta sesión**
>
> ¿Qué perfil de aprendiente usa bien el subjuntivo? Que los datos hagan las preguntas.

Contenidos: Anatomía de un árbol · Impureza de Gini · El mejor corte · Crecer y podar · Sobreajuste · Matriz de confusión · Árboles de regresión.

5.1 Un árbol es un juego de preguntas

Antes elegimos la prueba con un **árbol hecho por nosotros**. Ahora será **R** quien construya el árbol a partir de los datos.

- **La pregunta del curso**: ¿Quién usa correctamente el subjuntivo en un contexto obligatorio?
- **Lo que hace el árbol**: Busca la **pregunta de sí/no** que mejor separa a quienes aciertan de quienes fallan… y vuelve a preguntar dentro de cada grupo.
- **Lo que obtenemos**: Reglas que se leen en voz alta: «*Si tiene más de 243 horas y es C1, acierta el 82 %*».



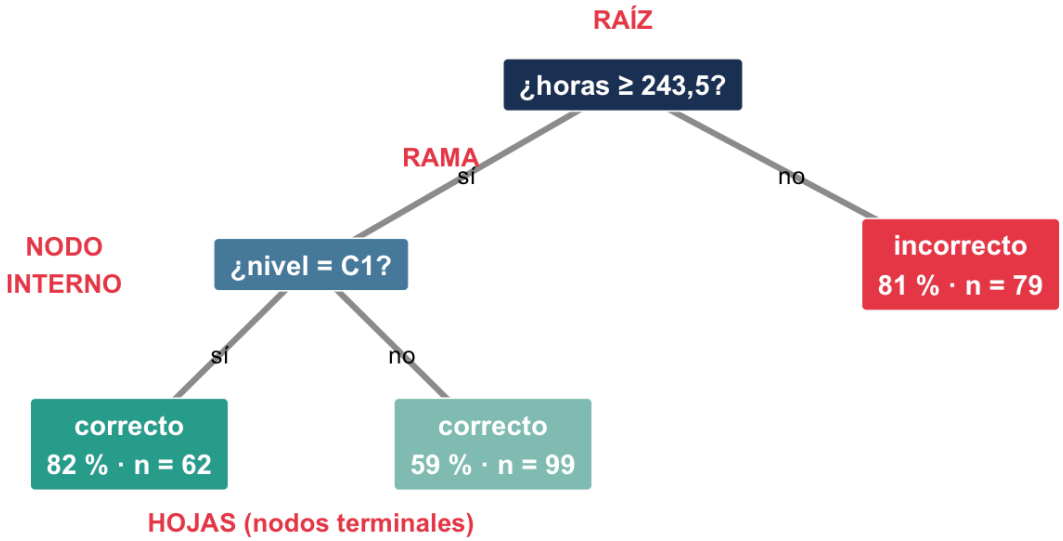
En pocas palabras

Un **árbol de decisión** funciona como el juego de adivinar un personaje con preguntas de sí o no. Cada pregunta divide a las personas en dos grupos, y dentro de cada grupo se vuelve a preguntar.

En la sesión 3 construimos nosotros un árbol para elegir la prueba estadística. Ahora será R quien lo construya a partir de los datos: buscará automáticamente qué preguntas separan mejor a los aprendientes que usan bien el subjuntivo de los que no.

La gran ventaja es que el resultado se lee como reglas en lenguaje natural: «si tiene más de 243 horas y es C1, acierta el 82 %». Cualquier docente puede entenderlo sin saber estadística.

5.2 Anatomía de un árbol



En pocas palabras

Las partes de un árbol de decisión, con un fragmento del árbol real que construiremos:

- **Raíz**: el punto de partida, con todos los aprendientes. Contiene la primera pregunta, la más informativa.
- **Ramas**: las respuestas a cada pregunta. En los gráficos de `rpart.plot`, la rama izquierda es siempre el «sí».
- **Nodos internos**: grupos intermedios en los que se hace otra pregunta.
- **Hojas**: los grupos finales, donde ya no se pregunta más. Cada hoja da una predicción y el porcentaje de casos que la cumplen.

Una hoja con un 82 % de aciertos es bastante **pura**: casi todos sus miembros se comportan igual. Una con un 59 % es **impura**: el árbol duda.

5.3 Dos tipos de árbol

- **Árbol de clasificación**: Respuesta **categórica** *subjuntivo: correcto / incorrecto* Cada hoja predice una **clase** y una proporción. `rpart(..., method = "class")`
- **Árbol de regresión**: Respuesta **numérica** *nota_dele* Cada hoja predice una **media**. `rpart(...)` (método «anova»)

! Idea clave

El paquete `rpart` (*recursive partitioning*) hace las dos cosas. `rpart.plot` los dibuja.

En pocas palabras

Hay dos tipos de árbol según lo que queramos predecir:

- **Árbol de clasificación**: la respuesta es una **categoría**, como subjuntivo correcto o incorrecto. Cada hoja predice la categoría más frecuente en su grupo y su proporción.
- **Árbol de regresión**: la respuesta es un **número**, como la nota del DELE. Cada hoja predice la media de las personas de su grupo.

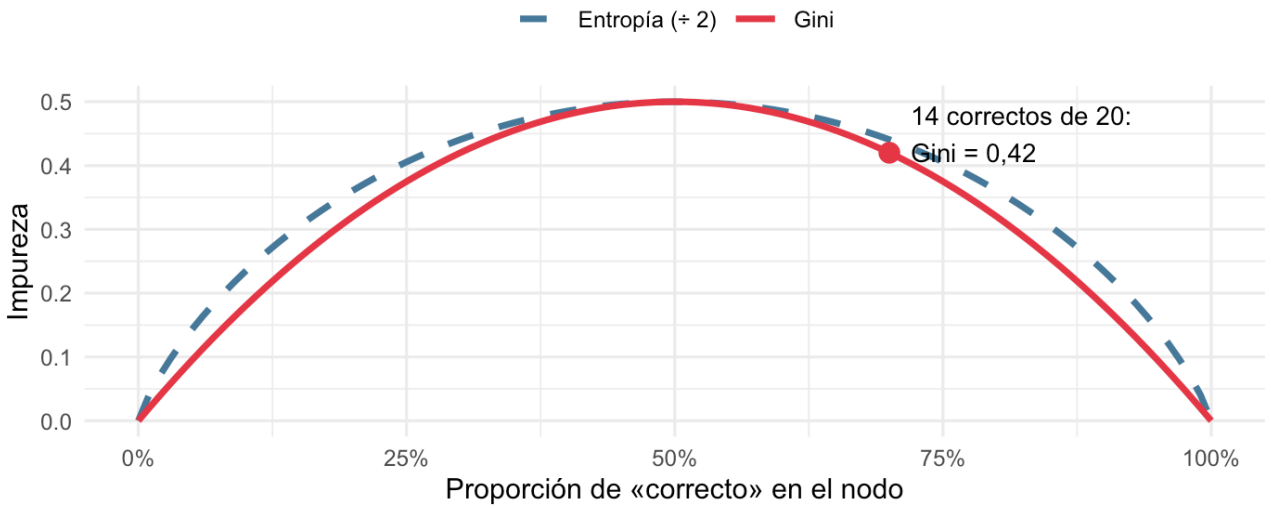
El paquete `rpart` (de *recursive partitioning*, partición recursiva) construye los dos. Con `method = "class"` hace un árbol de clasificación; si la respuesta es numérica, hace uno de regresión. El paquete `rpart.plot` los dibuja.

(«Regresión» es el nombre que recibe en estadística cualquier método para predecir un **número**. En el material Plus se presenta la regresión lineal, que lo hace con una recta en lugar de con un árbol.)

Para decidir qué preguntas hace, el árbol necesita medir lo **mezclado** que queda cada grupo: es la **impureza**, que veremos en el apartado siguiente.

5.4 ¿Qué es una buena pregunta? La impureza

Impureza = cuánto se mezclan, en un grupo, quienes aciertan y quienes fallan. **Gini** = probabilidad de equivocarse si a una persona del grupo le ponemos una etiqueta al azar según las proporciones del grupo. **Entropía** = cuánta incertidumbre hay sobre la clase de una persona del grupo (en bits).



En pocas palabras

El problema. Un árbol hace preguntas de sí o no («¿tiene más de 243 horas?», «¿es C1?») para separar a quienes usan bien el subjuntivo de quienes lo usan mal. Para elegir la mejor pregunta necesita un número que diga lo **mezclado** que queda cada grupo después de preguntar. Ese número es la **impureza**.

- Un grupo **puro**, en el que todos aciertan (o todos fallan), tiene impureza 0: si sabemos que alguien está en ese grupo, sabemos qué hará.
- Un grupo mezclado al 50 %, la peor situación, tiene la impureza máxima: no sabemos qué predecir.

El índice de Gini es la medida que usa `rpart` por defecto. Se calcula como $1 - p^2 - (1 - p)^2$, donde p es la proporción de aciertos. Tiene una lectura intuitiva: es la **probabilidad de equivocarnos** si tomamos a una persona al azar del grupo y adivinamos su clase sorteándola con las proporciones del grupo. Con el grupo inicial del gráfico, 14 aciertos y 6 fallos de 20 (p = 0,7):

Gini = $1 - 0{,}7^2 - 0{,}3^2 = 1 - 0{,}49 - 0{,}09 =$ **0,42**

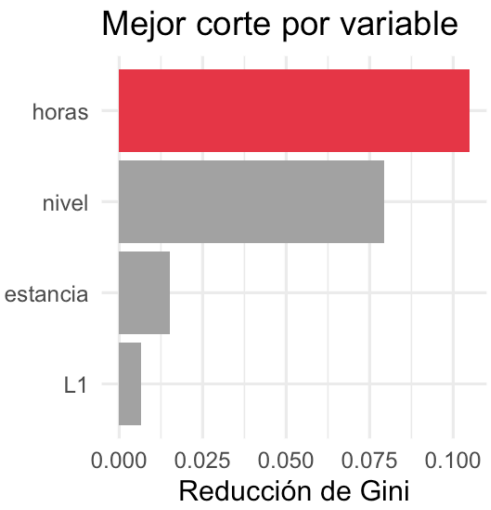
Es decir, nos equivocaríamos un 42 % de las veces. Con 20 aciertos de 20, Gini = 0 (nunca nos equivocamos); con 10 de 20, Gini = 0,5 (el máximo con dos clases).

La entropía es otra medida con la misma idea, procedente de la teoría de la información. Mide la incertidumbre en **bits**: el número medio de preguntas de sí o no que necesitaríamos para adivinar la clase de una persona del grupo. Con un grupo 50/50 hace falta una pregunta completa (1 bit); con un grupo puro, ninguna (0 bits). Con 14 de 20, la entropía es 0,88. Se puede activar en el gráfico: su curva tiene la misma forma que la de Gini, solo que llega hasta 1 en lugar de 0,5. En la práctica, las dos suelen elegir las mismas preguntas. En `rpart` se usa la entropía con `parms = list(split = "information")`.

Para qué sirven en un árbol. Una **buena pregunta** es la que divide un grupo mezclado en dos grupos lo más puros posible. El árbol calcula la impureza de los dos grupos que resultarían de cada pregunta posible, la promedia (según cuántas personas caen en cada lado) y la compara con la del grupo de partida. La diferencia es la **ganancia**: cuánto ha reducido la impureza esa pregunta. Se queda con la pregunta de mayor ganancia. El apartado siguiente lo hace en directo con nuestros datos.

5.5 El mejor corte, en directo





En pocas palabras

Esta herramienta muestra cómo elige el árbol su primera pregunta.

Con la variable **horas**, se puede mover el punto de corte, por ejemplo «¿más o menos de 200 horas?». Los dos recuadros de la derecha muestran los grupos resultantes, y la curva inferior indica cuánto reduce la impureza cada posible corte.

El botón **«Buscar el mejor corte»** prueba todos los valores y se detiene en **243,5 horas**, exactamente el que elige R.

Las cuentas de ese mejor corte, con el índice de Gini:

- **Antes de preguntar**: 124 aciertos de 240 (51,7 %). Gini = $1 - 0{,}517^2 - 0{,}483^2$ = **0,499**, casi la mezcla máxima.
- **Menos de 243,5 horas**: 79 aprendientes, de los que solo aciertan 15 (19 %). Gini = **0,308**.
- **243,5 horas o más**: 161 aprendientes, de los que aciertan 109 (68 %). Gini = **0,437**.
- **Impureza media después de preguntar**, ponderada por el tamaño de cada grupo: (79 × 0,308 + 161 × 0,437) / 240 = **0,395**.
- **Ganancia**: 0,499 − 0,395 = **0,105**. Es el valor máximo de la curva inferior.

Después se puede repetir con el nivel, la L1 y la estancia. Al **comparar** las cuatro variables, las horas consiguen la mayor reducción de impureza. Por eso son la primera pregunta del árbol: la raíz.

5.6 El algoritmo en cuatro pasos

- **1 · Probar**: Todas las variables y todos los puntos de corte posibles.
- **2 · Elegir**: El corte que más reduce la impureza (Gini).
- **3 · Repetir**: Dentro de cada hijo, como si fuera un conjunto de datos nuevo (recursivo).
- **4 · Parar**: Cuando mejorar «cuesta» más que el parámetro de complejidad **cp**, o hay pocos casos.

! Idea clave

Un árbol muy profundo **memoriza** los datos (sobreajuste). Hay que **podarlo** y evaluarlo con datos nuevos.

En pocas palabras

El procedimiento que sigue R para construir el árbol se resume en cuatro pasos:

1. **Probar** todas las variables y todos los puntos de corte posibles.
2. **Elegir** el corte que más reduce la impureza.
3. **Repetir** el proceso dentro de cada uno de los dos grupos resultantes, como si cada uno fuera un conjunto de datos nuevo. Por eso se llama *recursivo*.
4. **Parar** cuando una nueva división mejora muy poco (menos de lo que marca el parámetro de complejidad, **cp**) o cuando quedan muy pocas personas en un grupo.

Si no se detiene a tiempo, el árbol sigue dividiendo hasta **memorizar** los datos, incluido el ruido. Es el **sobreajuste**, y la solución es **podar** el árbol.

5.7 Construye tu primer árbol

Paso 1 · Ajustar el árbol de clasificación

```r
set.seed(2026)   # la validación cruzada interna usa números aleatorios
arbol <- rpart(subjuntivo ~ horas + estancia + L1 + nivel,
               data = ele, method = "class", cp = 0.01)
```

Paso 2 · Verlo en texto

```r
arbol
#> n= 240
#>
#> node), split, n, loss, yval, (yprob)
#>       * denotes terminal node
#>
#>  1) root 240 116 correcto (0.5166667 0.4833333)
#>    2) horas>=243.5 161  52 correcto (0.6770186 0.3229814)
#>      4) nivel=C1 62  11 correcto (0.8225806 0.1774194) *
#>      5) nivel=A2,B1,B2 99  41 correcto (0.5858586 0.4141414)
#>       10) horas>=351.5 19   3 correcto (0.8421053 0.1578947) *
#>       11) horas< 351.5 80  38 correcto (0.5250000 0.4750000)
#>         22) horas< 256.5 8   1 correcto (0.8750000 0.1250000) *
#>         23) horas>=256.5 72  35 incorrecto (0.4861111 0.5138889)
#>           46) L1=inglés,italiano 33  14 correcto (0.5757576 0.4242424) *
#>           47) L1=árabe,chino 39  16 incorrecto (0.4102564 0.5897436)
#>             94) horas< 293.5 14   6 correcto (0.5714286 0.4285714) *
#>             95) horas>=293.5 25   8 incorrecto (0.3200000 0.6800000) *
#>    3) horas< 243.5 79  15 incorrecto (0.1898734 0.8101266) *
```

En pocas palabras

`rpart()` construye el árbol con una fórmula como las de `t.test()` o `aov()`: a la izquierda de la tilde (`~`), lo que queremos predecir; a la derecha, las variables candidatas a hacer preguntas, unidas con `+`. Se lee: el subjuntivo en función de las horas, la estancia, la L1 y el nivel.

- `method = "class"`: es un árbol de clasificación, porque la respuesta es una categoría.
- `cp = 0.01`: el **parámetro de complejidad**. Una división solo se hace si mejora el árbol al menos un 1 %. Cuanto más bajo, más grande será el árbol.
- `set.seed(2026)`: fija los números aleatorios que usa la validación interna, para que todos obtengamos el mismo resultado.

Al escribir `arbol` se imprime en forma de texto. El apartado siguiente explica cómo leerlo.

5.8 Cómo leer `print(arbol)`

```
n= 240

node), split, n, loss, yval, (yprob)
      * denotes terminal node

 1) root 240 116 correcto (0.5166667 0.4833333)
   2) horas>=243.5 161  52 correcto (0.6770186 0.3229814)
     4) nivel=C1 62  11 correcto (0.8225806 0.1774194) *
     5) nivel=A2,B1,B2 99  41 correcto (0.5858586 0.4141414)
      10) horas>=351.5 19   3 correcto (0.8421053 0.1578947) *
      11) horas< 351.5 80  38 correcto (0.5250000 0.4750000)
        22) horas< 256.5 8   1 correcto (0.8750000 0.1250000) *
        23) horas>=256.5 72  35 incorrecto (0.4861111 0.5138889)
```

```
          46) L1=inglés,italiano 33  14 correcto (0.5757576 0.4242424) *
          47) L1=árabe,chino 39  16 incorrecto (0.4102564 0.5897436)
            94) horas< 293.5 14   6 correcto (0.5714286 0.4285714) *
            95) horas>=293.5 25   8 incorrecto (0.3200000 0.6800000) *
   3) horas< 243.5 79  15 incorrecto (0.1898734 0.8101266) *
```

Cómo se lee cada parte de la salida:

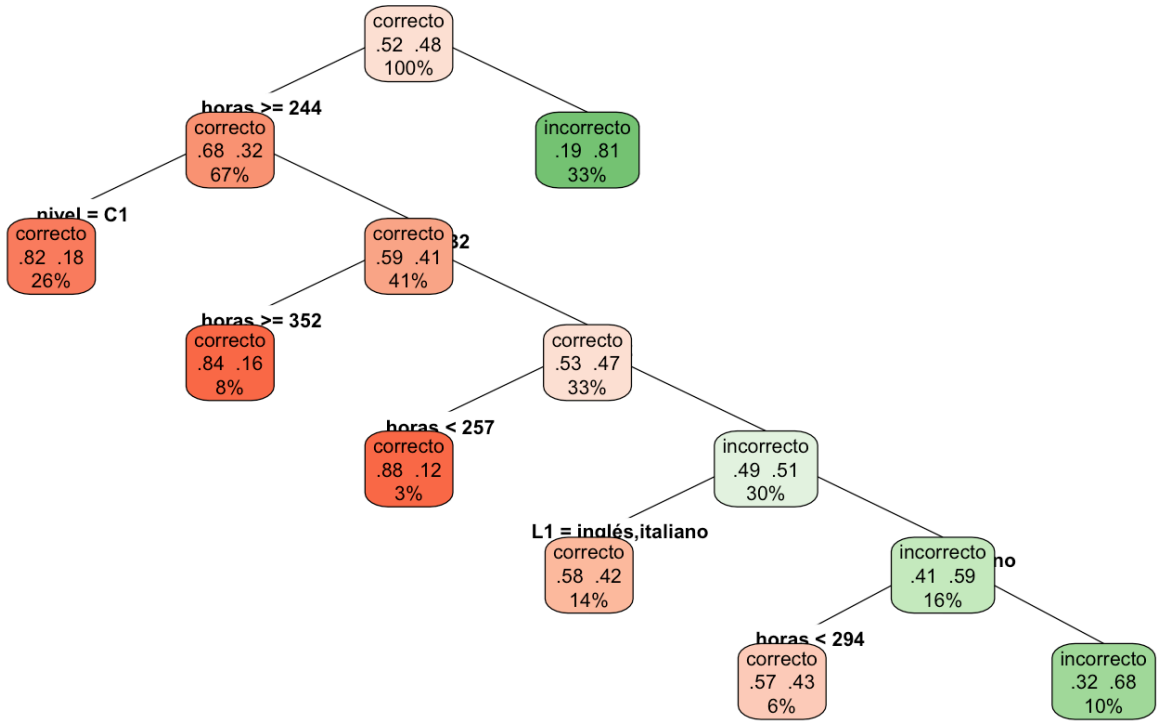
- `node), split, n, loss, yval, (yprob)` **La cabecera** explica cada columna: número de nodo, condición, n de casos, casos mal clasificados (**loss**), clase predicha (**yval**) y proporciones de cada clase.
- `1) root 240 116 correcto (0.5166667 0.4833333)` **Nodo 1, la raíz**: los 240 aprendientes. Si predijéramos siempre «correcto» fallaríamos 116 veces. Hay un 51,7 % de correctos.
- `2) horas>=243.5 161 52 correcto (0.6770186 0.3229814)` **Primera división**: 161 aprendientes con 243,5 horas o más. El 67,7 % acierta. La sangría indica la profundidad.
- `4) nivel=C1 62 11 correcto (0.8225806 0.1774194) *` **Una hoja** (el asterisco): con muchas horas y nivel C1, el 82 % acierta. Solo 11 de 62 fallan.
- `23) horas>=256.5 72 35 incorrecto (0.4861111 0.5138889)` **Sospechoso**: un corte en 256,5 horas justo después de otro en 243,5, con solo 8 personas a un lado. Huele a **sobreajuste**: el árbol está aprendiendo el ruido.
- `3) horas< 243.5 79 15 incorrecto (0.1898734 0.8101266) *` **La otra rama de la raíz**: con menos de 243,5 horas, el 81 % **falla** el subjuntivo. Es una hoja muy pura y muy útil.

> En pocas palabras
>
> La salida en texto del árbol tiene una línea por nodo.
>
> Cada línea indica: el número de nodo, la condición que lo define, cuántas personas hay, cuántas se clasificarían mal (**loss**), la predicción y las proporciones de cada categoría. Un **asterisco** marca las hojas, y la **sangría**, la profundidad.
>
> Las líneas clave:
>
> - **Raíz**: 240 aprendientes, el 51,7 % acierta.
> - Con **243,5 horas o más**: acierta el 67,7 %. Si además es **C1**, el 82 %.
> - Con **menos de 243,5 horas**: falla el 81 %. Es la regla más clara y útil.
>
> Algunos cortes muy próximos entre sí, como 243,5 y 256,5 horas, hacen sospechar de sobreajuste.

5.9 Dibújalo

Paso 3 · rpart.plot()

```r
rpart.plot(arbol, type = 4, extra = 104, box.palette = "RdGn",
           fallen.leaves = FALSE, tweak = 1.5)
```



`type = 4`: etiqueta las dos ramas · `extra = 104`: probabilidades de cada clase y % de casos en cada nodo.

En pocas palabras

`rpart.plot()` dibuja el árbol. Es mucho más fácil de leer que la versión en texto.

Cada recuadro es un nodo y muestra la predicción (correcto o incorrecto), la proporción de cada categoría y el porcentaje de aprendientes que llegan a él. El color indica la predicción: verde para «correcto» y rojo para «incorrecto». Cuanto más intenso, más puro es el nodo.

Las opciones del código:

- `type = 4`: escribe la condición en las dos ramas de cada división.
- `extra = 104`: muestra las probabilidades de cada clase y el porcentaje de casos.
- `box.palette = "RdGn"`: la escala de rojo a verde.
- `tweak`: el tamaño del texto.

5.10 Mira cómo crece

Profundidad	Hojas	Acierto en estos datos
0	1	51,7 %
1	2	72,1 %
2	3	72,1 %
3	4	72,1 %
4	5	72,9 %
5	6	75 %
6	7	75,8 %

En pocas palabras

Esta animación construye el árbol nivel a nivel y muestra su **acierto** en cada paso.

- **Profundidad 0**, sin preguntas: el árbol predice siempre «correcto», la categoría mayoritaria, y acierta el 51,7 %, igual que lanzar una moneda.
- **Profundidad 1**, con una sola pregunta (las horas): el acierto sube al 72 %. Es un salto enorme.
- **Profundidades siguientes**: cada nueva pregunta mejora muy poco.

La pregunta que queda abierta es importante: esas mejoras pequeñas, ¿reflejan algo real o el árbol está aprendiendo las particularidades de estos 240 aprendientes? Si es lo segundo, no servirán con aprendientes nuevos.

5.11 Explora las reglas

Regla	Predicción	% correcto	n
horas≥ 243,5 y nivel=A2, B1, B2 y horas< 351,5 y horas< 256,5	correcto	88	8
horas≥ 243,5 y nivel=A2, B1, B2 y horas≥ 351,5	correcto	84	19
horas≥ 243,5 y nivel=C1	correcto	82	62
horas≥ 243,5 y nivel=A2, B1, B2 y horas< 351,5 y horas≥ 256,5 y L1=inglés, italiano	correcto	58	33
horas≥ 243,5 y nivel=A2, B1, B2 y horas< 351,5 y horas≥ 256,5 y L1=árabe, chino y horas< 293,5	correcto	57	14
horas≥ 243,5 y nivel=A2, B1, B2 y horas< 351,5 y horas≥ 256,5 y L1=árabe, chino y horas≥ 293,5	incorrecto	32	25
horas< 243,5	incorrecto	19	79

En pocas palabras

Cada hoja del árbol equivale a una **regla** que se puede leer en voz alta.

Por ejemplo: «SI tiene 243,5 horas o más Y su nivel es C1, ENTONCES acierta el subjuntivo en el 82 % de los casos».

Esta es la mayor ventaja de los árboles: transforman un modelo estadístico en reglas comprensibles para cualquier docente. Para la práctica en el aula, la regla más útil es probablemente la más sencilla: con menos de 243,5 horas, el 81 % falla el subjuntivo. Es un umbral claro a partir del cual reforzar su enseñanza.

5.12 Sigue a un aprendiente por el árbol

Prueba esto en R: predecir para aprendientes nuevos

```
nuevos <- tibble(horas = c(300, 200, 400),
                 nivel = factor(c("B1", "A2", "C1"), levels = levels(ele$nivel)),
                 L1 = c("chino", "inglés", "italiano"),
                 estancia = c("no", "sí", "no"))
rpart.predict(arbol, nuevos, rules = TRUE)
#>    correcto incorrecto
#> 1 0.3200000  0.6800000
#> 2 0.1898734  0.8101266
#> 3 0.8225806  0.1774194
#>
#> 1 because horas is 294 to 352 & nivel is A2 or B1 or B2 & L1 is árabe or chino
```

```
#> 2 because horas <  244
#> 3 because horas >=        244 & nivel is             C1
```

En pocas palabras

Esta herramienta muestra cómo usa el árbol para **predecir**: se define un aprendiente con sus horas, su nivel, su L1 y su estancia, y una ficha recorre el árbol respondiendo a cada pregunta hasta llegar a una hoja, que da la predicción.

Un detalle interesante: al cambiar la **estancia** no ocurre nada, porque el árbol no la usa en ninguna pregunta. Las horas y el nivel ya separan mejor a los aprendientes, y la estancia no aporta información adicional suficiente.

Los árboles, por tanto, también **seleccionan variables**: solo usan las que ayudan a separar los grupos.

5.13 Las reglas, en texto y en tabla

Paso 4 · Una regla por hoja

```
rpart.rules(arbol, cover = TRUE)
#>  subjuntivo                                                                                cover
#>        0.12 when horas is 244 to 257 & nivel is A2 or B1 or B2                              3%
#>        0.16 when horas >=        352 & nivel is A2 or B1 or B2                              8%
#>        0.18 when horas >=        244 & nivel is             C1                             26%
#>        0.42 when horas is 257 to 352 & nivel is A2 or B1 or B2 & L1 is inglés or italiano   14%
#>        0.43 when horas is 257 to 294 & nivel is A2 or B1 or B2 & L1 is     árabe or chino    6%
#>        0.68 when horas is 294 to 352 & nivel is A2 or B1 or B2 & L1 is     árabe or chino   10%
#>        0.81 when horas <  244                                                               33%
```

En pocas palabras

`rpart.rules()` extrae todas las reglas del árbol en forma de tabla, con una fila por hoja.

Cada fila muestra:

- La **probabilidad** de la categoría predicha en esa hoja.
- Las **condiciones** que definen la hoja, unidas por «&» (y).
- Con `cover = TRUE`, el **porcentaje de aprendientes** que cumplen la regla.

Es una forma cómoda de pasar el árbol a un informe o a un artículo, o de comprobar qué reglas afectan a más personas. Una regla muy precisa pero que solo cumplen tres personas es mucho menos útil que una que cubre a un tercio de la muestra.

5.14 ¿Acierta?

Paso 5 · Predecir y comparar con la realidad

```
pred <- predict(arbol, type = "class")
table(real = ele$subjuntivo, predicho = pred)
#>             predicho
#> real         correcto incorrecto
#>   correcto        101         23
#>   incorrecto       35         81
mean(pred == ele$subjuntivo)      # exactitud en los MISMOS datos
#> [1] 0.7583333
```

```r
mean(ele$subjuntivo == "correcto")   # sin árbol: predecir siempre la clase mayoritaria
#> [1] 0.5166667
```

> ! Idea clave
>
> Pasamos de un 52 % a un 76 %. Pero lo hemos medido con los datos con los que **aprendió**: es como poner el examen con las preguntas resueltas en clase.

> En pocas palabras
>
> Para saber si el árbol es útil comparamos sus predicciones con la realidad.
>
> - `predict()` obtiene la predicción del árbol para cada aprendiente.
> - `table()` cruza lo real con lo predicho: es la **matriz de confusión**. En la diagonal están los aciertos.
> - `mean(pred == ele$subjuntivo)` calcula la **exactitud**, la proporción de aciertos: un 76 %.
>
> Hay que compararla con un punto de partida: predecir siempre la categoría más frecuente acierta un 52 %. El árbol mejora mucho.
>
> Pero hay una trampa: hemos medido el acierto con los mismos datos con los que el árbol aprendió. Es como poner un examen con las preguntas resueltas en clase.

5.15 El sobreajuste: la tabla de complejidad

La idea: dejamos crecer un árbol enorme y comprobamos, tamaño a tamaño, cuánto acierta con los datos que ya conoce y cuánto con aprendientes que **no ha visto**.

Paso 6 · Un árbol sin freno (cp = 0)

```r
set.seed(2026)
arbol_total <- rpart(subjuntivo ~ horas + estancia + L1 + nivel,
                     data = ele, method = "class",
                     control = rpart.control(cp = 0, minsplit = 6))
printcp(arbol_total)
#>
#> Classification tree:
#> rpart(formula = subjuntivo ~ horas + estancia + L1 + nivel, data = ele,
#>     method = "class", control = rpart.control(cp = 0, minsplit = 6))
#>
#> Variables actually used in tree construction:
#> [1] estancia horas    L1       nivel
#>
#> Root node error: 116/240 = 0.48333
#>
#> n= 240
#>
#>          CP nsplit rel error  xerror     xstd
#> 1 0.4224138      0   1.00000 1.00000 0.066738
#> 2 0.0215517      1   0.57759 0.58621 0.060180
#> 3 0.0129310      3   0.53448 0.66379 0.062341
#> 4 0.0114943     11   0.41379 0.68103 0.062757
#> 5 0.0086207     14   0.37931 0.68103 0.062757
#> 6 0.0071839     19   0.33621 0.71552 0.063522
#> 7 0.0043103     25   0.29310 0.75862 0.064358
#> 8 0.0034483     31   0.26724 0.84483 0.065644
#> 9 0.0000000     36   0.25000 0.87069 0.065933
```

Paso 7 · La misma tabla, en porcentaje de acierto

```
error_raiz <- 116 / 240      # errores sin preguntar nada
as.data.frame(arbol_total$cptable) |>
  transmute(hojas = nsplit + 1,
            acierto_conocidos = round(100 * (1 - `rel error` * error_raiz)),
            acierto_nuevos    = round(100 * (1 - xerror * error_raiz)))
#>   hojas acierto_conocidos acierto_nuevos
#> 1     1               52             52
#> 2     2               72             72
#> 3     4               74             68
#> 4    12               80             67
#> 5    15               82             67
#> 6    20               84             65
#> 7    26               86             63
#> 8    32               87             59
#> 9    37               88             58
```

En pocas palabras

Qué buscamos. Un árbol puede seguir haciendo preguntas hasta acertar casi con todos los aprendientes que conoce. La pregunta es si esas preguntas extra le sirven con **aprendientes nuevos** o si solo está memorizando los 240 que tiene delante. Para averiguarlo construimos un árbol sin freno: `cp = 0` y `minsplit = 6` le permiten dividir hasta tener grupos de muy pocas personas (`minsplit = 6`: no divide un grupo con menos de 6).

La tabla de `printcp()` tiene una fila por cada tamaño de árbol que R ha probado, del más simple (arriba) al más grande (abajo). Antes de la tabla, R escribe *Root node error: 116/240*: sin hacer ninguna pregunta, prediciendo siempre «correcto», fallaríamos con 116 de los 240 aprendientes (un 48 %). Todos los errores de la tabla se expresan **en relación con ese punto de partida**:

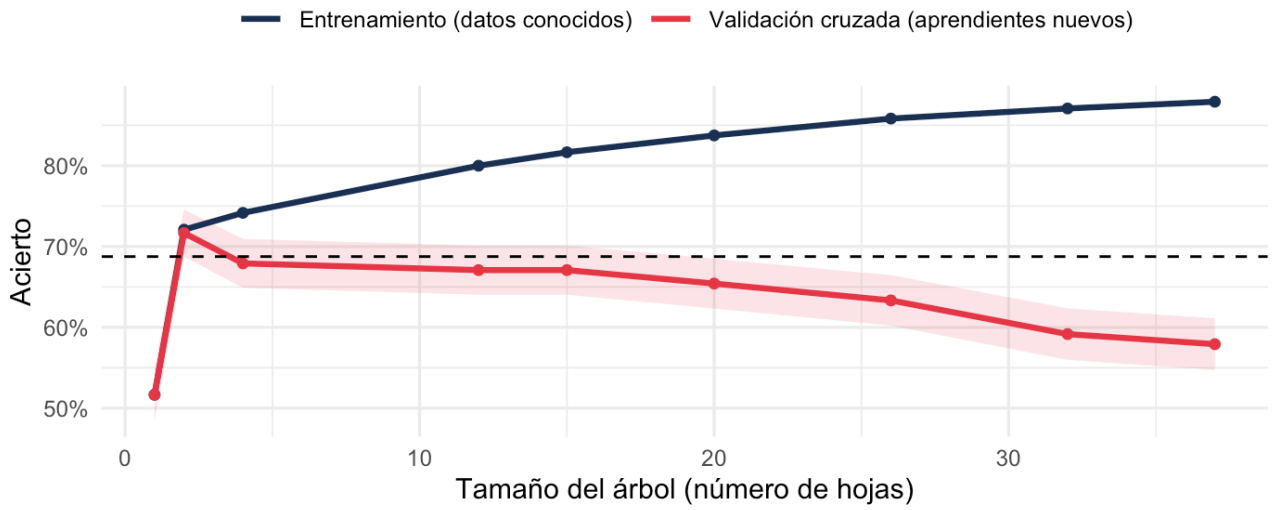
- **nsplit**: el número de preguntas (cortes). El árbol tiene nsplit + 1 hojas.
- **rel error**: el error con los **datos de entrenamiento** (los 240 que el árbol ya conoce), como fracción de 116. En la segunda fila, nsplit = 1 y rel error = 0,58: con una sola pregunta, el árbol comete el 58 % de los 116 errores iniciales, unos 67 errores, y acierta con el 72 %. Esta columna **siempre baja**: en la última fila (36 cortes) es 0,25, solo 29 errores.
- **xerror**: el mismo error, pero estimado con **validación cruzada**, es decir, simulando aprendientes nuevos. R reparte a los 240 en 10 grupos de 24; construye el árbol con 9 grupos (216 personas) y lo examina con el décimo, que no ha visto; repite la operación 10 veces, cambiando el grupo examinado, y promedia. Con una pregunta, xerror = 0,59: casi igual que en entrenamiento. Con 36 cortes, xerror = 0,87: ¡mucho peor que con una sola pregunta!
- **xstd**: el margen de incertidumbre de xerror (su error estándar).
- **CP**: el parámetro de complejidad de cada fila; lo usaremos para podar.

El paso 7 traduce la tabla a porcentajes de acierto, que se entienden mejor: acierto = 1 − error relativo × 116/240. El resultado:

Hojas	Acierto con los datos conocidos	Acierto estimado con aprendientes nuevos
1	52 %	52 %
2	72 %	72 %
4	74 %	68 %
37	88 %	58 %

Esta es la definición de **sobreajuste**: el árbol grande acierta más con lo que ya conoce (88 %) y menos con lo nuevo (58 %), porque sus preguntas extra describen casualidades de estos 240 aprendientes, no patrones que se repitan en otros. Es como un estudiante que se aprende de memoria las respuestas de los exámenes de años anteriores: saca un 10 en ellos y suspende el examen nuevo.

5.16 Entrenamiento frente a validación cruzada



En pocas palabras

Este gráfico dibuja las dos últimas columnas del paso 7: el **porcentaje de acierto** según el tamaño del árbol (número de hojas, en el eje horizontal).

- **Línea azul: acierto con los datos de entrenamiento**, los 240 aprendientes con los que se construyó el árbol. Siempre sube: cuantas más hojas, mejor se ajusta a lo que ya conoce, hasta memorizarlo. Pasa del 52 % (1 hoja) al 88 % (37 hojas).
- **Línea roja: acierto estimado con aprendientes nuevos**, mediante la validación cruzada. Sube mucho con la primera pregunta (del 52 % al 72 %) y después **baja**: con 37 hojas, solo el 58 %.
- La **franja roja** alrededor de esa línea es su margen de incertidumbre (± 1 error estándar). La validación cruzada es una estimación, no una medida exacta.

La distancia entre las dos líneas es el **sobreajuste**: lo que el árbol «cree» que acierta frente a lo que acertaría de verdad con alumnos nuevos.

¿Qué tamaño elegimos? La regla de 1 EE (un error estándar). El árbol con mejor acierto estimado es el de 2 hojas. La regla dice: entre todos los árboles cuyo acierto con datos nuevos esté **dentro del margen de incertidumbre del mejor** (por encima de la línea discontinua), quédate con el **más sencillo**. Si dos árboles son indistinguibles dentro del margen de error, no hay razón para preferir el complicado. En nuestros datos, la regla elige el árbol con **un solo corte**, el de las horas.

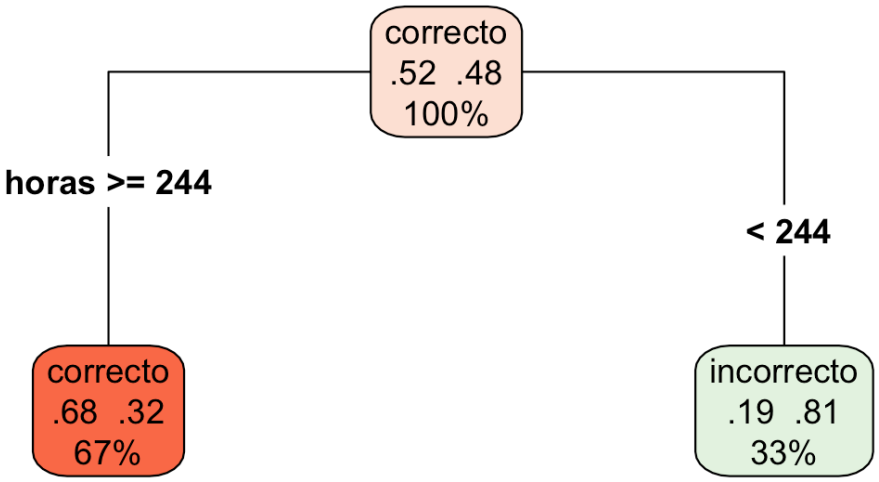
El deslizador recorre las filas de la tabla: para cada tamaño, el panel muestra los dos aciertos y avisa cuando hay sobreajuste.

5.17 Podar el árbol

Paso 8 · prune() con el cp elegido

```
tabla_cp <- arbol_total$cptable
minimo <- which.min(tabla_cp[, "xerror"])
umbral <- tabla_cp[minimo, "xerror"] +
          tabla_cp[minimo, "xstd"]
fila_1ee <- which(tabla_cp[, "xerror"] <= umbral)[1]
arbol_podado <- prune(arbol_total,
  cp = tabla_cp[fila_1ee, "CP"])
```

```
rpart.plot(arbol_podado, type = 4,
  extra = 104, box.palette = "RdGn")
```



En pocas palabras

Podar consiste en quitar al árbol grande las ramas que no mejoran la predicción con datos nuevos.

El código aplica la regla de 1 error estándar:

1. Localiza la fila de la tabla con el menor error de validación cruzada (`xerror`).
2. Calcula el umbral: ese error mínimo más su error estándar.
3. Elige la primera fila (el árbol más simple) que queda por debajo del umbral.
4. `prune()` recorta el árbol con el cp de esa fila.

El resultado es un árbol con una sola pregunta: ¿tiene 243,5 horas o más? Es menos espectacular que el árbol grande, pero más fiable con aprendientes nuevos.

5.18 Ponerlo a prueba con datos nuevos

Paso 9 · Separar entrenamiento (70 %) y prueba (30 %)

```
set.seed(2026)
idx <- sample(nrow(ele), size = 0.7 * nrow(ele))
entreno <- ele[idx, ]
prueba  <- ele[-idx, ]
arbol_e <- rpart(subjuntivo ~ horas + estancia + L1 + nivel, data = entreno,
```

```
                method = "class", control = rpart.control(cp = 0, minsplit = 6))
cp_opt <- arbol_e$cptable[which.min(arbol_e$cptable[, "xerror"]), "CP"]
arbol_p <- prune(arbol_e, cp = cp_opt)
acierto <- function(m, d) mean(predict(m, d, type = "class") == d$subjuntivo)
c(grande_entreno = acierto(arbol_e, entreno), grande_prueba = acierto(arbol_e, prueba),
  podado_entreno = acierto(arbol_p, entreno), podado_prueba = acierto(arbol_p, prueba)) |> round(3)
#> grande_entreno  grande_prueba podado_entreno  podado_prueba
#>          0.881          0.639          0.833          0.708
```

En pocas palabras

La forma más clara de evaluar un modelo es guardar datos que **no ve durante el entrenamiento**.

1. Se reparten los aprendientes al azar: el 70 % para **entrenar** y el 30 % para **probar**.
2. Se construye un árbol grande con los datos de entrenamiento y se poda.
3. La función `acierto()` calcula la exactitud de cada árbol en cada conjunto.

Los resultados lo dicen todo:

- El árbol **grande** acierta el 88 % en entrenamiento, pero solo el 64 % en prueba.
- El árbol **podado** acierta algo menos en entrenamiento (83 %), pero más en prueba (71 %).

El árbol grande había memorizado; el podado generaliza mejor.

5.19 La matriz de confusión

Árbol	Hojas	Acierto en entrenamiento	Acierto en prueba
Sin podar	24	88 %	64 %
Podado (mín. xerror)	12	83 %	71 %
Un solo corte	2	71 %	75 %

Realidad	Predice correcto	Predice incorrecto
correcto	25	15
incorrecto	11	21

En pocas palabras

La **matriz de confusión** cruza lo que pasó realmente con lo que predijo el árbol. Tiene cuatro celdas:

- **Verdaderos positivos** y **verdaderos negativos**: los aciertos.
- **Falsos positivos**: el árbol predice «correcto», pero el aprendiente falló.
- **Falsos negativos**: el árbol predice «incorrecto», pero el aprendiente acertó.

De la matriz salen tres medidas:

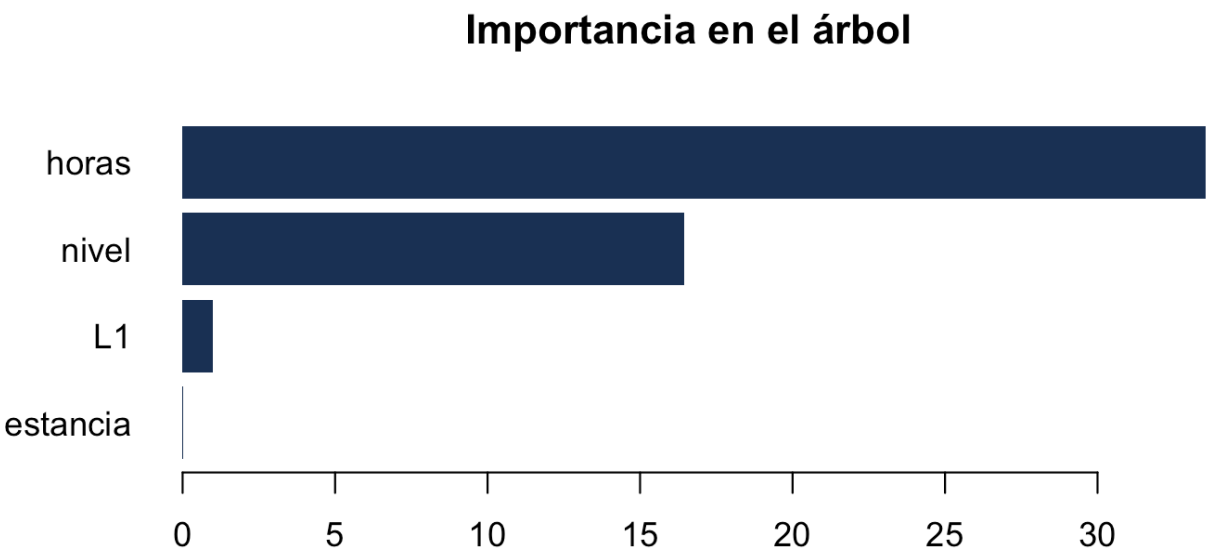
- **Exactitud**: proporción total de aciertos.
- **Sensibilidad**: de los que aciertan, cuántos detecta el árbol.
- **Especificidad**: de los que fallan, cuántos detecta.

Los botones permiten comparar el árbol grande y el podado con datos de entrenamiento y de prueba.

5.20 ¿Qué variables importan?

Paso 10 · Importancia de las variables

```r
imp <- arbol$variable.importance
round(imp, 1)
#>    horas    nivel       L1 estancia
#>     33.5     16.5      1.0      0.0
barplot(sort(imp), horiz = TRUE, las = 1,
        col = "#1D3557", border = NA,
        main = "Importancia en el árbol")
```



La importancia suma lo que cada variable reduce la impureza, también como **sustituta** (*surrogate*) en los cortes en que no fue elegida.

En pocas palabras

La **importancia de las variables** indica cuánto contribuye cada una a reducir la impureza a lo largo de todo el árbol.

En nuestro árbol, las **horas** son con diferencia la variable más importante, seguidas del **nivel**. La L1 apenas aporta y la estancia no aporta nada.

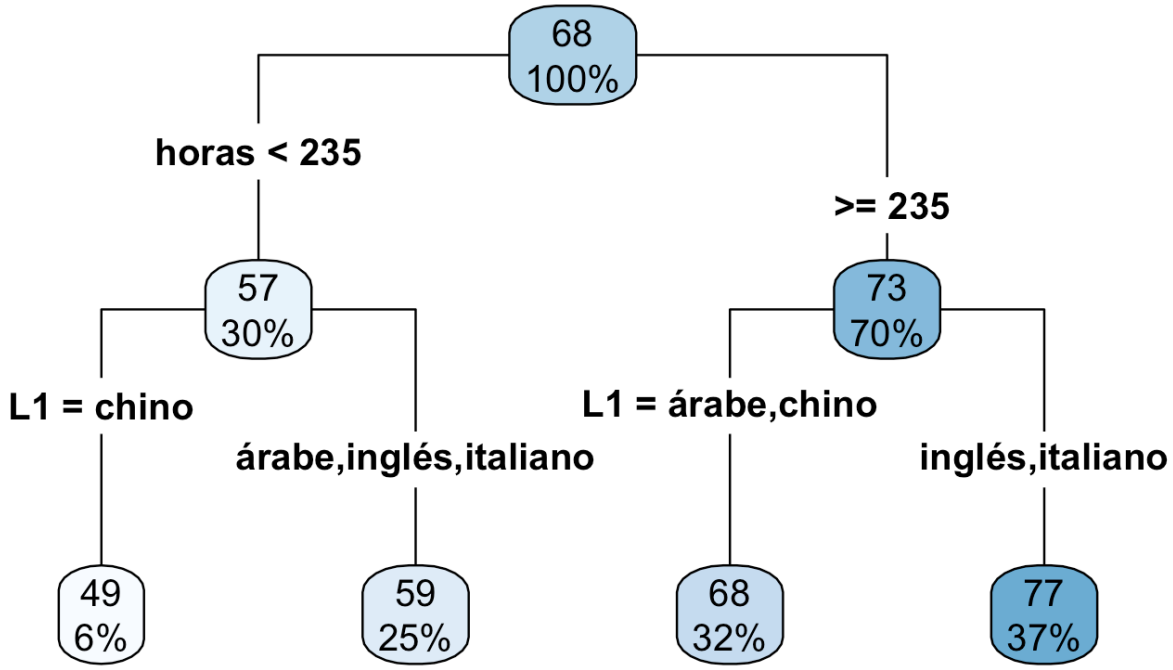
Un detalle técnico: la importancia no solo suma las divisiones en las que se eligió la variable. También tiene en cuenta las divisiones en las que habría sido una buena **sustituta** (*surrogate*). Por eso una variable puede tener cierta importancia aunque no aparezca en el dibujo del árbol.

Es una forma rápida de saber qué factores pesan más en la predicción.

5.21 Árbol de regresión: la nota

Paso 11 · Respuesta numérica

```r
arbol_nota <- rpart(nota_dele ~ horas + estancia + L1, data = ele, cp = 0.03)
rpart.plot(arbol_nota, type = 4, box.palette = "Blues", tweak = 1.1)
```



En pocas palabras

Ahora la respuesta es numérica: la **nota del DELE**. `rpart()` detecta que la variable es un número y construye un **árbol de regresión**. Cada hoja predice la **nota media** de los aprendientes que llegan a ella.

En el árbol resultante:

- La primera pregunta vuelve a ser las **horas**, con un corte en 235.
- Después, en ambas ramas, interviene la **L1**.
- La nota prevista va de unos 49 puntos (pocas horas, sinohablantes) a unos 77 (muchas horas, anglo-parlantes o italohablantes).

El parámetro `cp = 0.03` lo mantiene pequeño y fácil de leer. Los colores azules más intensos indican notas más altas.

5.22 Predice la nota de un aprendiente

Prueba esto en R (usa los mismos tres aprendientes)

```
predict(arbol_nota, nuevos)                 # nota prevista para cada aprendiente
#>        1        2        3
#> 67.89744 59.38983 77.06742
rpart.predict(arbol_nota, nuevos, rules = TRUE)
#>   nota_dele
#> 1  67.89744 because horas >= 235 & L1 is             árabe or chino
#> 2  59.38983 because horas <  235 & L1 is árabe or inglés or italiano
#> 3  77.06742 because horas >= 235 & L1 is          inglés or italiano
```

En pocas palabras

Con el árbol de regresión se puede predecir la nota de un aprendiente concreto. Se definen sus horas, su L1 y su estancia, y la ficha recorre el árbol hasta una hoja, que da la nota prevista.

Algunos ejemplos:

- Con 200 horas y L1 chino, la nota prevista es de unos 49 puntos.
- Con 400 horas y L1 italiano, de unos 77.

De nuevo, la **estancia** no cambia nada, porque este árbol no la utiliza.

Es importante recordar que la predicción es la media de un grupo. Personas con el mismo perfil pueden tener notas muy distintas.

5.23 ¿Árbol o regresión logística?

La **regresión logística** (en el material Plus) predice un sí/no con una fórmula: cada variable suma o resta a la probabilidad de acierto y R indica si su efecto es significativo.

- **Ventajas del árbol**: · Reglas fáciles de explicar a no especialistas · Detecta **umbrales** e **interacciones** solo · No exige normalidad ni linealidad · Selecciona variables
- **Limitaciones**: · **Inestable**: pocos datos distintos → árbol distinto · Tiende al **sobreajuste**: hay que podar y validar · No da valores p ni intervalos de confianza · Cortes solo paralelos a los ejes

Para recordar

Logística para cuantificar y contrastar efectos · **árbol** para explorar perfiles y comunicar reglas. Usarlos juntos es una buena práctica.

En pocas palabras

Los árboles no son la única forma de predecir un sí o un no. La alternativa clásica es la **regresión logística**, que se explica en el material Plus. En lugar de preguntas encadenadas, construye una fórmula en la que cada variable suma o resta a la probabilidad de acierto, y para cada variable da un valor p y un intervalo de confianza. Por ejemplo, con la misma pregunta del subjuntivo, la logística dice que la estancia multiplica por 2,3 las posibilidades de acierto y que ese efecto es significativo; el árbol, en cambio, ni siquiera usa la estancia.

Los árboles y las regresiones no compiten: responden a necesidades distintas.

Ventajas del árbol:

- Produce reglas fáciles de explicar a cualquier persona.
- Detecta solo umbrales e interacciones.
- No exige que los datos sean normales ni que las relaciones sean rectas.
- Selecciona las variables útiles.

Limitaciones:

- Es **inestable**: con unos pocos datos distintos, el árbol puede cambiar mucho.
- Tiende al sobreajuste, así que hay que podarlo y validarlo.
- No da valores p ni intervalos de confianza.
- Solo hace cortes del tipo «más/menos de X horas»: si la relación es gradual (cada hora suma un poco), la aproxima con escalones.

La recomendación es usar la **regresión logística** para medir y contrastar efectos, y el **árbol** para explorar perfiles y comunicar reglas.

5.24 Para saber más: bosques aleatorios

Muchos árboles votando (install.packages('ranger'))

```
library(ranger)
bosque <- ranger(factor(subjuntivo) ~ horas + estancia + L1 + nivel,
                 data = entreno, num.trees = 500, importance = "impurity")
mean(predict(bosque, prueba)$predictions == prueba$subjuntivo)
bosque$variable.importance
```

Un bosque promedia cientos de árboles hechos con remuestreos: es más estable y suele predecir mejor, pero **no hay un árbol que interpretar**: se pierden las reglas y los umbrales, y solo queda la importancia de cada variable.

En pocas palabras

Una forma de resolver la inestabilidad de los árboles es construir muchos. Un **bosque aleatorio** (*random forest*) crea cientos de árboles, cada uno con una muestra distinta de los datos y de las variables, y los hace **votar**. La predicción final es la opción más votada.

El paquete `ranger` lo hace en pocas líneas:

- `num.trees = 500`: el número de árboles.
- `importance = "impurity"`: calcula la importancia de cada variable.

Los bosques suelen predecir mejor y son más estables que un único árbol. El precio es que se pierde la interpretabilidad: ya no hay un árbol que se pueda leer en voz alta, sino quinientos.

Conviene decirlo claramente: **con un bosque perdemos los factores de clasificación interpretables**. Ya no hay reglas del tipo «si tiene menos de 243,5 horas, falla el 81 %», ni un umbral ni una rama que mirar. Lo único que conservamos es la **importancia de las variables** (`bosque$variable.importance`): nos dice **qué** variables pesan más en las predicciones (previsiblemente, las horas y el nivel, como en el árbol), pero no **cómo** influyen: ni a partir de qué valor, ni en qué dirección, ni para qué perfil de aprendiente. Si el objetivo es explicar y comunicar perfiles, el árbol único sigue siendo la herramienta; si el objetivo es predecir con la mayor precisión posible, el bosque.

5.25 Cómo informar de un árbol

> Se ajustó un árbol de clasificación (paquete *rpart*, índice de Gini) para predecir el uso correcto del subjuntivo a partir de las horas de instrucción, la estancia, la L1 y el nivel. La primera división, y la más informativa, fue la de las **horas de instrucción**: con 243,5 horas o más, el 67,7 % de los aprendientes usó correctamente el subjuntivo, frente al 19,0 % con menos horas. El árbol se podó mediante validación cruzada (10 particiones). En una muestra de prueba independiente (30 %), el árbol podado acertó el 70,8 % de los casos, frente al 63,9 % del árbol sin podar.

Siempre: variables candidatas, criterio de división, cómo se podó, **exactitud con datos de prueba** y comparación con la clase mayoritaria.

En pocas palabras

Este párrafo es un modelo de cómo redactar el resultado de un árbol en un trabajo académico. Incluye todos los elementos necesarios:

- **Qué se hizo**: un árbol de clasificación con *rpart* y el índice de Gini.
- **Qué se quería predecir y con qué variables**.
- **El resultado principal**: la primera división y los porcentajes de cada grupo.
- **Cómo se controló el sobreajuste**: poda mediante validación cruzada.
- **Cómo se evaluó**: la exactitud con una muestra de prueba independiente, comparando el árbol podado y el grande.

Conviene añadir siempre la comparación con la **clase mayoritaria**: un árbol que acierta un 70 % no impresiona si predecir siempre lo mismo ya acierta un 65 %.

5.26 Práctica: árboles (20 min)

1. Ajusta un árbol para predecir `estancia` a partir de `horas`, `nota_dele`, `L1` y `nivel`. ¿Tiene sentido?
2. Dibuja el árbol y lee en voz alta la regla de la hoja más pura.
3. Haz `printcp()` y `plotcp()`. ¿Cuántas hojas recomienda la regla de 1 EE?

4. Pódalo y compara el acierto con datos de entrenamiento y de prueba.
5. Ajusta un árbol de regresión para `err_ser_estar`. ¿Qué variable aparece primero?
6. Redacta dos frases de resultados como en la diapositiva anterior.

Pista: Copia el Paso 9 y cambia `subjuntivo` por `estancia` en la fórmula y en la función `acierto()`.

5.27 Práctica de árboles: soluciones

Soluciones

```
set.seed(2026)
a_est <- rpart(estancia ~ horas + nota_dele + L1 + nivel, data = ele,
               method = "class", control = rpart.control(cp = 0, minsplit = 6))
rpart.plot(a_est, type = 4, extra = 104)
printcp(a_est); plotcp(a_est)
a_est_p <- prune(a_est, cp = a_est$cptable[which.min(a_est$cptable[, "xerror"]), "CP"])
rpart.plot(a_est_p, type = 4, extra = 104)
a_err <- rpart(err_ser_estar ~ horas + L1 + nivel + estancia, data = ele, cp = 0.02)
rpart.plot(a_err, type = 4)
```

En pocas palabras

Una posible solución para la práctica de árboles:

1. Árbol de clasificación para predecir la **estancia** a partir de las horas, la nota, la L1 y el nivel. Se construye sin freno (`cp = 0`), como en el paso 6.
2. `rpart.plot()` lo dibuja.
3. `printcp()` y `plotcp()` muestran la tabla y el gráfico de complejidad para decidir la poda.
4. `prune()` recorta el árbol con el cp de menor error de validación cruzada.
5. Árbol de regresión para los **errores de *ser/estar***, que es un recuento y, por tanto, una variable numérica.

Cabe preguntarse si tiene sentido predecir la estancia a partir de la nota: la relación causal va más bien en la otra dirección.

6 Plus: Estadística de corpus y modelos

Plus · material complementario

> ℹ La pregunta de esta sesión
>
> ¿Qué palabras delatan a un aprendiente? ¿Cuánto sube la nota con cada hora de clase? ¿Qué predice que use bien el subjuntivo?

Contenidos: Frecuencias normalizadas · Log-likelihood · Log-ratio · Simple Maths · Mapa de palabras clave · Regresión lineal y logística.

6.1 Aprendientes frente a nativos

Paso 1 · Tamaños de los corpus y frecuencias normalizadas

```
N_aprend <- 150000; N_nativo <- 200000   # palabras de cada corpus
frec <- frec |>
  mutate(pmw_aprend = aprend / N_aprend * 1e6,
         pmw_nativo = nativo / N_nativo * 1e6)
frec |> select(palabra, aprend, nativo, pmw_aprend, pmw_nativo) |> head(5)
#> # A tibble: 5 × 5
#>   palabra aprend nativo pmw_aprend pmw_nativo
#>   <chr>    <dbl>  <dbl>      <dbl>      <dbl>
#> 1 muy        620    410      4133.       2050
#> 2 cosa       410    250      2733.       1250
#> 3 bueno      380    300      2533.       1500
#> 4 pues        90    380       600        1900
#> 5 o sea       25    190       167.        950
```

> ! Idea clave
>
> Corpus de distinto tamaño → comparar frecuencias **por millón** de palabras (pmw).

> En pocas palabras
>
> En la lingüística de corpus comparamos con frecuencia dos corpus: aquí, textos de aprendientes y textos de nativos. El problema es que los corpus rara vez tienen el mismo tamaño: el de aprendientes tiene 150 000 palabras y el de nativos, 200 000.
>
> Comparar frecuencias brutas sería injusto: una palabra aparecería más en el corpus grande solo porque es más grande. La solución es la **frecuencia normalizada**, normalmente **por millón de palabras** (pmw): cuántas veces aparecería la palabra si el corpus tuviera un millón de palabras.
>
> Se calcula dividiendo la frecuencia entre el tamaño del corpus y multiplicando por un millón. A partir de aquí las cifras son comparables.

6.2 Log-likelihood (G^2): el caso de *pues*

	Aprendientes	Nativos	Total
Observadas	90	380	470

	Aprendientes	Nativos	Total
Tamaño del corpus	150 000	200 000	350 000
Esperadas	470 × 150/350 = **201**	470 × 200/350 = **269**	470

$$G^2 = 2\sum O \ln \frac{O}{E} = 2 \times [90\ln(90/201) + 380\ln(380/269)] \approx 118$$

Para recordar

$G^2 > 3{,}84 \rightarrow p < 0{,}05$ · $G^2 > 15{,}13 \rightarrow p < 0{,}0001$. Con frecuencias bajas es más fiable que χ^2.

En pocas palabras

La frecuencia normalizada nos dice si una palabra aparece más en un corpus que en otro, pero no si esa diferencia es fiable. Para eso se usa el **log-likelihood** (G^2), la prueba estándar en lingüística de corpus.

El razonamiento es como el del chi cuadrado. *Pues* aparece 470 veces en total. Si los dos corpus la usaran igual, se repartiría según su tamaño: unas 201 veces en el de aprendientes y 269 en el de nativos. Pero los aprendientes solo la usan 90 veces.

G^2 mide la distancia entre lo observado y lo esperado: aquí, 118. Cualquier valor mayor que 3,84 es significativo ($p < 0{,}05$), y mayor que 15,13, muy significativo. Los aprendientes **infrautilizan** *pues*.

6.3 LL y log-ratio en R

Paso 2 · Dos funciones y una tabla

```
ll <- function(a, b, N1, N2) {
  E1 <- N1 * (a + b) / (N1 + N2); E2 <- N2 * (a + b) / (N1 + N2)
  2 * (a * log(a / E1) + b * log(b / E2))
}
frec <- frec |> mutate(LL = ll(aprend, nativo, N_aprend, N_nativo),
                       LR = log2(pmw_aprend / pmw_nativo))
frec |> arrange(desc(abs(LR))) |>
  transmute(palabra, LL = round(LL, 1), LR = round(LR, 2),
            sig = ifelse(LL > 15.13, "***", ifelse(LL > 3.84, "*", "")))
#> # A tibble: 10 × 4
#>    palabra        LL    LR sig
#>    <chr>       <dbl> <dbl> <chr>
#>  1 o sea       100.  -2.51 "***"
#>  2 pues        119.  -1.66 "***"
#>  3 es que       50.7 -1.39 "***"
#>  4 cosa         98.8  1.13 "***"
#>  5 muy         125.   1.01 "***"
#>  6 bueno        46.5  0.76 "***"
#>  7 sin embargo   6.3  0.64 "*"
#>  8 hacer        45.2  0.62 "***"
#>  9 tener        22.6  0.45 "***"
#> 10 además        3.4  0.32 ""
```

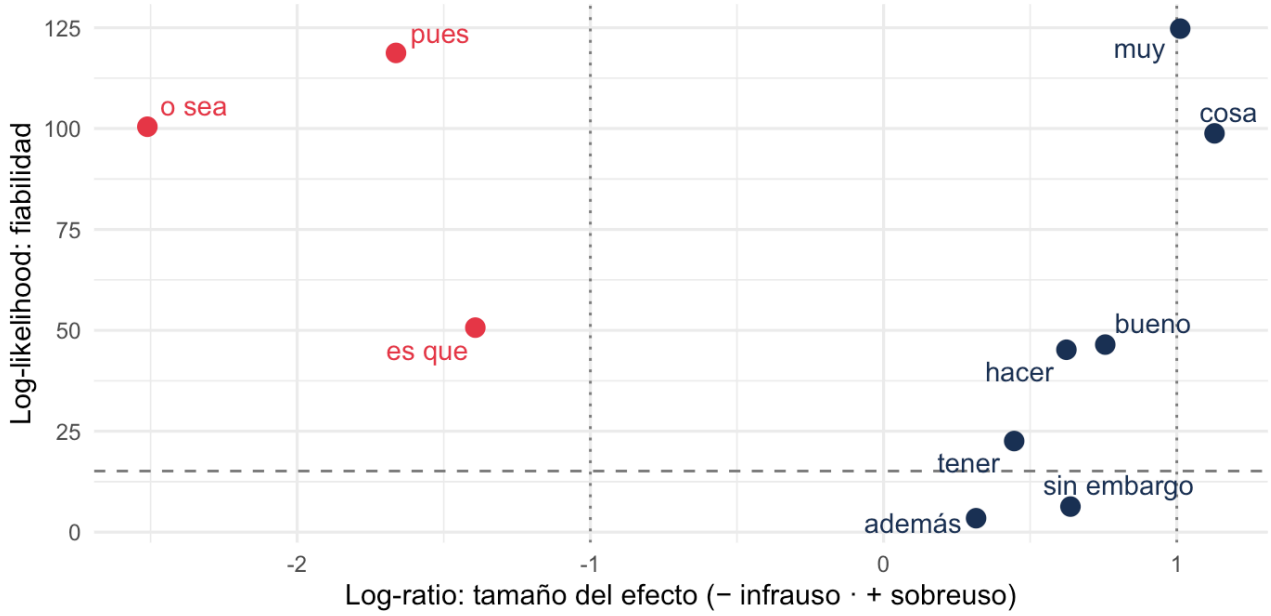
En pocas palabras

Este código calcula dos medidas para cada palabra:

- **LL (log-likelihood)**: si la diferencia entre los corpus es **fiable**. La función `ll()` aplica la fórmula del apartado anterior.
- **LR (log-ratio)**: lo **grande** que es la diferencia. Es el logaritmo en base 2 del cociente de frecuencias normalizadas. LR = 1 significa que la palabra es el doble de frecuente en los aprendientes; LR = −1, la mitad; LR = 0, igual de frecuente.

La tabla final ordena las palabras por la magnitud de la diferencia y añade asteriscos según su significación. Se necesitan las dos medidas: LL indica si podemos fiarnos y LR, cuánto difieren.

6.4 El mapa de palabras clave



En pocas palabras

Este gráfico sitúa cada palabra según las dos medidas a la vez:

- **Eje vertical** (LL): cuanto más arriba, más fiable es la diferencia.
- **Eje horizontal** (log-ratio): a la derecha, palabras que los aprendientes **sobreutilizan**; a la izquierda, palabras que **infrautilizan**.

Las **palabras clave** de verdad están arriba y hacia los lados: diferencias a la vez fiables y grandes. Una palabra muy frecuente puede tener un LL alto con una diferencia pequeña, y una palabra rara puede tener un log-ratio llamativo sin ser fiable.

Los controles permiten cambiar los umbrales y ver cómo cambia la lista. *Pues* y *o sea* son ejemplos claros de marcadores que los aprendientes usan poco.

6.5 Simple Maths: la fórmula de Sketch Engine

$$SM = \frac{pmw_{aprend} + n}{pmw_{nativo} + n} \qquad (n = 1 \ \text{ por defecto})$$

Paso 3 · Simple Maths con dos valores de n

```r
sm <- function(pmw1, pmw2, n = 1) (pmw1 + n) / (pmw2 + n)
frec |> mutate(SM_1 = sm(pmw_aprend, pmw_nativo),
               SM_1000 = sm(pmw_aprend, pmw_nativo, n = 1000)) |>
  arrange(desc(SM_1)) |> select(palabra, SM_1, SM_1000) |> head(4)
#> # A tibble: 4 × 3
#>   palabra      SM_1 SM_1000
#>   <chr>       <dbl>   <dbl>
#> 1 cosa         2.19    1.66
#> 2 muy          2.02    1.68
#> 3 bueno        1.69    1.41
#> 4 sin embargo  1.55    1.13
```

En pocas palabras

Simple Maths es la medida de palabras clave que usa Sketch Engine, propuesta por Adam Kilgarriff. Es un cociente de frecuencias normalizadas al que se suma un número *n* arriba y abajo.

Atención: este *n* no es un tamaño de muestra. No depende de cuántas palabras tenga el corpus ni es un número aleatorio: es una **constante de suavizado** que elige quien investiga (Sketch Engine usa n = 1 por defecto). Es el mismo *n* que se mueve con el deslizador del apartado siguiente.

Ese *n* resuelve un problema: sin él, una palabra que aparece una vez en un corpus y ninguna en el otro daría un cociente infinito (algo dividido entre 0). Al sumar *n*, las palabras raras dejan de dominar el ranking.

Un ejemplo con *pues*: aparece 600 veces por millón en el corpus de aprendientes (90 / 150 000 × 1 000 000) y 1900 veces por millón en el de nativos (380 / 200 000 × 1 000 000).

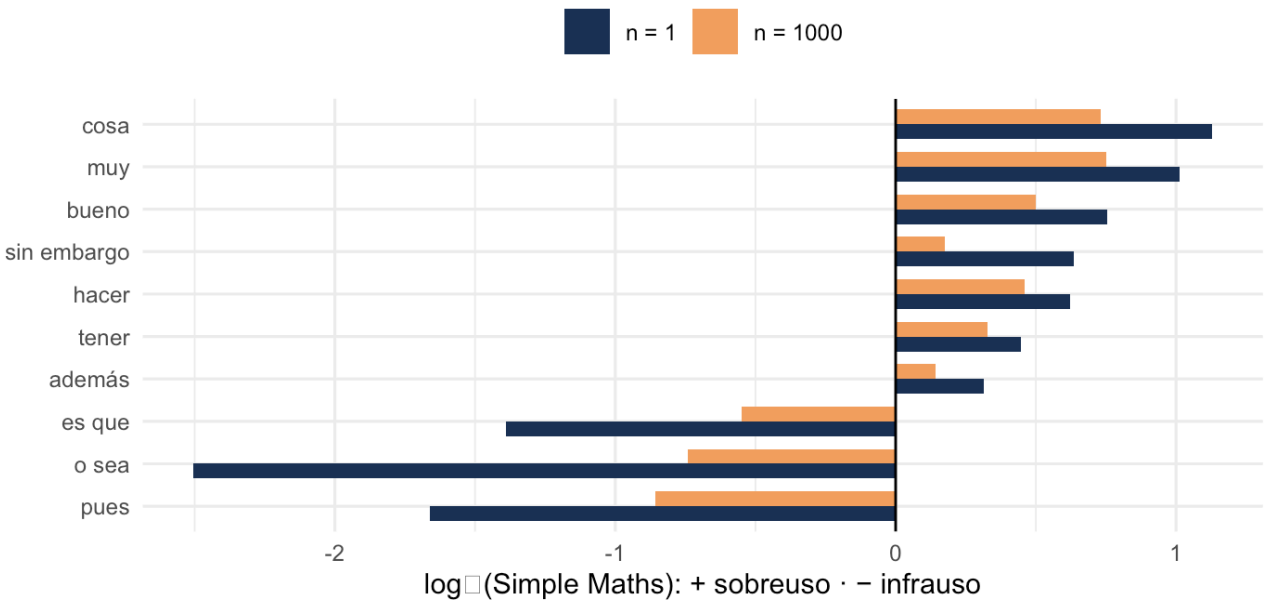
- Con n = 1: SM = (600 + 1) / (1900 + 1) = **0,32**. Los aprendientes usan *pues* aproximadamente un tercio de lo que lo usan los nativos.
- Con n = 1000: SM = (600 + 1000) / (1900 + 1000) = **0,55**. El valor se acerca a 1, porque el *n* grande «diluye» la diferencia.

Cómo leer el valor: **SM > 1**, la palabra es más frecuente en los aprendientes (sobreuso); **SM < 1**, menos frecuente (infrauso); **SM = 1**, igual de frecuente.

El código calcula la medida con dos valores:

- Con **n = 1**, destacan las palabras con diferencias relativas grandes, aunque sean poco frecuentes.
- Con **n = 1000**, todos los valores se acercan a 1 y destacan las palabras frecuentes.

6.6 ¿Cómo cambia el ranking con *n*?



En pocas palabras

Esta visualización muestra cómo el ranking de palabras clave depende del valor de *n* en Simple Maths, la constante de suavizado del apartado anterior (el mismo *n*).

Qué son los valores. Cada barra es una palabra y el número de su extremo es su Simple Maths: (pmw en aprendientes + n) / (pmw en nativos + n). La línea vertical marca el **1** (igual de frecuente en los dos corpus). Las barras azules, a la derecha, son palabras que los aprendientes **sobreusan**; las rojas, a la izquierda, las que **infrautilizan**. El eje está en escala logarítmica para que «el doble» (2) y «la mitad» (0,5) queden a la misma distancia del 1.

Por ejemplo, con n = 1, *muy* tiene SM = (4133 + 1) / (2050 + 1) ≈ **2,0**: los aprendientes la usan el doble que los nativos. *O sea* tiene SM = (167 + 1) / (950 + 1) ≈ **0,18**: la usan unas cinco veces menos. El panel muestra el cálculo de la palabra más sobreusada para el *n* elegido.

Al mover el deslizador:

- Con **n bajo**, destacan las palabras **raras** con diferencias relativas grandes.
- Con **n alto**, todos los cocientes se acercan a 1 y destacan las palabras **frecuentes**.

Por ejemplo, *muy* adelanta a *cosa* al subir *n*, y *sin embargo* pierde posiciones.

No hay un *n* correcto: depende de si nos interesa el vocabulario poco frecuente y característico o las palabras comunes con diferencias de uso. Lo importante es saber que el ranking cambia y declarar qué valor se ha usado.

6.7 Con textos reales: `quanteda`

Para tus propios corpus

```
library(quanteda); library(quanteda.textstats)
corp <- corpus(readtext::readtext("corpus/*.txt", docvarsfrom = "filenames"))
dfm_g <- tokens(corp, remove_punct = TRUE) |> dfm() |>
  dfm_group(groups = docvar1)                  # aprendientes / nativos
kw <- textstat_keyness(dfm_g, target = "aprendientes", measure = "lr")
```

```r
head(kw, 20)                                  # G² y valor p
quanteda.textplots::textplot_keyness(kw)
```

> ! Idea clave
>
> Cómo informar: *pues* aparece significativamente menos en el corpus de aprendientes (LL = 118,7, p < 0,0001; log-ratio = −1,66): lo usan unas 3,2 veces menos.

> En pocas palabras
>
> En nuestro ejemplo, las frecuencias ya estaban contadas. Con textos propios, el paquete `quanteda` hace todo el proceso:
>
> 1. `readtext()` lee una carpeta de archivos de texto.
> 2. `corpus()` los convierte en un corpus.
> 3. `tokens()` divide los textos en palabras, y `dfm()` cuenta cuántas veces aparece cada palabra en cada texto.
> 4. `dfm_group()` agrupa los textos en dos conjuntos: aprendientes y nativos.
> 5. `textstat_keyness()` calcula las palabras clave y `textplot_keyness()` las dibuja.
>
> El recuadro inferior muestra cómo redactar el resultado: *pues* aparece unas tres veces menos en el corpus de aprendientes (LL = 118, p < 0,0001).

6.8 Segunda parte del Plus: de comparar a predecir

- **Hasta ahora: comparar**: ¿Sacan más nota quienes han hecho una estancia? (prueba t) ¿Se relacionan las horas y la nota? (correlación: r = 0,58) La respuesta es **sí o no** y una medida de fuerza.
- **Ahora: predecir**: Lin tiene **300 horas** de clase y no ha hecho estancia. ¿Qué nota cabe esperar? Un **modelo de regresión** da una fórmula: nota prevista = punto de partida + lo que suma cada hora.

> Para recordar
>
> Sin modelo, la mejor apuesta para Lin es la media de todos: **68 puntos**, con un error típico de 13. Con un modelo que tiene en cuenta sus horas, la predicción se ajusta a su caso y el error baja.

En pocas palabras

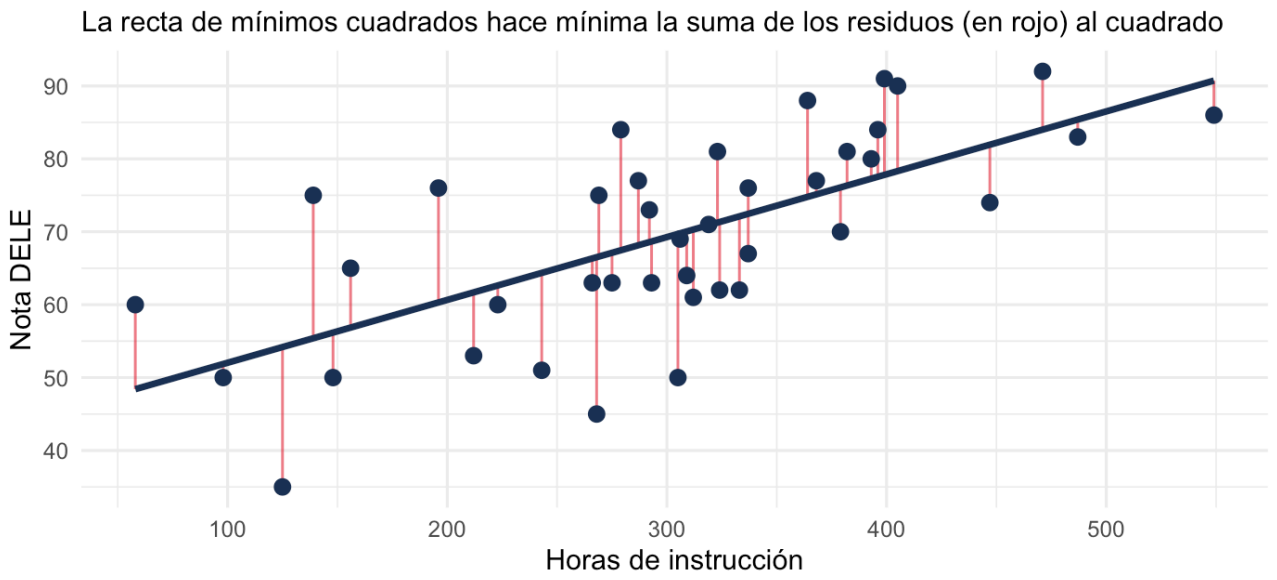
Este apartado marca el cambio de tema dentro del Plus: dejamos los corpus y volvemos a los datos de los 240 aprendientes.

Hasta ahora hemos **comparado**: ¿difiere la nota entre grupos?, ¿se relacionan dos variables? La correlación nos dijo que horas y nota van juntas (r = 0,58), pero no **cuánto** sube la nota con cada hora ni qué nota esperar de una persona concreta.

Un **modelo de regresión** da un paso más: convierte la relación en una **fórmula para predecir**. Pensemos en Lin, una aprendiente inventada con 300 horas de clase y sin estancia. Si no supiéramos nada de ella, lo más sensato sería apostar por la media de todo el grupo, 68 puntos, y nos equivocaríamos, de media, unos 13 puntos. Pero sabemos que quienes tienen más horas tienden a sacar más nota, así que podemos afinar.

La regresión lineal busca la **recta** que mejor resume esa relación y la usa para predecir. En los apartados siguientes veremos cómo se encuentra esa recta y qué nota predice para Lin. La regresión **logística** hará lo mismo cuando lo que queremos predecir es un sí o un no (acertar o fallar el subjuntivo).

6.9 La idea de la regresión



En pocas palabras

La **regresión lineal** busca la recta que mejor resume la relación entre dos variables numéricas, para poder **predecir** una a partir de la otra: la nota a partir de las horas. El gráfico usa 40 aprendientes elegidos al azar, para que los puntos se vean bien.

Una recta queda definida por dos números:

- El **intercepto** (b_0): dónde corta el eje vertical, es decir, la nota prevista con 0 horas.
- La **pendiente** (b_1): cuánto sube la nota por cada hora más de clase.

La fórmula es: nota prevista = $b_0 + b_1 \times$ horas. Por ejemplo, con $b_0 = 45$ y $b_1 = 0{,}08$, alguien con 300 horas tendría una nota prevista de 45 + 0,08 × 300 = **69 puntos**. El panel calcula la predicción para 300 horas con la recta que hayas elegido.

¿Cuál es la «mejor» recta? La que hace más pequeños los errores de predicción. Cada línea roja vertical es el error de un aprendiente: la distancia entre su nota real y la que predice la recta (su **residuo**). Si una aprendiente con 300 horas sacó 75 y la recta predice 69, su residuo es +6. En el gráfico, esos errores se dibujan también como **cuadrados rojos**: el error elevado al cuadrado. La mejor recta es la que minimiza la suma de sus áreas. Se llama método de **mínimos cuadrados**, y es exactamente lo que hace la función `lm()` (de *linear model*, modelo lineal).

6.10 Regresión lineal: predecir la nota

Paso 4 · lm() con dos predictores

```
modelo_lm <- lm(nota_dele ~ horas + estancia, data = ele)
summary(modelo_lm)$coefficients |> round(3)
#>             Estimate Std. Error t value Pr(>|t|)
#> (Intercept)   43.957      2.148  20.462    0.000
#> horas          0.078      0.007  11.185    0.000
#> estanciasí     4.495      1.346   3.339    0.001
summary(modelo_lm)$r.squared
#> [1] 0.3708927
```

Paso 5 · Predecir la nota de tres aprendientes

```
lin <- tibble(horas = c(200, 300, 300),
              estancia = c("no", "no", "sí"))
predict(modelo_lm, lin) |> round(1)
#>    1    2    3
#> 59.6 67.5 72.0
```

Para recordar

Cada hora de instrucción suma 0,078 puntos (unos 7,8 cada 100 horas). Lin (300 h, sin estancia) tiene una nota prevista de 67,5; con estancia, 72,0. El modelo explica el 37 % de la variación.

En pocas palabras

`lm()` ajusta un modelo de regresión con dos **predictores**: las horas y la estancia. La fórmula se lee «la nota en función de las horas y de la estancia».

La tabla de coeficientes (columna *Estimate*) se interpreta así:

- **(Intercept) = 43,96**: la nota prevista para alguien con 0 horas y sin estancia. Es un punto de partida teórico: nadie en los datos tiene 0 horas.
- **horas = 0,078**: cada hora de instrucción suma unos 0,08 puntos. Como una hora sola apenas se nota, es más útil pensarlo en bloques: **100 horas más suman unos 7,8 puntos**.
- **estanciasí = 4,49**: la estancia suma unos 4,5 puntos **a igualdad de horas**. Es decir, comparando a dos personas con las mismas horas, la que ha hecho una estancia saca de media 4,5 puntos más.

Aterrizado en casos concretos (paso 5), con la fórmula nota = 43,96 + 0,078 × horas + 4,49 × (estancia sí):

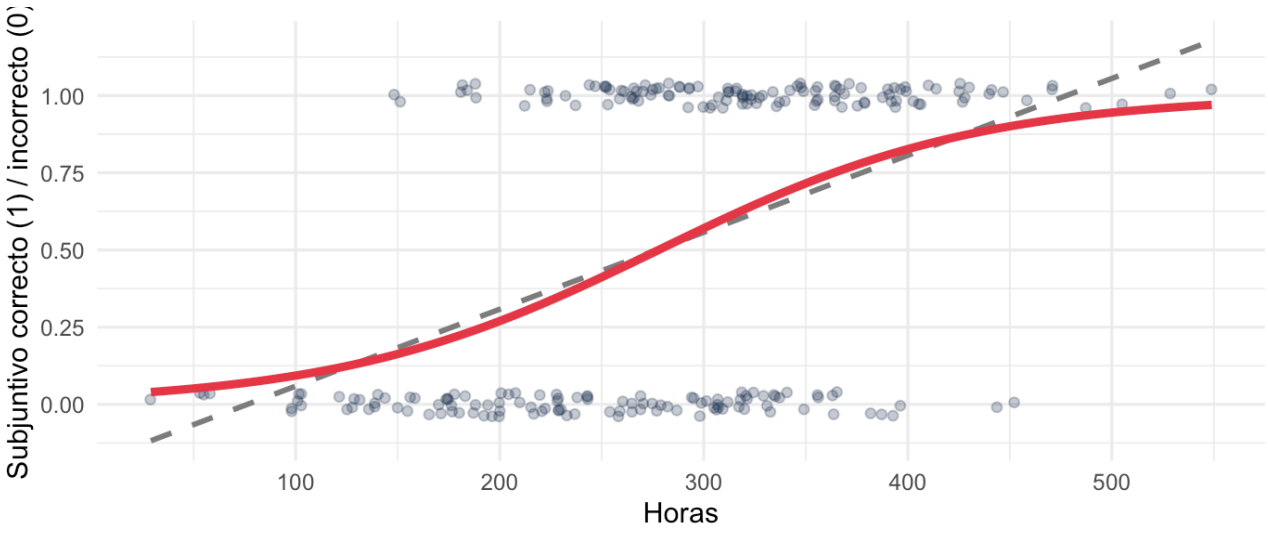
- Un aprendiente con **200 horas y sin estancia**: 43,96 + 0,078 × 200 = **59,6 puntos**.
- **Lin**, con **300 horas y sin estancia**: 43,96 + 0,078 × 300 = **67,5 puntos**. Las 100 horas de diferencia con el anterior suponen esos 7,8 puntos más.
- Alguien con **300 horas y con estancia**: 67,5 + 4,5 = **72,0 puntos**.

Las columnas *Std. Error*, *t value* y *Pr(>|t|)* contrastan si cada coeficiente es distinto de 0: aquí, los dos predictores son claramente significativos.

¿Un R^2 del 37 % es mucho o poco? El R^2 indica qué parte de la variación de la nota explica el modelo. Un 37 % significa que, conociendo las horas y la estancia, el error típico al predecir la nota baja de unos 12,9 puntos (sin modelo) a unos 10,2. Para un fenómeno humano tan complejo como el aprendizaje de una lengua, que depende de muchísimos factores, es una proporción **considerable**: equivale a una correlación de 0,61, «grande» según las referencias de Cohen y de los estudios de L2 (Plonsky y Oswald, 2014). Al mismo tiempo, recuerda que el 63 % de las diferencias entre aprendientes se debe a factores que el modelo no incluye. Las horas solas explicaban un 34 % (r^2 de la correlación); la estancia añade 3 puntos más.

La ventaja de la regresión frente a la correlación es que separa el efecto de cada variable **controlando las demás** y permite predecir casos concretos.

6.11 ¿Por qué no una recta para un sí/no?



La recta (gris) predice probabilidades **< 0 o > 1**; la curva **en S** (roja) se queda entre 0 y 1.

En pocas palabras

¿Qué pasa si lo que queremos predecir no es un número sino un **sí o no**, como acertar o fallar el subjuntivo? Podemos codificarlo como 1 (acierto) y 0 (fallo), pero una recta no funciona bien.

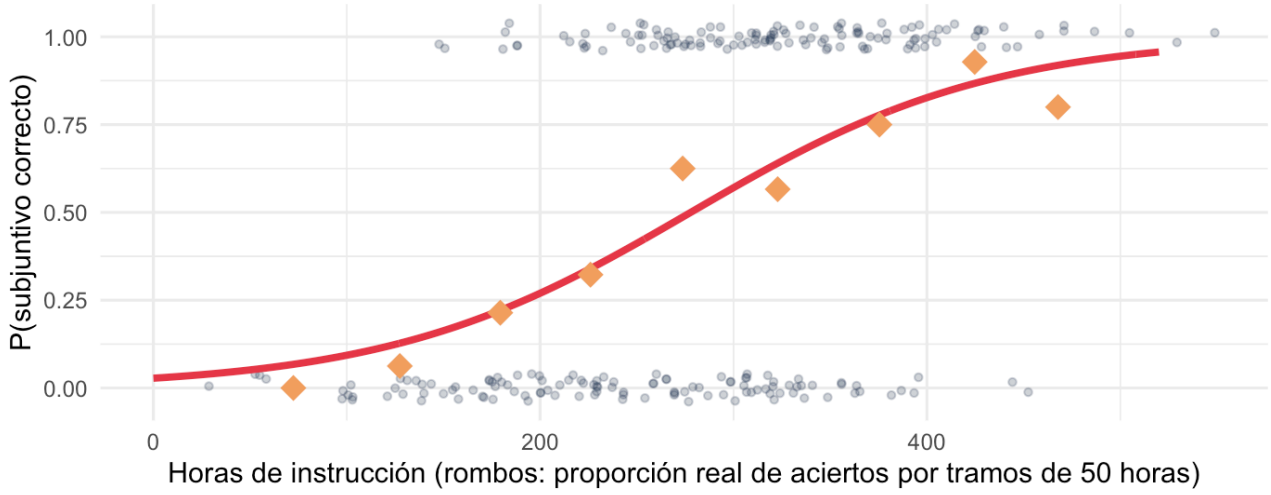
En el gráfico, los puntos están en dos franjas: arriba los aciertos, abajo los fallos. La **recta gris** acaba prediciendo valores menores que 0 o mayores que 1, que no tienen sentido como probabilidades.

La **curva en S roja** es la de la **regresión logística**: siempre se mantiene entre 0 y 1. Con pocas horas, la probabilidad de acierto es baja; aumenta en la zona media y se estabiliza cerca de 1 con muchas horas.

Qué significa aquí «probabilidad»: la proporción de aprendientes que aciertan entre quienes tienen esas horas. Por ejemplo, de los 31 aprendientes que tienen entre 200 y 250 horas, aciertan 10 (un 32 %); de los 36 que tienen entre 350 y 400, aciertan 27 (un 75 %). La curva roja resume esas proporciones para cualquier cantidad de horas. El apartado siguiente explica cómo se construye.

6.12 Probabilidad, odds y log-odds

La curva en S de la diapositiva anterior, con controles. Con 14 aciertos y 6 fallos: **probabilidad** = 14/20 = 0,70 · ***odds*** = 14/6 ≈ 2,3 (aciertos por cada fallo) · **log-odds** = ln(2,3) ≈ 0,85.



horas	log-odds	odds	probabilidad
100	−2,28	0,10	0,09
250	−0,36	0,70	0,41
400	1,56	4,77	0,83

En pocas palabras

Qué muestra este apartado. Es la curva roja del apartado anterior, ahora con controles. No es una alternativa a la regresión lineal de la nota, sino su equivalente para una respuesta de **sí o no**: si la variable que queremos predecir es un número (la nota), usamos la recta de `lm()`; si es acertar o fallar (el subjuntivo), usamos esta curva en S de `glm()`. Para simplificar, el modelo tiene un solo predictor: las horas.

Qué hay en el gráfico. Cada punto es un aprendiente: arriba (1), los que usan bien el subjuntivo; abajo (0), los que fallan. Los rombos naranjas son la proporción real de aciertos en cada tramo de 50 horas. La curva azul es la **probabilidad** de acierto que predice el modelo para cada cantidad de horas, y debería pasar cerca de los rombos.

Las tres escalas, con un ejemplo. Imaginemos un grupo de 20 aprendientes con unas 350 horas, de los que aciertan 14 y fallan 6:

- **Probabilidad**: aciertos / total = 14 / 20 = **0,70** (un 70 %). Va de 0 a 1.
- ***Odds*** (en español, a veces «momios» o «razón de posibilidades»): aciertos / fallos = 14 / 6 ≈ **2,3**. Se lee «2,3 aciertos por cada fallo». Si acertara la mitad, las *odds* serían 1 (uno a uno); si acertaran menos de la mitad, serían menores que 1. Son las «apuestas» de las casas de apuestas: 3 a 1.
- **Log-odds**: el logaritmo natural de las *odds*: ln(2,3) ≈ **0,85**. Vale 0 cuando la probabilidad es del 50 %, es positivo cuando hay más aciertos que fallos y negativo cuando hay más fallos. Su ventaja es que puede tomar cualquier valor, de −∞ a +∞, así que se puede modelizar con una recta sin salirse de los límites.

Cómo encadena el modelo las tres escalas. La regresión logística traza una **recta en la escala de los log-odds**: log-odds = b_0 + b_1 × horas. Después la traduce a *odds* (con la función exponencial, e^x) y a probabilidad (odds / (1 + odds)), y así obtiene la curva en S. Con los valores que estima `glm()` (b_0 = −3,55; b_1 = 0,0128):

- **250 horas**: log-odds = −3,55 + 0,0128 × 250 = −0,36 ⟶ odds = e^−0,36 = 0,70 ⟶ probabilidad = 0,70 / 1,70 = **41 %**.
- **350 horas**: log-odds = +0,92 ⟶ odds = 2,5 ⟶ probabilidad = **72 %**.

El panel de la derecha hace esta misma cadena para las horas que elijas con el tercer deslizador. Los otros dos deslizadores mueven la recta de los log-odds: **b_0** desplaza la curva a izquierda o derecha y **b_1** la hace más o menos empinada (con b_1 = 0, las horas no influirían y la curva sería plana). El botón «Ajuste de glm()» coloca los valores que mejor se ajustan a los datos.

6.13 Regresión logística en R

Paso 6 · glm() con family = binomial

```
ele <- ele |> mutate(subj_ok = as.integer(subjuntivo == "correcto"))
modelo_glm <- glm(subj_ok ~ horas + estancia + L1, data = ele, family = binomial)
summary(modelo_glm)$coefficients |> round(3)
#>             Estimate Std. Error z value Pr(>|z|)
#> (Intercept)   -4.316      0.673  -6.416    0.000
#> horas          0.013      0.002   6.485    0.000
#> estanciasí     0.841      0.309   2.725    0.006
#> L1chino        0.298      0.444   0.670    0.503
#> L1inglés       0.389      0.397   0.981    0.327
#> L1italiano     0.797      0.435   1.831    0.067
exp(coef(modelo_glm)) |> round(2)      # odds ratios
#> (Intercept)       horas  estanciasí     L1chino    L1inglés  L1italiano
#>        0.01        1.01        2.32        1.35        1.48        2.22
```

Paso 7 · ¿Para qué sirven? Dos aprendientes ficticios

```r
fichas <- tibble(nombre = c("Emma", "Wang"), horas = c(300, 300),
                 estancia = c("sí", "no"), L1 = c("inglés", "chino"))
predict(modelo_glm, fichas, type = "response") |> round(2)
#>    1    2
#> 0.69 0.47
```

Para recordar

OR > 1 → **aumenta** la probabilidad de acierto. La estancia multiplica las *odds* de acierto por 2,3. Con 300 horas, Emma (inglés, con estancia) acierta con una probabilidad del 69 %; Wang (chino, sin estancia), del 47 %.

En pocas palabras

`glm()` con `family = binomial` ajusta una regresión logística. Primero creamos `subj_ok`, que vale 1 si el subjuntivo es correcto y 0 si no.

Los coeficientes están en la escala de los log-odds, difícil de interpretar. Por eso los transformamos con `exp()` en ***odds ratios*** (OR):

- **OR > 1**: la variable **aumenta** la probabilidad de acierto.
- **OR < 1**: la **disminuye**.
- **OR = 1**: no tiene efecto.

La estancia multiplica las *odds* de acierto por 2,3. Cada hora las multiplica por 1,013, lo que, acumulado en 100 horas, equivale a multiplicarlas por unas 3,7 ($1{,}013^{100}$). La L1 no tiene un efecto significativo (sus valores p están por encima de 0,05), aunque su OR sea mayor que 1.

¿Para qué sirven los coeficientes? Un ejemplo con dos aprendientes ficticios (paso 7). Emma y Wang tienen las mismas horas de clase, 300. Emma, de L1 inglés, ha vivido en Salamanca; Wang, de L1 chino, no ha hecho ninguna estancia. El modelo combina sus características:

- Wang: log-odds = –4,32 + 0,013 × 300 + 0,30 (chino) = –0,12 → probabilidad de acierto del **47 %**.
- Emma: log-odds = –4,32 + 0,013 × 300 + 0,84 (estancia) + 0,39 (inglés) = +0,81 → probabilidad del **69 %**.

Los coeficientes permiten, por tanto, **predecir** la probabilidad de acierto de cualquier perfil y **comparar** el peso de cada factor. La estancia es el factor que más separa a Emma de Wang: sola, multiplica sus *odds* por 2,3. En términos de un aula: si un grupo de aprendientes sin estancia acertara la mitad de las veces (odds = 1, uno a uno), un grupo igual pero con estancia tendría unas odds de 2,3, es decir, acertaría aproximadamente 7 de cada 10 veces (2,3 / 3,3 ≈ 0,70).

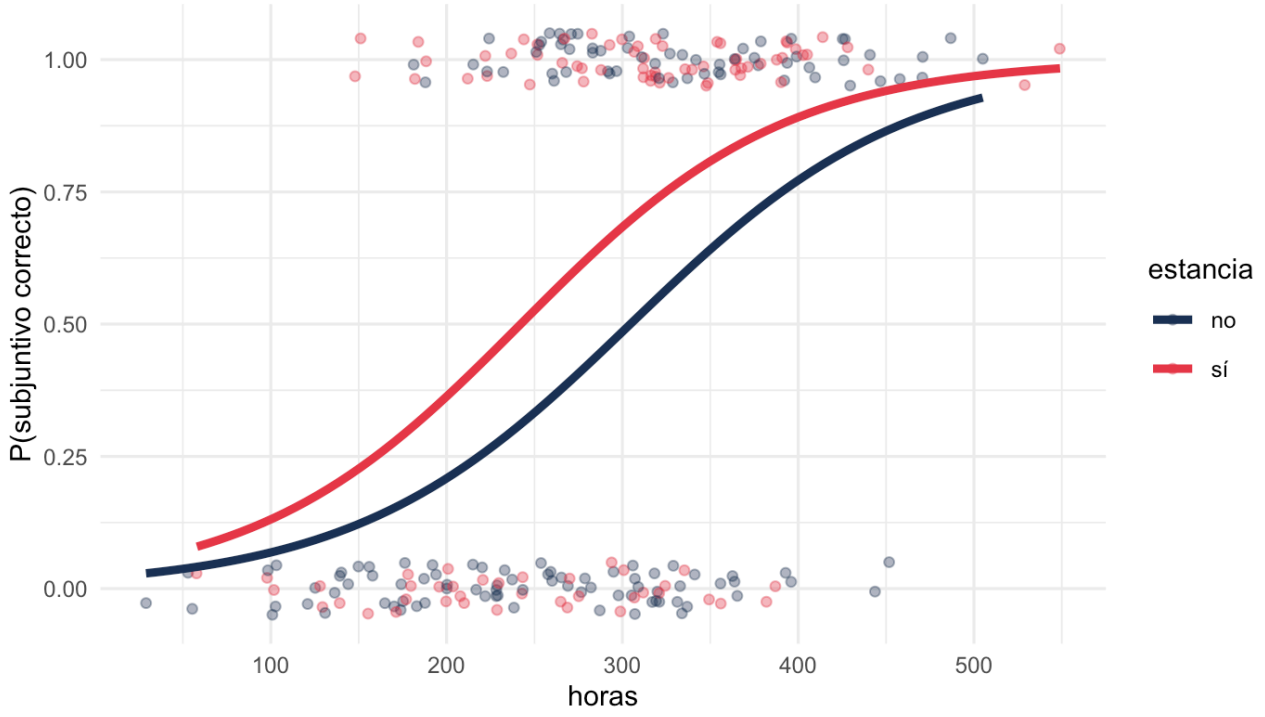
`predict(..., type = "response")` devuelve directamente la probabilidad; sin `type = "response"`, devolvería los log-odds.

6.14 Probabilidades predichas

Paso 8 · Curvas por grupo

```r
ggplot(ele, aes(horas, subj_ok,
                colour = estancia)) +
  geom_jitter(height = .05, alpha = .4) +
  geom_smooth(method = "glm",
    method.args = list(family = binomial),
    se = FALSE, linewidth = 1.4) +
```

```
  scale_colour_manual(
    values = c("#1D3557", "#E63946")) +
  labs(y = "P(subjuntivo correcto)")
```



En pocas palabras

La mejor forma de comunicar una regresión logística es dibujar las **probabilidades predichas**.

El gráfico muestra dos curvas en S, una para quienes han hecho estancia (rojo) y otra para quienes no (azul). Cada curva indica la probabilidad de usar bien el subjuntivo según las horas de instrucción. Los puntos, arriba y abajo, son los aprendientes reales.

Se observan dos cosas:

- Las dos curvas suben con las horas: más instrucción, más probabilidad de acierto.
- La curva de la estancia está siempre por encima: con las mismas horas, quienes han vivido en un país hispanohablante aciertan más.

Es la misma información que las *odds ratios*, pero mucho más fácil de entender.

6.15 Práctica Plus: mini-investigación (20 min)

1. Calcula LL y log-ratio de `frecuencias.csv`. ¿Qué palabras son clave?
2. Calcula Simple Maths con n = 1 y n = 100: ¿cambia el orden?
3. Regresión lineal: `nota_dele ~ horas + L1`. ¿Qué L1 difiere del árabe (la categoría de referencia)?
4. Regresión logística: `subj_ok ~ nivel + estancia`. Interpreta las OR.
5. Redacta **un párrafo** de resultados para uno de los análisis.

6.16 Práctica Plus: soluciones

Soluciones de la práctica Plus

```r
frec |> mutate(LL = ll(aprend, nativo, 150000, 200000),
               LR = log2((aprend / 150000) / (nativo / 200000))) |> arrange(desc(LL))
frec |> mutate(SM1 = sm(pmw_aprend, pmw_nativo, 1),
               SM100 = sm(pmw_aprend, pmw_nativo, 100)) |> arrange(desc(SM1))
summary(lm(nota_dele ~ horas + L1, data = ele))
exp(coef(glm(subj_ok ~ nivel + estancia, data = ele, family = binomial)))
```

En pocas palabras

Una posible solución para la mini-investigación final:

1. **Log-likelihood y log-ratio** para todas las palabras de `frecuencias.csv`, ordenadas por LL.
2. **Simple Maths** con n = 1 y n = 100, para comprobar si cambia el orden.
3. **Regresión lineal** de la nota con las horas y la L1. Cada coeficiente de lengua indica la diferencia respecto al **árabe**, la categoría de referencia (la primera por orden alfabético).
4. **Regresión logística** del subjuntivo con el nivel y la estancia, con `exp()` para obtener las *odds ratios*.

El último paso, redactar un párrafo de resultados, es el más importante: un análisis no está terminado hasta que se comunica con claridad.

7 De la pregunta al párrafo de resultados

Cierre

> **La pregunta de esta sesión**
>
> Elegir la prueba, ejecutarla, comprobar los supuestos e informar con rigor.

Contenidos: ¿Qué método elijo? · Cómo informar · Errores frecuentes.

7.1 ¿Qué método elijo?

Pregunta	Método	Función
¿Qué perfiles hay? ¿Reglas fáciles de explicar?	Árbol de decisión (sesión 5)	`rpart()`, `rpart.plot()`
¿Qué nota cabe esperar? (predecir un número con reglas)	Árbol de regresión (sesión 5)	`rpart()`
¿Palabra más frecuente en un corpus que en otro?	Log-likelihood + log-ratio (Plus)	`ll()`, `log2()`
¿Cuánto más frecuente es?	Log-ratio / Simple Maths (Plus)	`sm()`
¿Cuánto suma cada factor a una variable numérica?	Regresión lineal (Plus)	`lm()`
¿Cuánto influye cada factor en un sí/no?	Regresión logística (Plus)	`glm(family = binomial)`

Para las pruebas de contraste, vuelve al árbol interactivo de la sesión 3.

> En pocas palabras
>
> Esta tabla resume los métodos de la última sesión y del material Plus según la pregunta de investigación:
>
> - ¿Qué perfiles hay y cómo explicarlos con reglas sencillas? **Árbol de decisión**; si lo que se predice es un número, como la nota, **árbol de regresión**.
> - ¿Una palabra es más frecuente en un corpus que en otro? **Log-likelihood**, para saber si la diferencia es fiable, y **log-ratio**, para saber cuánto difiere.
> - ¿Cuánto más frecuente? **Log-ratio** o **Simple Maths**.
> - ¿Qué factores predicen una variable numérica, como la nota? **Regresión lineal**.
> - ¿Qué factores predicen un sí o no, como acertar el subjuntivo, y cuánto pesa cada uno? **Regresión logística**.
>
> Para las pruebas de comparación de grupos (t, ANOVA, chi cuadrado…), el enlace lleva al árbol interactivo de la sesión 3.

7.2 Cómo informar de un resultado

> Los aprendientes que habían realizado una estancia en un país hispanohablante obtuvieron una nota media más alta (M = 71,1; DE = 13,0) que quienes no la habían realizado (M = 65,9; DE = 12,4), t(207,3) = 3,08, p = 0,002, d = 0,41.

- **Descriptivos**: M y DE de cada grupo
- **Estadístico**: t, F, χ^2… con sus gl

- **p**: Exacto, con 3 decimales
- **Efecto**: d, r, V, η^2, LR…

En pocas palabras

Este párrafo es un modelo de cómo comunicar el resultado de una prueba t en un artículo, siguiendo las normas habituales (APA).

Un buen informe incluye cuatro elementos, recogidos en las tarjetas:

- **Descriptivos**: la media (*M*) y la desviación típica (*DE*) de cada grupo.
- **Estadístico**: el valor de la prueba (t, F, χ^2…) con sus grados de libertad entre paréntesis.
- **Valor p**: el valor exacto, con tres decimales, en lugar de escribir solo «p < 0,05».
- **Tamaño del efecto**: aquí, la *d* de Cohen.

Con estos elementos, cualquier lector puede valorar no solo si el efecto existe, sino también si es relevante.

7.3 Errores frecuentes

- **«Es significativo, así que es importante»** → Con muestras grandes todo es significativo. Informa siempre del **tamaño del efecto**.
- **«He probado veinte pruebas y una ha salido»** → Eso es **p-hacking**: corrige por comparaciones múltiples o preregistra.
- **«Tengo varios textos por aprendiente y los trato como independientes»** → No lo son: usa **modelos mixtos** (`lme4`).
- **«Mi árbol acierta un 90 %»** → ¿Con qué datos? Mide el acierto con **datos de prueba** o validación cruzada.

En pocas palabras

Cuatro errores habituales al usar la estadística en investigación, con su solución. Al pulsar cada tarjeta se gira.

- **Confundir significativo con importante**: con muestras grandes, diferencias minúsculas resultan significativas. Hay que informar siempre del tamaño del efecto.
- **Hacer muchas pruebas y quedarse con la que sale** (*p-hacking*): aumenta enormemente las falsas alarmas. Hay que corregir por comparaciones múltiples o decidir los análisis antes de ver los datos.
- **Tratar como independientes datos que no lo son**: varios textos de una misma persona se parecen entre sí. Para estos casos existen los **modelos mixtos**.
- **Medir el acierto con los datos de entrenamiento**: hay que evaluar con datos nuevos o con validación cruzada.

7.4 Lo que hemos recorrido

- **Describir**: media, mediana, DE, cuartiles, group_by()
- **Visualizar**: ggplot2, ggplotly, motion charts, mapas de calor
- **Inferir**: IC, p, potencia, t, Wilcoxon, r
- **Comparar**: χ^2, residuos, ANOVA, Tukey, interacción
- **Clasificar**: rpart, Gini, poda, matriz de confusión
- **Plus: modelar**: LL, log-ratio, lm, glm

7.5 Recursos

- **Fundamentos de estadística con R para lingüistas** (Cabedo Nebot, 2021, Tirant Lo Blanch)

- **R for Data Science** (2.ª ed.): https://r4ds.hadley.nz
- **Statistics for Linguists: An Introduction Using R** (Winter, 2019)
- **Statistics for Linguistics with R** (Gries, 2021)
- **Log Ratio: an informal introduction** (Hardie, 2014) · **Simple maths for keywords** (Kilgarriff, 2009)
- **LADAL**, Language Technology and Data Analysis Laboratory: https://ladal.edu.au
- **rpart.plot**: http://www.milbo.org/rpart-plot/prp.pdf · **plotly para R**: https://plotly-r.com